

CAD project research:

QUESTIONS:

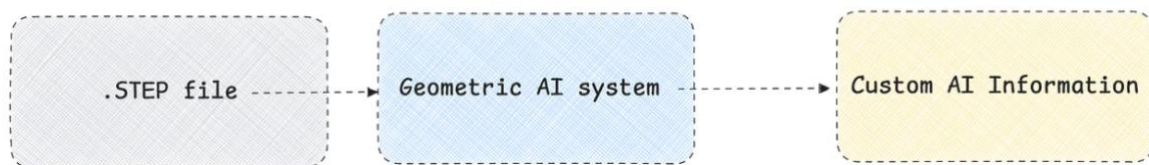
- Do you want a project where labelers manually annotate CAD files and manually convert them to JSON?
- Will the data be provided by clients or created by labelers?
- AI-powered labelers already exist, is the aim to create one?

CAD data annotation is the process of labeling, tagging and structuring 2D drawings or 3D models to train machine learning models for engineering, architecture and manufacturing applications.

<https://medium.com/@elisowski/can-ai-make-cad-analysis-smarter-and-faster-for-engineers-63537f001a6d>

Working with CAD files is challenging. Massive assemblies, countless layers and confusing annotations make analysis time-consuming and error-prone. Small mistakes such as a misnamed layer or misplaced part can lead to costly redesigns and production delays. CAD files (STEP or DGN) weren't built for easy machine reading. But with AI-driven parsing, we can convert them into structured formats like JSON. Assemblies, parts and features become data points AI can work with.

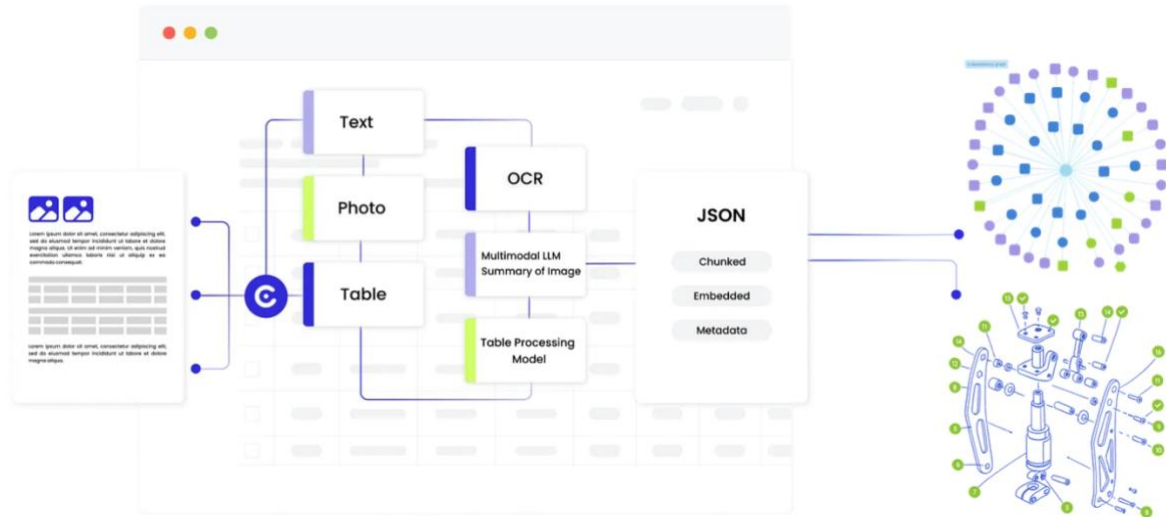
- * New approaches use geometric deep learning to process STEP files directly and there is no need to convert to meshes or point clouds first.



Object hierarchy and spatial understanding

- One of the most valuable things AI brings to CAD is the ability to map out object hierarchies. Assemblies, components and layers are no longer just loosely connected items; they're structured into parent-child relationships.
- On top of that, AI can extract spatial positioning: coordinates, rotations, bounding boxes and placement logic. That means an automated system can understand not just what a part is, but where it fits and how it relates to everything else.

ContextClue: Turning CAD into intelligent engineering assets



- **AI-Driven Parsing:** Reads CAD files like STEP or DGN and converts them into structured, machine-readable JSON.
- **Hierarchy & Spatial Mapping:** Organizes assemblies into parent–child structures and extracts placement logic, positions, and bounding boxes.
- **Vision Analysis:** Captures annotations, labels, and instructions directly from CAD models.
- **Compliance Checks:** Flags naming errors, detects misplaced components, and validates spatial constraints.
- **Metadata Matching:** Cross-references BOMs, compliance documents, and standards to ensure accuracy.

<https://arxiv.org/html/2504.13580v2>

Unlike images, CAD files contain geometric + parametric + feature data so labeling can happen at different levels:

1) Geometry-level labeling:

Label:

- Faces (planar, cylindrical, conical, etc.)
- Edges (sharp, fillet, chamfer)
- Vertices
- Solid bodies

Used for:

- Feature recognition
- Automated machining
- AI-based geometry analysis

2) Feature-level labeling

Label:

- Holes (through, blind, tapped)
- Fillets
- Chamfers
- Slots
- Bosses
- Pockets
- Threads

Used for:

- Manufacturing automation
- Cost estimation
- CAM automation
- ML training for feature detection

3) Functional labeling:

Label:

- Mounting surface
- Load-bearing surface
- Heat dissipation surface
- Sealing face
- Interface face

Used for:

- Simulation AI
- Assembly automation
- Engineering knowledge extraction

4) Assembly-level labeling:

Label:

- Part categories
- Fasteners
- Bearings
- Shafts

- Structural parts
- Purchased components

Used for:

- BOM automation
- Procurement AI

If your goal is AI training, you typically:

Convert CAD → Graph

Each:

- Face = node
- Edge adjacency = connection
- Face type = feature
- Area / curvature = attributes

Or Convert CAD → Voxel / Point Cloud

Then label:

- Regions
- Feature volumes
- Surface clusters

Popular datasets:

- ABC Dataset (Autodesk)
- Fusion 360 Gallery Dataset
- MFCAD

Converting CAD to a Graph

- When you want to use Graph Neural Networks (GNN)
- Feature recognition, manufacturability prediction, cost estimation, part classification, surface function prediction
- CAD is already a topological structure:
- Faces are connected to edges, edges connect to vertices, faces share boundaries, geometry is relational!!!
- THIS IS NATURALLY A GRAPH

Typical Graph Representation

Node = Face

Each node contains features like:

- Surface type (plane, cylinder, cone, sphere)
- Area
- Curvature
- Normal vector
- Bounding box

Edge = Adjacency between faces

Edge features:

- Shared edge length
- Angle between faces
- Convex / concave relation

When do you convert to JSON?

- JSON is not a geometry format.
- When you need to store labels or annotations

```
{
  "file": "part_001.step",
  "faces": {
    "face_12": "through_hole",
    "face_15": "pocket",
    "face_22": "mounting_surface"
  }
}
```

- You need to store graph data for ML

```
{
  "nodes": [
    {"id": 0, "type": "cylinder", "area": 12.4},
    {"id": 1, "type": "plane", "area": 55.2}
  ],
  "edges": [
    {"source": 0, "target": 1, "angle": 90}
  ]
}
```

Then you load this into: PyTorch Geometric, DGL, TensorFlow GNN

Goal	Convert to Graph?	Convert to JSON?
Train GNN	✓ Yes	✓ Yes (for storage)
Train 3D CNN	✗ No	Maybe
Just label faces	✗ No	✓ Yes
Feature recognition ML	✓ Yes	✓ Yes
Web-based viewer	✗ No	✓ Yes
Manufacturing automation	Sometimes	Often

Real-world ML pipeline example

STEP → B-Rep → Graph → JSON → PyTorch → Train Model

Where:

1. STEP file parsed with OpenCascade
2. Extract faces + adjacency
3. Build graph structure
4. Save as JSON dataset
5. Load into GNN

Important clarification

Graph = mathematical structure for ML

JSON = storage format

YOU OFTEN USE BOTH TOGETHER.

The AI model is trained on tensors (numerical arrays), the B-Rep graph is the source structure. JSON is just a storage/transport format.

Practical example (Feature recognition)

If you're training a model to detect holes:

Your label is per-face:

Face 0 → hole

Face 1 → planar surface

Face 2 → fillet

Your training data becomes:

X_node: geometry features

edge_index: adjacency

y: label per face

That's what the model sees.

YOU DO NOT TRAIN ON B-REP FILES OR JSON FILES, you train on numerical graph tensors derived from the B-Rep.

Academic papers:

GRAPH REPRESENTATION OF 3D CAD MODELS FOR MACHINING FEATURE

WeiJuan Cao¹

RECOGNITION WITH DEEP LEARNING Trevor Robinson, Yang Hua,

In this paper, the application of deep learning methods to CNN the task of machining feature recognition in CAD models is studied. Four contributions are made:

- 1. An automatic method to generate large datasets of 3D Network CAD models is proposed, where each model contains multiple machining features with face labels.*
- 2. A concise and informative graph representation for 3D CAD models is presented. This is shown to be applicable to graph neural networks.*
- 3. The graph representation is compared with voxels on their performance of training deep neural networks to segment 3D CAD models.*
- 4. Experiments are also conducted to evaluate the effectiveness of graph-based deep learning for interacting feature recognition.*

Results show that the proposed graph representation is a more efficient representation of 3D CAD models than voxels for deep learning. It is also shown that graph neural networks can be used to recognize individual features on the model and also identify complex interacting features.

- In industry, products are designed using CAD tools, analysed using CAE tools and the manufacturing process is planned using CAM tools. In each different tool, the product is represented in different forms. Specifically, the shape of the final component is usually defined in a CAD tool.
- B-Rep is the de facto industry standard for how they represent the shape, defining only its boundary.
- CAE tools can use the finite element method to predict physical phenomena on meshed models, where the elements fill the entire 3D domain, but the actual shape of the product is approximated to a greater or lesser extent.
- Traditional CAM tools simulate how to cut materials from a raw stock following specified paths to get the final shape.

Considering an engineering component, its shape can be thought of as consisting of engineering features (e.g. holes, bosses and chamfers). The ability to identify engineering features would allow many CAE/CAM process to be automated.

HOWEVER engineering features are usually not explicitly defined in CAD models as they are often not equitable with design features used when creating the CAD model.

Traditionally machining features are mostly created using subtractive processes, but the same shapes are easily modelled in the CAD system using additive features. Furthermore, the CAD model construction history is often proprietary and is not available after the model has been transferred into CAE/CAM tools using common formats such as STEP. Therefore, it would be extremely valuable in engineering processes to be able to automatically identify machining features from CAD shapes.

Machining feature recognition can be viewed as a semantic segmentation problem, where faces of a 3D model are divided into groups and each group of faces represents a feature. The

goal of the deep learning algorithm is then to assign the correct feature label to each face in the model.

To apply deep learning methods to CAD-based tasks, two obstacles need to be overcome:

- The success of deep learning depends on having access to a sufficiently large set of labelled data, where the labels can be used to identify the types of features in the CAD models that are of interest.
- Required dataset size can be in the range of thousands to millions. Sourcing these is not a trivial task
- Datapoints as input the model, output as answers using code to questions about models: how many teeth? input includes cad file, output is a numerical value or a text, concise final answer, example tasks
- Generative task, way to grade CAD produced files by the model, given 2 cad files, reliably make an edit and produce a reward penalty system based on the output, LLM judge? way to grade the CAD output
- programmatic grading across the board, one script to grade but config script comes with it,
- Example task of GTFA task where a request is inputted and the model churns out an answer which then needs to be verified to make sure it is correct
- Second dataset task would involve CAD file generation through a series of detailed or one long prompt, the challenge lies in grading the AI output whether it is through a LLM judge, or a coded script that it goes through
- Figure out which AIs can open and read CAD files (STEP or else)

Input = CAD file + natural language question

Output = guaranteed true answer about the CAD

This becomes a CAD-grounded question answering system with verifiable outputs.

If truthfulness is required, the system must:

- Ground every answer in deterministic geometry computation
- Cannot rely on LLM's internal reasoning about geometry, it will hallucinate.

The architecture must separate:

- Language understanding
- Geometry reasoning
- Final answer generation

User Question



LLM (intent parsing only)



CAD analysis engine (deterministic tools)



Verified numeric / logical result



LLM formats answer

The LLM does NOT compute geometry. The CAD kernel does.

1) Parse CAD file

- OpenCascade, Parasolid, STEP parser, CAD kernel API
- Extract:
- B-Rep topology, Faces, edges, vertices, Surface equations, Volume, Adjacency graph

You Need Three Layers

Geometry Truth Layer

CAD kernel / B-Rep

Query Layer

Structured geometry queries

Language Layer

LLM that maps English → queries

Example Query Types You Can Support

- Count features
- Detect surface types
- Check adjacency
- Measure distance
- Compute area
- Compute bounding box
- Check symmetry
- Detect pattern repetition
- Identify feature types (if deterministic)

Natural language is vague.

“Does this part have a hole?”

What qualifies as a hole? Cylindrical void, blind, through, threaded
WE NEED A FORMAL ONTOLOGY

To Guarantee Truth

You must:

1. Never let the LLM directly reason about geometry.
2. Always compute from the CAD kernel.
3. Log the computation path.
4. Optionally return trace evidence.

Example:

Answer: 6 cylindrical faces

Verified by: B-Rep face classification

Multi-modal evaluation pipeline:

- 1) Render CAD to Images
 - Generate 4-6 standard views (front, back, top, bottom, isometric angles)
 - Use consistent lighting and camera positions
 - Save as high-res PNGs or JPEGs
- 2) Extract structured data
 - Bounding box dimensions (height, width, depth)
 - Vertex/face count
 - Material assignments
 - Component hierarchy
 - Key measurements (handle width for a mug, opening diameter, etc...)
- 3) LLM judge evaluation
 - Feed images + structured data + original prompt to a vision capable LLM
 - Get scored responses on specific criteria

Evaluation Rubric

Score each response on:

Functional Correctness (0-10)

- Does it match the basic object type requested?
- Are essential features present? (e.g., mug has handle, opening, base)

Prompt Adherence (0-10)

- Captures specific details in prompt (color, style, size descriptors)
- Implements requested modifications

Geometric Quality (0-10)

- Proportions look reasonable
- No obvious artifacts or errors
- Topology makes sense

Usability/Realism (0-10)

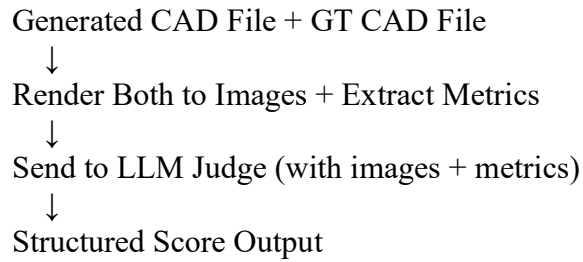
- Would this work as a real object?
- Are dimensions practical?
- Structurally sound?

Quality Control Layers

1. **Automated filters first** (fast, cheap)
 - File validity checks
 - Dimension sanity checks (mug isn't 10 meters tall)
 - Minimum face count thresholds
 - Render success verification
2. **LLM batch evaluation** (main system)
 - Process in batches of 10-50
 - Use consistent temperature (0.3-0.5 for consistency)
 - Track inter-rater reliability by re-scoring samples
3. **Human spot-checks** (calibration)
 - Manually review 5-10% of outputs
 - Compare to LLM scores
 - Adjust rubric/prompts if systematic disagreements

Cost Optimization

- Use cheaper models (Claude Sonnet vs Opus) for initial filtering
- Only send high-quality renders, not raw CAD files
- Batch API calls
- Cache evaluations for identical outputs



Recommended approach:

- Hybrid approach:
- Fast geometric filter eliminates obvious failures (70% of cases)
- LLM judge handles edge cases (30% of cases)
- Cost effective – reduces LLM calls by 70%
- Best accuracy – combines quantitative + qualitative

<https://arxiv.org/pdf/2409.11239>

LLM-as-a-judge and reward models are widely used alternatives of MCQ or human annotators for LLM evaluation. Their efficacy shines in evaluating long-form responses, serving a critical role as evaluators of leaderboards and as proxies to align LLMs via reinforcement learning.

Despite their popularity, their effectiveness in diverse contexts, such as non-english prompts, factual verification or challenging questions remains unexplored.

As scalable alternatives to costly human annotation, they contribute to the development of user-friendly chat systems like ChatGPT and Claude.

LLM-as-a-judge applications typically include two evaluation methods: pointwise and pairwise. Pointwise evaluation involves assigning integer scores to individual responses, whereas pairwise evaluation contrasts two responses to determine a preference.

Using an LLMJ for CAD generation is doable but structure is needed. CAD is geometric and constraint-heavy.

- 1) Define evaluation dimensions such as:

A) Geometric correctness:

- Does the model satisfy dimensional constraints?
- Are specified measurements correct?
- Are tolerances respected?
- Are features positioned correctly?

B) Topological validity:

- Is the solid manifold?
- Any self-intersections?
- Broken sketches?
- Missing faces?

C) Parametric consistency:

- Are constraints properly defined?
- Is the model editable without breaking?
- Does feature history regenerate cleanly?

D) Functional intent:

- Does it match the design spec?
- Do holes align?
- Does assembly fit?

E) Code quality (if generating CAD scripts):

- Clean feature tree?
- Proper naming?
- No redundant features?
- Efficient modeling sequence?

The judge LLM cannot reason over raw STEP geometry directly. Preprocessing is required.

Option 1: Text-based CAD (best for LLM judge)

If the CAD AI outputs:

- OpenSCAD
- CadQuery
- Feature history scripts
- Parametric Python
- Fusion API scripts

Then I can feed the code directly to the LLM judge.

Option 2: Structured geometry summary

If the model outputs STEP/STL, then preprocess into:

- Bounding box
- Volume
- Surface area
- Feature count
- Hole diameters + positions
- Feature tree reconstruction
- Constraint list

- Topology validity checks

Target spec:

- Base plate 100x50x10mm
- 4 holes diameter 5mm at corners offset 10mm

Generated model analysis:

- Bounding box: 100.2x49.9x10.1mm
- Volume: 49950 mm³
- Holes detected: 4
- Hole diameters: 5.02, 5.01, 4.98, 5.03
- Hole offsets: 9.8–10.2mm
- Manifold: Yes

Option 3: Multimodal (Images + data)

Render:

- Orthographic projections
- Section views
- Isometric view

Feed those into a multimodal LLM judge + structured measurements. This is good for visual sanity checks.

Judge prompt architecture:

Use a structured rubric with explicit scoring weights.

You are a mechanical CAD evaluator.

Score the model on:

1. Dimensional accuracy (0–10)
2. Feature completeness (0–10)
3. Constraint correctness (0–10)
4. Manufacturability (0–10)

Provide:

- Score per category
- Justification
- Final weighted score

Force the judge to:

- Think step by step
- Extract evidence
- Cite mismatches explicitly

Add determinism and stability

LLM judges can drift. To stabilize:

- Use temperature = 0
- Force JSON output
- Use chain-of-thought internally but not expose reasoning
- Run multiple passes and average

Better:

Use pairwise comparison instead of absolute scoring

Use pairwise comparison (stronger)

- Instead of 'score this CAD model'
- Do 'Between model A and model B, which better satisfies the specification?'
- LLMs are much more reliable at comparison than absolute scoring.

Hybrid system:

- Don't rely purely on LLM reasoning

Combine:

- 1) Deterministic checks (programmatic)
 - Dimensional tolerances
 - Constraint completeness
 - Manifold check
 - Regeneration success
- 2) LLM checks
 - Design intent
 - Feature modeling quality
 - Redundancy
 - Readability
 - Human-likeness of feature tree

Final Score = 70% deterministic metrics + 30% LLM qualitative judgment

What does each option measure?

Option 1: Generation evaluation

Tests:

- Geometry synthesis ability
- Parametric correctness
- Constraint reasoning
- Spec adherence
- Manufacturability
- Robustness to ambiguity

This measures the core CAD capability

Option 2: CAD understanding/reasoning

Tests:

- 3D perception
- Topological reasoning
- Counting
- Semantic understanding

This measures CAD interpretation ability

Option 2 is much easier (question answering over CAD)

- We already know the correct answers
- We can auto-grade deterministically
- No subjective scoring
- No geometric comparison pipeline needed

We just:

- Prepare ground truth CAD files

- Precompute feature counts, topology, etc.
- Ask model questions
- Compare answers

This is clean and scalable.

Option 1 is much harder

We must:

- Parse arbitrary CAD output
- Check geometry
- Measure dimensional accuracy
- Detect features
- Handle ambiguity in “make a chair”

We need:

- STEP/STL parser
- Geometry analysis engine
- Tolerance handling
- Possibly an LLM judge on top

Step 1 — Controlled Prompt Suite

Instead of vague prompts like “make a chair”, define:

Level 1 — Primitive shapes

- “Create a 100x50x10mm plate with 4 5mm holes.”

Level 2 — Multi-feature parts

- “Create a bracket with two perpendicular flanges...”

Level 3 — Parametric parts

- “Create a configurable table with adjustable height.”

Level 4 — Assemblies

Step 2 — Automatic Geometry Analyzer

When model returns CAD:

You compute:

- Bounding box
- Volume
- Feature detection (holes, fillets)
- Dimensional tolerances
- Constraint regeneration success

No LLM needed here.

Step 3 — Scoring Engine

Dimensional accuracy (0–40%)

Feature completeness (0–30%)

Topological validity (0–15%)

Parametric robustness (0–15%)

Fully deterministic if possible.

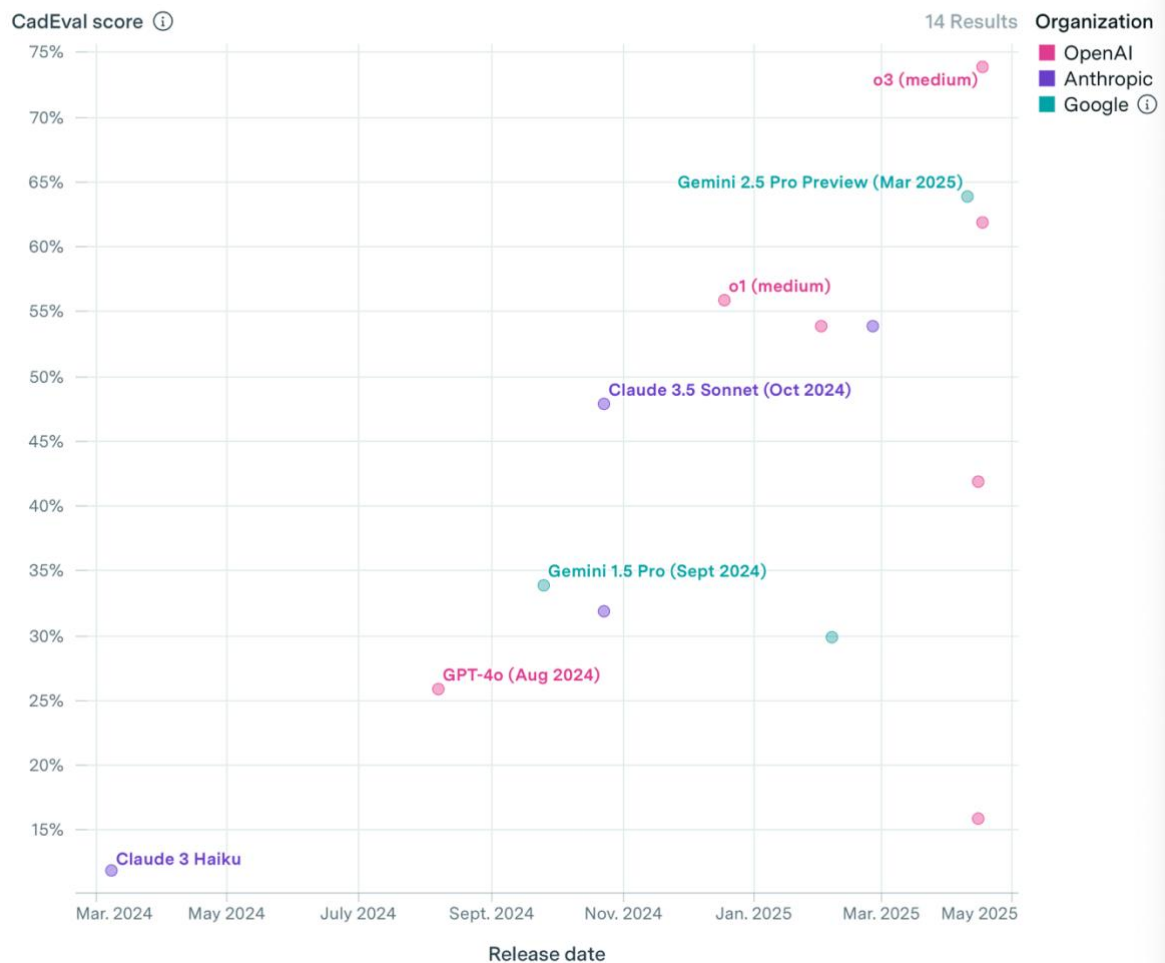
LLM judge only for:

- Modeling elegance
- Feature redundancy
- Human-likeness

CadEval

- A text-to-CAD benchmark evaluating whether models can generate valid parametric 3D designs that meet geometric and rendering checks

CadEval



CadEval measures end-to-end CAD synthesis: given a textual prompt, a model must produce a CAD program (e.g., OpenSCAD-style) that renders to an object matching the specification. The benchmark reports both success rates and geometric fidelity via metrics like rendering success, **Chamfer/Hausdorff** distances and volume/consistency checks. The tasks stress long-horizon reasoning, tool-augmented generation and an understanding of geometry and constraints. Strong results require not only producing syntactically valid code but also achieving shape similarity that satisfies strict quantitative thresholds.

CADEVAL SUMMARY

- Framework for evaluating the capability of LLMs to generate 3D models (specifically OpenSCAD code or direct STL files) from textual descriptions.

Goal: Assess the reliability and geometric accuracy of various LLMs in producing single, simple mechanical parts based on text prompts, including analysis based on task complexity, execution time and estimated cost.

Scope:

- Focus on single-part generation
- Input is primarily text descriptions defined in task files
- Evaluation involves automated generation, rendering (where applicable), geometric checks against reference models, and visualization of results via a dashboard

- Supports standard LLMs (via API)

Workflow overview:

- 1) Configuration: Defines models to test, API keys, prompt templates, directories, geometry check thresholds, cost parameters (token/time-based) and evaluation parameters (e.g. number of replicates per task).
- 2) Task definition: each task YAML file specifies task_id, description, reference_stl, manual_operations (optional, indicates the complexity of the task), requirements (optional, can include bounding_box or topological_requirements)
- 3) Execution: main script that orchestrates the evaluation
 - Loads configuration and selected tasks
 - Iterates through specified models prompts and the requested number of replicates
 - for standard LLMs: calls scripts/generate_scad.py to interact with LLM APIs, saving generated .scad files to results/{run_id}/scad/. Captures LLM API call duration and token counts
 - Rendering: for successfully generated .scad files, calls OpenSCAD via scripts/render_scad.py to render them into .stl files in results/{run_id}/stl/. captures rendering duration
 - Geometry checks: for all successfully generated/rendered .stl files, calls scripts/geometry_check.py to perform checks against the reference STL.
 - Aggregates raw results (generation status, render status, check results, metrics, timings, token counts) for each attempt into results/{run_id}/results_{run_id}/run_{run_id}.log
- 4) Post-processing (scripts/process_results.py):
 - reads the raw results from results_{run_id}.json file from a specified run
 - Calculates summary statistics (meta_statistics grouped by model/prompt, task_statistics grouped by task), including average time and estimated cost metrics.
 - Calculates complexity analysis based on manual_operations from task YAMLs
 - Formats the processed data and statistics into dashboard/dashboard_data.json
- 5) Dashboard (dashboard/):
 - a web-based dashboard reads dashboard_data.json
 - displays interactive charts (using chart.js) and tables visualizing model performance across various metrics, including average time and cost.

CadEval/

```

├── config.yaml          # Main configuration file
├── environment.yml      # Conda environment definition (or requirements.txt)
├── .env                 # API Keys and environment variables (add to .gitignore!)
├── .gitignore           # Specifies intentionally untracked files
├── README.md           # This file
├── tasks/               # Task definition YAML files
│   └── *.yaml
├── reference/           # Reference/ground truth STL models
│   └── *.stl
├── scripts/             # Core Python scripts
│   ├── run_evaluation.py # Main orchestration script
│   ├── process_results.py # Post-processing for dashboard
│   └── geometry_check.py  # Performs geometric comparisons

```

```

|   |   |— render_scad.py      # Handles OpenSCAD rendering
|   |   |— generate_scad.py   # Handles LLM API calls for SCAD generation
|   |   |— task_loader.py     # Loads task definitions
|   |   |— config_loader.py   # Loads config.yaml
|   |   |— logger_setup.py    # Configures logging
|   |— results/               # Output directory for evaluation runs
|   |   |— {run_id}/          # Each run gets its own directory
|   |   |   |— results_{run_id}.json # Raw detailed results for the run, including status,
metrics, timings, and token counts.
|   |   |   |— run_{run_id}.log    # Log file for the run
|   |   |   |— scad/              # Generated .scad files (for non-Zoo models)
|   |   |       |— *.scad
|   |   |   |— stl/              # Generated/rendered .stl files
|   |   |       |— *.stl
|   |— dashboard/            # Web dashboard files
|   |   |— dashboard.html
|   |   |— dashboard.js
|   |   |— dashboard.css
|   |   |— dashboard_data.json # Processed data consumed by the dashboard, including
summary statistics for metrics, time, and cost.
|   |— # Other files/dirs (.git, .pytest_cache, tests/, etc.)

```

Things to look at for Friday:

- CadEval, download it on conda and try it out myself
- Read the papers:
- Human-in-the-loop: quantitative evaluation of 3d models generation
- A research paper that proposes a quantitative evaluation methodology for 3D models generated by LLMs, including metrics like:
 - volumetric accuracy, surface alignment, dimensional fidelity and topology complexity
- CAD-Judge: morphological grading and verification for text-to-cad
- A system focused on grading and validating AI-generated CAD models from text prompts, aiming at morphological correctness and verification.
- This is especially relevant if I want to grade and verify client model outputs automatically.
- EvoCAD: evolutionary CAD Code generation with VLMs
- In addition to presenting a generation algorithm, this paper introduces evaluation metrics that include geometric and also topology-based scores.
- A survey on deep learning in 3D CAD reconstruction

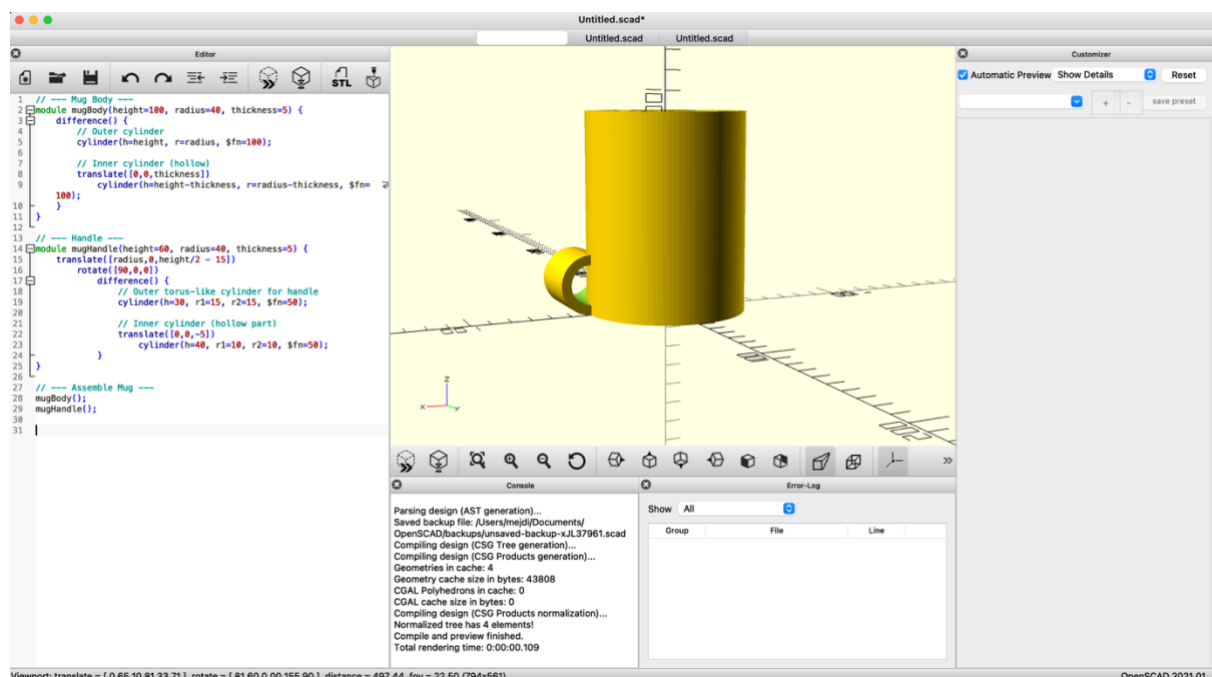
- A comprehensive survey of deep learning methods applied to 3D CAD reconstruction – including evaluation, datasets and challenges. This paper has sections on evaluation metrics, CAD data formats and review of reconstruction approaches – useful for positioning my work in the literature.
- Geometric deep learning for CAD – survey
- A broader survey on geometric deep learning applied to CAD:
- CAD data formats (B-Rep)
- existing generation and reconstruction approaches
- evaluation challenges
- DesignQA: Engineering understanding benchmark
- A benchmark by MIT and Autodesk focused on MLLM understanding of engineering requirements, multimodal CAD images, and design reasoning.
- It is not pure CAD generation but it's directly applicable for language and reasoning over engineering designs.
- Classic CAD LLM work (evaluation sections worth reading):
- A NeurIPS workshop paper on large models for CAD generation. It includes CAD-specific evaluation metrics like:
 - entity accuracy
 - sketch accuracy
 - CAD F1

CAD EVAL:

- Using OpenSCAD

OpenSCAD is a code-to-CAD open source , free to use software

The prompt “make me a simple mug” was put into ChatGPT and it generated the following code which generated this mug. The mug handle is out of place and too small.



CadEval judges how good the AI-generated CAD model is by running geometric and structural tests against the reference STL.

1) Structural / validity checks

- Watertight (manifold check):
 - Question: Is the mesh a closed, valid solid? No holes in surface, no non-manifold edges, no open boundaries
- Single connected component
 - Question: Is the part one solid object? Example failure: mug body and handle floating separately, detached features

2) Bounding box accuracy

- Metric: axis-aligned bounding box comparison (after ICP alignment)
- What it checks: overall width, depth, height
- It compares AI model dimensions vs reference dimensions
- Example failure: mug is 80mm tall instead of 100mm, handle extends too far outward

3) Volume accuracy:

- Metric: percentage difference in volume, controlled by “volume_threshold_percent: 2.0”
- It checks: $\text{ai_volume} - \text{reference_volume} / \text{reference_volume}$
- this catches: wrong wall thickness, missing hollow interior, extra geometry, wrong dimensions

4) Chamfer distance (surface similarity):

- Metric: most important geometric similarity metric
 - It measures: how far apart are the two surfaces on average?
- Process: convert both models to point clouds, align with ICP?, compute average nearest-neighbour distance
- Controlled by: chamfer_threshold_mm: 1.0
- Low = very similar, high = different geometry
- This detects: wrong curvature, wrong handle shape, subtle deviations

5) Hausdorff distance (95th percentile)

<https://doi.org/10.1016/j.patcog.2021.107857>

The Hausdorff distance (HD) is an important algorithm to measure the similarity of two geometric objects because it is defined as max-min distance between two point sets and it has the ability to determine the degree of resemblance between two point sets without defining the correlation between points.

- This measures: the worst-case surface deviation (excluding extreme outliers)
- Controlled by: hausdorff_threshold_mm: 1.0
- This catches: one part sticking out too far, a misplaced feature, an incorrect edge location
- Hausdorff is stricter than Chamfer

6) ICP Fitness score

- ICP = Iterative closest point alignment
- This measures: how well the two shapes align geometrically
- higher fitness = better alignment
- If ICP fails: the shapes may be completely wrong, geometry comparison may not be meaningful

- 7) Time metrics:
 - not geometric but tracked: LLM generation time, OpenSCAD rendering time, total time
 - Used for performance benchmarking
- 8) Cost metrics:
 - based on: input tokens, output tokens, model pricing
 - Used for: efficiency analysis, cost-performance tradeoffs

Think of it like:

- Bounding box → overall envelope check
- Volume → material amount check
- Chamfer → average surface deviation
- Hausdorff → worst feature deviation
- Watertight → manufacturability check
- Single component → assembly correctness

6) Metrics Used in Text-to-3D / Text-to-CAD Generation Research

Many recent papers on *text-to-3D/point cloud/voxel CAD generation* propose hybrid evaluation schemes such as:

✓ Geometric metric suite

- Chamfer Distance / Hausdorff Distance
- Volume difference
- Surface error

✓ Functional constraint checks

- Are constraints satisfied?
- Are critical dimensions correct within tolerance?

✓ Semantic correctness

- Does the model contain the requested features?

You can borrow these directly into your scoring grids.

7) Benchmark Studies on Geometry & Reconstruction

Examples include:

- **“Neural Geometry Processing” evaluation tasks**
- **“3D Reconstruction Metrics” compendia**
- **Topology correctness evaluations**

While not CAD generation per se, they often include:

- Topology integrity checks
- Mesh manifoldness
- Geometric fidelity scoring

These directly map to constraints that matter in parametric CAD evaluation.

8) Hybrid Evaluation Frameworks

Some recent projects combine both:

- **Programmatic geometry checking**
- **Structural evaluation**
- **Language model judgments**

Examples include:

- Pairwise comparison metrics
- Regression analysis between ground truth and generated CAD
- Feature entity alignment scoring

While not publicly polished benchmarks, they appear in research on *ML for CAD understanding and generation*.

9) Papers on “Black-Box Evaluation of Generative Models”

While general, this class of research defines *good practices* for benchmarking:

- **Pairwise comparison over absolute scoring**
- **Adversarial prompts**
- **Consistency and stability metrics**
- **Human vs. model ranking alignment**

You can adapt these high-level principles to CAD AI evaluation.

Summary — Where to Look

Here’s how the existing landscape fits your needs:

Benchmark / Source	Focus	Most Applicable To
CadEval	Text-to-CAD scoring	Primary generation evaluation
CAD-LLM metrics	Feature/Sketch/CAD-F1	Structured CAD generation
DesignQA	Engineering + reasoning	CAD understanding (Track B)
DeepCAD / ShapeNet / PartNet	Geometric fidelity	Mesh/geometry evaluation
3D reconstruction papers	Distance & topological scores	Geometry + topology checks
Hybrid evaluation frameworks	Human-aligned scoring	LLM judge design practices

Why This Matters

For a robust CAD AI evaluation system, you generally need **three kinds of scoring sources**:

1. **Deterministic geometric measures**
(distances, tolerances, topological checks)
2. **Structured feature-based scoring**
(entity presence, constraint correctness)
3. **Qualitative judgment**
(via LLM judges or human proxies)

CadEval primarily covers (1) and (2), but the other papers and benchmarks fill in gaps — especially around semantic, functional, and reasoning evaluations.

How would I go about adding my own metrics and thresholds to the code? Instead of using CadEval, using my own benchmark which combines cadeval with other benchmarks and thresholds, also what about taking screenshots of different angles and views in order to see whether it meets certain visual requirements? how feasible is that?

Build it as a thin web layer on top of CadEval, not a rewrite.

1. Platform Architecture (recommended)

1. Frontend: form + results dashboard.

- Inputs: task_name, prompt, reference_stl, model, replicates, optional custom thresholds.
- Output: pass/fail badge, metric table, plots, run history, STL/SCAD previews.

2. Backend API: orchestrates jobs.

- Creates one-off CadEval task/config.
- Runs generation/render/check pipeline.
- Stores artifacts + metrics.

3. Worker/Runner: isolated process for OpenSCAD + geometry checks.

- Use queue (Redis + RQ/Celery, or BullMQ if Node).

4. Storage:

- Object store (S3/GCS): uploaded reference STL, generated STL/SCAD, PNG renders.
- DB (Postgres): runs, metrics, thresholds, status, timing, cost.

2. Minimal User Flow

1. User submits prompt + reference STL + model + thresholds.
2. Backend creates run folder and dynamic [task.yaml](#).
3. Backend launches CadEval for that single task/model.
4. Backend parses [results {run_id}.json](#).
5. UI shows:
 - SCAD Gen, Render, Watertight, Single Component
 - Chamfer, Hausdorff(95p/99p), BBox, Volume
 - Overall pass/fail (your custom rule)
 - Trend charts and side-by-side STL renders/screenshots.

3. How to Integrate with CadEval cleanly

1. Keep CadEval scripts unchanged as core engine.

2. Add a service wrapper:

- `POST /runs` -> create run
- `GET /runs/:id` -> status + metrics
- `GET /runs/:id/artifacts` -> STL/SCAD/PNGs/logs

3. For each run, generate:

- `<run id>.yaml` (or temp dir)
- `config <run id>.yaml` (inherits base + overrides thresholds/tasks dir/model)

4. Execute:

- `run_evaluation_parallel.py --config ... --tasks ... --models ... --replicates ...`
- `process_results.py <results.json>`

4. Pass/Fail Logic (your benchmark, not raw CadEval only)

Define in config and compute server-side:

• Example:

- hard gates: render success, watertight, single component
- geometric gates: chamfer $\leq X$, haus95 $\leq Y$, volume diff $\leq Z\%$, bbox within tolerance
- optional weighted score:

- $\text{score} = 0.35 \cdot \text{chamfer_norm} + 0.35 \cdot \text{haus_norm} + 0.15 \cdot \text{vol_norm} + 0.15 \cdot \text{bbox_norm}$

• Final verdict:

- PASS if all hard gates pass and score $\geq$ target (or all metric gates pass)

5. Visual Validation (feasible and useful)

Yes, add it as secondary metrics:

1. Render fixed camera PNGs (front/side/top/iso) for generated + reference.
2. Compare with SSIM / silhouette IoU.
3. Add `visual_pass` threshold.
4. Display image diff overlays in UI.
Use this as complement to geometry metrics, not replacement.

6. Security / Reliability (critical)

1. Run OpenSCAD + model outputs in sandboxed containers.
2. Time/memory limits per job.
3. Virus-scan uploads and enforce STL size limits.
4. Strict file path isolation per run.
5. Full audit logs (prompt, model, config hash, code hash, timestamps).

7. Suggested Tech Stack

- Frontend: Next.js + Plotly/Chart.js + [three.js](#) STL viewer
- Backend: FastAPI (Python fits CadEval best)
- Queue: Celery + Redis
- DB: Postgres
- Storage: S3-compatible bucket
- Deployment: Docker + one worker pool

8. MVP in 2 phases

1. Phase 1 (fast): single-run UI + upload + run + metrics table + pass/fail.
2. Phase 2: run history, comparisons across models, plots, visual checks, cost/time analytics.

If you want, I can draft the exact API spec and DB schema next (tables + endpoint payloads) so your team can implement immediately.

Strong proposal: use a **hybrid benchmark** (curated real parts + synthetic canonical parts).

Where to get reference STLs

1. Build your own canonical set first (best for control).
 - Author parametric OpenSCAD references yourself, then export STL.
 - Pros: exact dimensions/ground truth/known constraints.
2. ABC Dataset (large CAD corpus from Onshape; includes STL/STEP).
 - [ABC Dataset](#)
3. Onshape public documents (good source for parametric CAD examples).
 - [Onshape public docs](#)
4. McMaster-Carr CAD models (industrial/mechanical realism; typically STEP, convert to STL).
 - [McMaster CAD models](#)
5. NIST AM Bench artifacts (benchmark-oriented geometry + measurement context; includes STL/STEP in challenge data).
 - [AM Bench home](#)
 - [Direct AM Bench data links](#)
 - [Example page noting STL/STEP CAD geometry](#)

How many STLs you need

1. Pilot: **30-50 tasks** (to validate pipeline and metric stability).
2. First credible benchmark: **120-180 tasks** across complexity buckets.
3. Strong publishable benchmark: **300+ tasks** with balanced categories + 3 replicates/model.

Practical target for your project

- Start with **150 reference parts**:
 - 50 simple primitives/boolean parts
 - 60 medium mechanical parts (holes, bosses, patterns, symmetry)
 - 40 hard parts (thin walls, blends/chamfers, complex handles/cutouts)
- Run **3 replicates per model per task**.

Datasets which could be used for reference STL files:

1) ABC-Dataset:

- Collection of one million CAD models for research of geometric deep learning methods and applications. Each model is a collection of explicitly parametrized curves and surfaces, providing ground truth for differential quantities, patch segmentation, geometric feature detection, and shape reconstruction

2) McMaster-Carr

- Offers CAD for over 700,000 parts
- Paid parts
- Usually for manufacturing purposes

Use a strict intake rubric before adding any library part to your benchmark.

1. License and reuse rights

- Must allow redistribution/use in benchmark.
- Reject if license unclear.

2. Ground-truth quality

- Prefer source CAD (STEP/parametric) plus STL export.
- Units must be known and correct.
- Reject meshes with unknown scale or obvious unit ambiguity.

3. Geometry health

- STL should be watertight/manifold and clean (no self-intersections, duplicate shells, broken normals).
- Reject "dirty" meshes unless your benchmark explicitly tests mesh repair.

4. Taskability from text

- Part must be describable in a prompt with objective constraints (dimensions, counts, symmetry, hole positions).
- Reject parts that require subjective interpretation ("looks ergonomic", "stylish").

5. Measurable constraints	• Must support numeric checks: bbox, volume, feature counts, distances, radii, topology/component count. • Reject parts where success is only visual.
6. Complexity fit	• Tag each part as simple/medium/hard by operation count. • Keep balanced buckets so results are not biased to easy shapes.
7. Coverage value	• Keep only parts that add new geometry patterns (threads, slots, loft-like forms, patterns, thin walls, etc.). • Reject near-duplicates.
8. Prompt fairness	• Prompt should contain enough info to reproduce intended part without hidden assumptions. • Reject "underspecified" tasks that create random failure noise.
9. Reference robustness	• Regenerate STL from source CAD once and verify metrics are stable across re-export. • Reject parts that cause unstable metrics/alignment failures.
10. Pilot pass	• Test each candidate on 2-3 baseline models. • Keep tasks that discriminate model quality; remove tasks that are always trivial or always impossible.

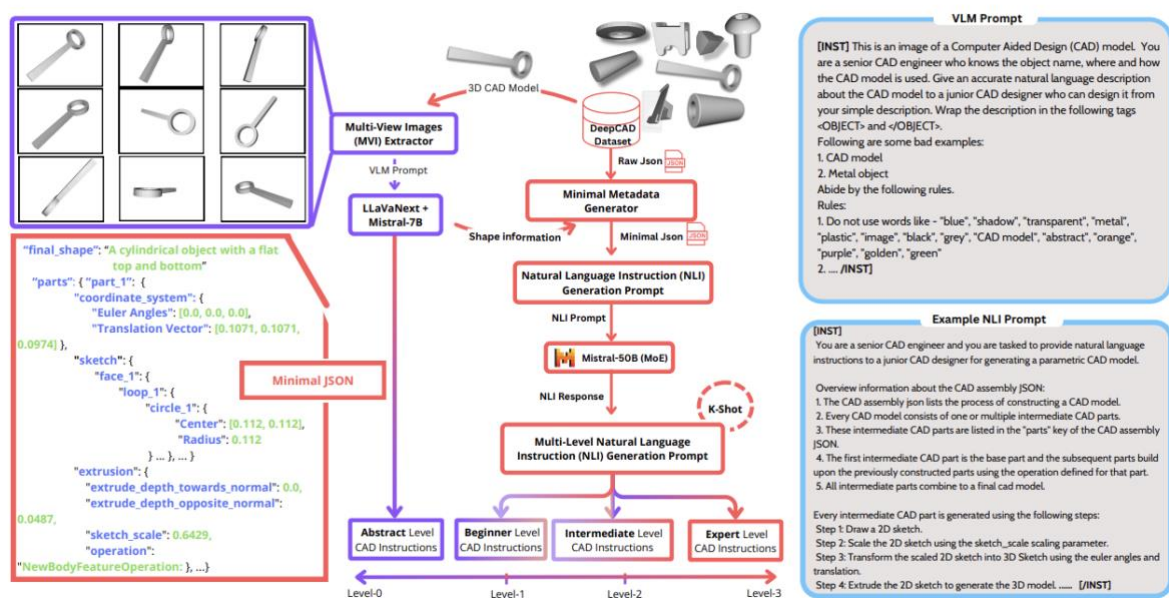
Useful additions (beyond chamfer/hausdorff/volume/bbox/watertight/components):

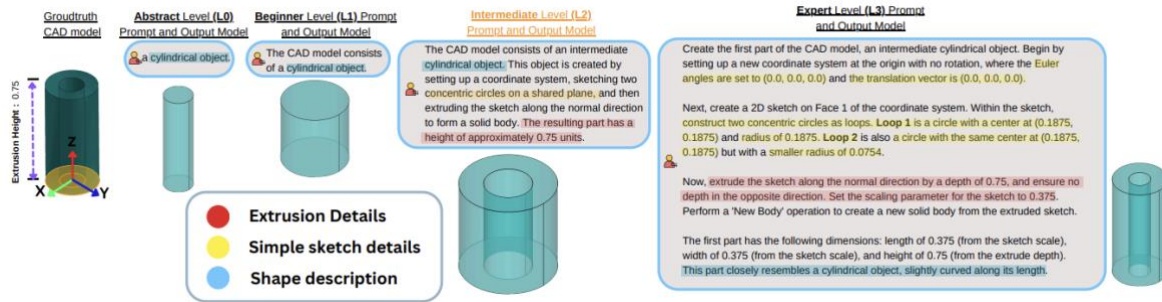
- 1) Surface area error (%):
 - compares total mesh surface area difference
 - catches shape changes that volume may miss
- 2) Normal consistency/orientation score:
 - Compare face normal directions after alignment
 - detects flipped/inverted local geometry
- 3) symmetry score:
 - measure mirror symmetry about expected planes
 - great for mechanical parts
- 4) Curvature distribution distance:
 - compare histogram of mean/gaussian curvature
 - detects missing fillets/chamfers/detail smoothing
- 5) Slice profile IoU (multi-slice):
 - intersect both meshes with several planes and compare 2D contours
 - good for functional cross-section fidelity

https://proceedings.neurips.cc/paper_files/paper/2024/file/0e5b96f97c1813bb75f6c28532c2ecc7-Paper-Conference.pdf

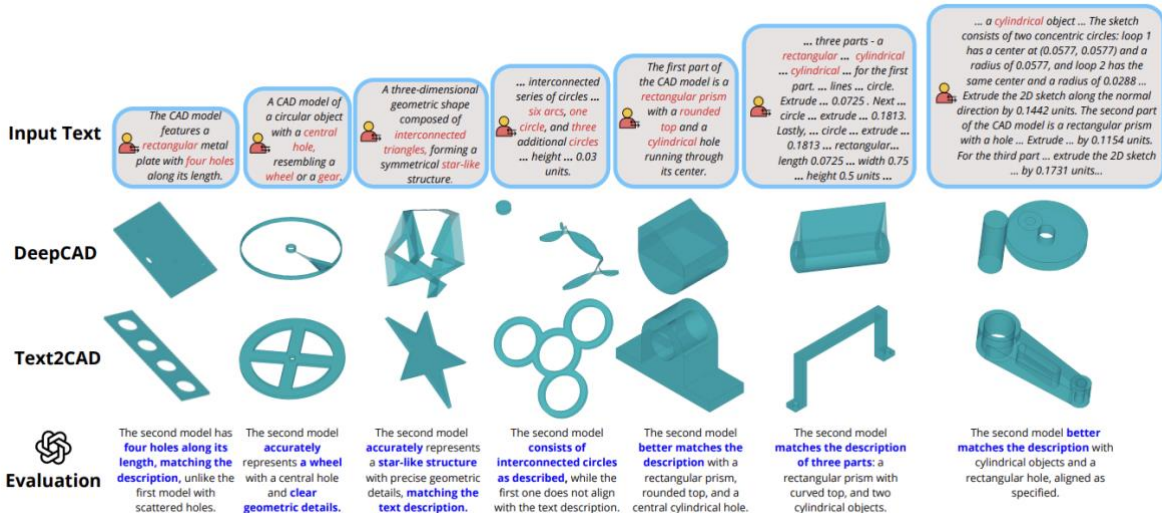
Text2CAD: Generating sequential CAD designs from beginner-to-expert level text prompts

- Text2CAD is the first AI framework for generating text-to-parametric CAD models using designer-friendly instructions for all skill levels.
- They also introduce a data annotation pipeline for generating text prompts based on natural language instructions for the DeepCAD dataset using Mistral and LLaVa-NeXT.
- The dataset contains 170k models and around 660k text annotations, from abstract CAD descriptions (e.g. generate two concentric cylinders) to detailed specifications (e.g. draw two circles with center (x,y) and radius r1,r2 and extrude along the normal by d
- They then evaluate the performance of their model through a mixture of metrics, including visual quality, parametric precision and geometrical accuracy
-





- They used the DeepCAD dataset which contains approximately 150k training CAD sequences and around 8k test and validation sequences in sketch-and-extrude format.
- Following, the sketches and the final CAD models are normalized within a unit bounding box.
- For each sample in the dataset, four design prompts ranging from abstract to expert levels (L0,L1,L2,L3) are generated using their data annotation pipeline resulting in around 600k training samples and around 32k test and validation samples.



EVALUATION

- Their goal was to evaluate how well the generated CAD sequences align with their respective input text prompts. They concentrate on two aspects:
 - 1) Examining the parametric correspondence between the predicted CAD sequences and the input text prompts
 - 2) Conducting visual inspections of the reconstructed 3D CAD models
- Currently, there is no standardized benchmark for text-to-3D models that can be directly applied to this task.
- Existing methods for text-to-3D utilize pre-trained CLIP?? models to measure the alignment between reconstructed 3D models and their text prompts.
- However, CLIP scores may not adequately evaluate the geometric and the parametric aspects of CAD designs.

Therefore, they employ three evaluation metrics to measure the parametric and geometric alignment between the generated CAD designs and the text prompts.

18/02/2026

Project #2: CAD feature recognition

One input type for v1:

- 1) Native CAD (.step, .stp, maybe .dwg)
- 2) 2D technical drawings/PDFs
- 3) Screenshots/renders

Screenshots/2D first would result in faster iteration

Then, creating a task suite is key (ground truth dataset)

Build 50-200 examples with known answers:

- Count gear teeth
- Count holes
- Detect chamfers/fillets
- Measure basic dimensions (if scale exists)
- Identify feature presence (keyway yes or no)

For each sample, store:

- Input file/image
- Prompt
- Correct answer
- Allowed tolerance (e.g. exact integer, vs 0.2mm tolerance)

Then, defining scoring (benchmark) is key and objective metrics should be used:

- Exact match rate
- MAE/MAPE (for numeric dimensions), mean absolute error and mean absolute percentage error
- Pass/fail per task
- Latency + cost per sample

I propose a 2-part benchmark:

1. Public baseline (so results are comparable)
2. Private challenge set (so models can't overfit our test)

If testing CAD-native feature recognition:

- MFCAD, MFCAD++/MFInstSeg pointers
- Newer hybrid set: HybridCAD (Zenodo)

If testing geometric understanding but not machining labels:

- ABC Dataset (strong CAD corpus, but not directly gear teeth count labels)

If testing screenshots/drawings with LLM vision models:

- Build your own benchmark (best option), e.g. GearCount-Bench with controlled renders + ground-truth counts

Datasets available online

Dataset/Source	Approximate number of CAD files	Labelled?	Free or paid?	Business-use/copyright status
ABC Dataset	1,000,000 CAD models	Partly (features/metadata)	Free download	Copyright stays with original creators, rights vary by source document
Fusion 360 Gallery (Autodesk)	8,251 assemblies, 154,468 parts	Yes (task-specific labels)	Free download	Non-commercial research only, not business-production-safe
DeepCAD dataset	178,238 models	Yes	Free download	Commercial rights are not uniformly guaranteed,
SketchGraphs	15,000,000 CAD sketches	Yes	Free download	Repo is MIT; generally business-friendly but still

				verify provenance of underlying models for legal policy
MFCAD	59,655 models	Yes (25 machining feature types)	Free access	

Notes from meeting with Antoine:

1) Lock the benchmark specification first:

- Inputs per task: task_id, prompt, reference_step, optional expected_bbox, expected_components
- Outputs per run: generated step *or generated scad then converted to step, metrics, pass/fail, logs
- Pass/fail rule: hausdorff_95p $\leq$ H, chamfer $\leq$ C
- Render/conversion succeeds
- Optional topology gate?

2) STEP-based comparison architecture:

- Keep STEP as canonical ground truth
- For Chamfer/Hausdorff, tessellate both STEP models to meshes with fixed tessellation settings
- Compute distances on aligned point clouds/meshes
- Store both raw values and pass/fail flags
- Make tessellation deterministic (same tolerance everywhere) to keep scores consistent

3) Prompt-writing documentation

- Create a prompt writing guide with required fields: dimensions, coordinate system, origin, units, constraints, feature order
- Recommended detail level: enough to fully define geometry, avoid ambiguity
- Bad vs good prompt examples
- Minimum viable prompt checklist
- * CREATE A STRICT PROMPT TEMPLATE USED BY ALL MODELS TO REDUCE VARIANCE

4) Build seed dataset (5-10 tasks)

- Create tasks_step with 5-10 parts across simple, medium, hard
- For each task include, reference_step, task.yaml, prompt_detailed.txt and prompt_minimal.txt

- Parts such as block with through-hole, flange, L-bracket, stepped shaft
- 5) Local harness (repeatable)
- CLI command like: `run_benchmark --tasks ... --models..--replicates 5`
- Save per run artifacts under `results/<run_id>/`
- Summary generator:
 - pass rate per model
 - mean/std of Chamfer/Hausdorff
- Conversion failure rate
- Cost/time

Prompt writing guide for STEP-based CAD benchmarking:

Goal: Write prompts that are:

- Fully specified (minimal ambiguity)
- Objectively measurable against a reference STEP
- Reproducible across models and replicates

Core rules:

- Always specify units (default: mmgs)
- Define coordinate frame and origin explicitly
- Specify all critical dimensions and positions
- State geometric relationships (centered, symmetric, tangent, concentric...)
- Specify cut behaviour (through vs blind + depth), although shouldn't this be assumed by the model? If we give the model all the information, it should do some stuff by itself at least
- Require single connected component unless task says otherwise
- Avoid subjective wording ("nice", "smooth", "ergonomic")

Required prompt structure:

Part identity:

- One line naming the part and objective

Units and frame:

- Units: mm
- Coordinate system: right-handed XYZ
- Origin location
- Axis meaning (which axis)

Base geometry:

- Primitive type, dimensions, exact placement

Additive features

- Bosses, ribs, extrusions with exact dimensions and positions

Subtractive features

- Holes, slots, pockets with exact dimensions, axis, center, depth

Constraints

- Symmetry, concentricity, equal spacing, tangency, coplanarity

Topology/output constraints

- Single connected solid

- No floating bodies
- Match dimensions exactly

Prompt template

- Create a single connected mechanical part

Units and frame:

- Units: millimeters
- Coordinate system: right-handed XYZ
- Origin: [0, 0, 0] at <define exact location>
- Orientation: <define axis roles>

Base geometry:

- Create <primitive> with dimensions <...>
- Place it at <...>

Additive features:

- Feature A: <type>, dimensions <...>, positioned at <...>

Subtractive features:

- Cut A: <type>, dimensions <...>, center <...>, axis <...>, depth <through/blind + value>

Constraints:

- <symmetry/concentric/equal spacing..>
- Keep exactly one connected solid component

Return CAD geometry matching these constraints exactly

	Good Prompt example	Bad Prompt example
Goal	Create a single connected L-bracket.	
Units and frame	Mm, origin at the outer bottom corner where both legs meet.	
Base geometry	Leg A: 80 x 30 x 8 (X x Y x Z), spanning X=0..80, Y=0..30, Z=0..8 Leg B: 30 x 8 x 60, spanning X=0...30, Y=0...8, Z=0..8	
Additive features	Through hole 1: diameter 8, axis +Z, center (20,15,4) Through hole 2: diameter 8, axis +Z, center (60,15,4)	

Subtractive features	Slot on Leg B front face, rounded slot, length 24, width 8, centered at (15,4,35), long axis +Z, through Y thickness	
Constraints	Exactly one connected solid component	
Topology/output constraints		

Replicate guidance:

- Use identical prompt template across models
- Run at least 3 replicates per model-task pair
- Track mean and std for Chamfer and Hausdorff 95p.
- Flag unstable tasks with high metric variance

task schema yaml:

\$schema: "https://json-schema.org/draft/2020-12/schema"

title: "CadEval STEP Task Schema"

description: "Schema for STEP-based text-to-CAD benchmark task files"

type: object

required:

- task_id
- description
- reference_step
- requirements

properties:

task_id:

type: string

pattern: "^[a-zA-Z0-9._-]+\$"

description: "Unique task identifier"

version:

type: string

default: "1.0"

description:

type: string

minLength: 10

description: "Canonical model prompt"

reference_step:

type: string

description: "Path to ground-truth STEP file, e.g. reference/part_001.step"

reference_stl:

type: string

description: "Optional pre-tessellated STL reference"

units:

type: string

enum: [mm, cm, m, in]

default: mm

manual_operations:

type: [integer, "null"]

minimum: 1

description: "Estimated CAD operation count"

difficulty:

type: string

enum: [easy, medium, hard]

tags:

type: array

items:

type: string

requirements:

type: object

required:

- bounding_box

properties:

bounding_box:

type: array

minItems: 3

maxItems: 3

items:

type: number

minimum: 0

description: "Expected [x, y, z] dimensions in task units"

topology_requirements:

type: object

properties:

expected_component_count:

type: integer

minimum: 1

default: 1

must_be_watertight:

type: boolean

default: true

additionalProperties: false

```
metric_thresholds_override:
  type: object
  properties:
    chamfer_threshold_mm:
      type: number
      minimum: 0
    hausdorff_threshold_mm:
      type: number
      minimum: 0
    volume_threshold_percent:
      type: number
      minimum: 0
    bounding_box_tolerance_mm:
      type: number
      minimum: 0
    additionalProperties: false
  additionalProperties: false

metadata:
  type: object
  properties:
    author:
      type: string
    source:
      type: string
    license:
      type: string
    created_at:
      type: string
      format: date-time
    notes:
      type: string
    additionalProperties: true

additionalProperties: false
```

```
task_id: "l_bracket_001"
version: "1.0"
description: >
```


Create a single connected L-bracket in mm. Right-handed XYZ. Origin at the outer bottom corner where legs meet.

Leg A: 80x30x8 (XxYxZ), X=0..80, Y=0..30, Z=0..8.

Leg B: 30x8x60, X=0..30, Y=0..8, Z=0..60.

Through-holes on Leg A: diameter 8 at centers (20,15,4) and (60,15,4), axis +Z.

Rounded slot on Leg B: length 24 width 8, center (15,4,35), long axis +Z, through Y thickness.

Exactly one connected solid component.

reference_step: "reference/l_bracket_001.step"

reference_stl: "reference/l_bracket_001.stl"

units: "mm"

manual_operations: 9

difficulty: "medium"

tags: ["bracket", "holes", "slot", "single_component"]

requirements:

bounding_box: [80, 30, 60]

topology_requirements:

expected_component_count: 1

must_be_watertight: true

metric_thresholds_override:

chamfer_threshold_mm: 1.0

hausdorff_threshold_mm: 1.2

volume_threshold_percent: 2.0

bounding_box_tolerance_mm: 1.0

metadata:

author: "Isidor AI"

source: "internal_cad_set_v1"

license: "internal-use"

created_at: "2026-02-18T10:00:00Z"

notes: "Pilot benchmark task"

“ If a prompt does not match the required guide template, hybrid linter combining Python code and AI review”

```
REQUIRED_HEADINGS = [  
    "Part identity",  
    "Units and frame",  
    "Base geometry",  
    "Additive features",  
    "Subtractive features",  
    "Constraints",  
    "Topology/output constraints",  
]
```

```
FORBIDDEN = ["nice", "smooth", "ergonomic", "approximately", "roughly"]
```

```
def lint_prompt(text: str):  
    errors = []  
    for h in REQUIRED_HEADINGS:
```

```

    if h.lower() not in text.lower():
        errors.append(f"Missing section: {h}")
    if "mm" not in text.lower():
        errors.append("Units not specified (expected mm or explicit units).")
    if any(w in text.lower() for w in FORBIDDEN):
        errors.append("Contains ambiguous wording.")
    if not any(ch.isdigit() for ch in text):
        errors.append("No numeric dimensions found.")
    return errors

```

1. Deterministic linter (hard fail)

- Enforce required sections from guide
- Check forbidden ambiguous words (nice, smooth, approximately, etc.)
- Check required numeric presence (dimensions, coordinates, units)
- Return precise error messages

2. AI reviewer (soft fail or warning)

- If deterministic checks pass, send prompt to an LLM reviewer rubric
- Ask it to score completeness/ambiguity/measurability and suggest fixes
- Optionally block if score < threshold

Use it in:

- pre-commit (local contributor feedback)
- CI (PR blocking)
- backend submission endpoint

```

REQUIRED_HEADINGS = [
    "Part identity",
    "Units and frame",
    "Base geometry",
    "Additive features",
    "Subtractive features",
    "Constraints",
    "Topology/output constraints",
]

```

```

FORBIDDEN = ["nice", "smooth", "ergonomic", "approximately", "roughly"]

```

```

def lint_prompt(text: str):
    errors = []
    for h in REQUIRED_HEADINGS:
        if h.lower() not in text.lower():
            errors.append(f"Missing section: {h}")
    if "mm" not in text.lower():
        errors.append("Units not specified (expected mm or explicit units).")
    if any(w in text.lower() for w in FORBIDDEN):
        errors.append("Contains ambiguous wording.")
    if not any(ch.isdigit() for ch in text):
        errors.append("No numeric dimensions found.")
    return errors

```

Define a rubric: score each prompt 0-5 on:

- structure_compliance
- dimensional_completeness
- positional_clarity
- constraint_clarity
- measurability
- ambiguity_risk

Compute weighted total (0-100)

Use a strict reviewer prompt

System prompt should say:

“You are a CAD prompt QA reviewer. Return JSON only. Use the rubric exactly. If anything is missing, list concrete fixes.”

“Should there be some information that is not given to make the model work a bit. What about information which the model should be assuming?”

It could be helpful to have two prompt modes per task:

1. Fully specified (primary benchmark)

- Give all critical geometry and constraints
- Measures faithful execution

2. Underspecified (secondary benchmark)

- Omit only low-risk details with documented defaults
- Measures reasonable inference

What can be safely omitted (if defaults are fixed)?

- Segment count / tessellation quality
- Fillet/chamfer when not function-critical
- Minor placement references if symmetry + main dimensions already pin them down
- Modeling sequence (unless order affects result)

What should never be omitted?

- Functional dimensions
- Hole/slot diameters and positions
- Through vs blind depths
- Coordinate frame/origin if placement matters
- Component count/connectivity expectations

Best practice:

- Add a default_assumptions block in each task YAML (e.g. if unspecified, no fillets/chamfers; centered on symmetry plane; through-cuts only when explicitly stated)
- Score strict against reference for fully specified tasks
- For underspecified tasks, allow tolerance bands or multiple acceptable references.

1. Deterministic benchmark:

- Prompts are fully specified
- One intended geometry

- Strict pass/fail against one reference STL/STEP
- Use this for “how accurate is the model?”

2. Inference benchmark:

- Prompts intentionally omit some non-critical details
- Model must make reasonable assumptions
- Use looser scoring or multiple acceptable references?
- Use this for “how well does the model infer missing details?”

- WHY IS THIS IMPORTANT?

- Deterministic track tells you precision/compliance
- Can the model follow exact engineering specs?
- Inference track tells you practical usefulness
- Can it still produce a good part when real prompts are incomplete?

If we only run deterministic, we miss real-world behaviour

If we only run inference, we can't quantify strict accuracy

Using both gives a complete picture for product decisions; reliability under strict specs and robustness under normal user prompts.

It maps directly to business risk and product value.

1. Deterministic track proves contract reliability: ‘given exact specs, model outputs exact geometry’, useful for enterprise buyers, QA teams, and safety/compliance use cases
2. Inference track proves user-product fit: ‘given incomplete prompts, model still delivers usable CAD., useful for growth, onboarding and real user workflows.

For an AI firm, this is attractive because it gives:

- Better model selection signals (not just one score)
- Clear product positioning (accurate mode vs assist mode)
- Actionable roadmap data (whether to improve parser/compliance vs reasoning/inference)
- Stronger sales narrative with two concrete KPIs:
- spec adherence rate
- inference robustness rate

CAD BENCHMARK: CADProof, TrueCAD Bench, STEP Score, Spec2Solid, CADAlign

1. Standardise geometry pipeline:

- Convert STEP→mesh with fixed tessellation settings
 - Use same mesher/settings for both reference and generated
 - Normalise units before comparison
2. Enforce alignment robustness
 - Use deterministic pre-alignment (centroid + PCA or bbox)
 - Then ICP refinement
 - Retry with multiple initial rotations for symmetric parts
 - Store ICP fitness and fail alignment explicitly if weak
 3. Use metric bundle, not one metric
 - Primary: Chamfer distance, Hausdorff distance 95p
 - Secondary: Hausdorff 99p, volume %, bbox delta, component count, watertightness
 - Tertiary: Compare render JPEGs
 - Gate on hard checks first (render/mesh/topology) then distance metrics
 4. Calibrate thresholds statistically
 - Build a calibration set (20-40 known-good and known-bad pairs)
 - Plot metric distributions
 - Choose thresholds that minimize false pass/false fail for our use case
 - Keep separate thresholds by difficulty tier if needed
 5. Handle ambiguity explicitly
 - Deterministic tasks: strict single-reference thresholds
 - Inference tasks: looser thresholds or multi-reference acceptance
 - Don't mix these when reporting a single score
 6. Add reproducibility controls
 - Fixed random seeds where possible
 - Same sampling count for point clouds
 - Same replicate count per model/task
 7. Validate benchmark quality
 - Run sanity cases:
 - identical mesh vs itself → near-zero distances
 - translated/rotated same mesh → should still pass after alignment
 - known bad geometry → should fail
 - track failure taxonomy: generation failure, conversion failure, alignment failure, metric failure
 8. Report confidence, not just pass/fail
 - For each metric report value + threshold + margin
 - Aggregate with mean/std over replicates
 - Flag unstable tasks (high variance)

*2: Enforce alignment robustness?

- Deterministic pre-alignment:

It means, put both shapes in a common starting pose, always the same way

- centroid: move each shape so center is at origin
- PCA or bbox axes: rotate each shape so its main axes line up with X/Y/Z
- this avoids random misalignment
- ICP refinement:
 - after rough alignment, run ICP (iterative closest point) to fine-tune translation/rotation for best overlap
 - ICP needs a decent starting pose; that's why pre-alignment comes first
- Multiple initial rotations for symmetric parts:
 - symmetric parts can look "equally valid" in multiple orientations (e.g. 90 degrees, 180 flips)
 - try several candidate starting relations, run ICP from each and keep the best result
 - This reduces false failures caused by symmetry ambiguity
- Store ICP fitness:
 - fitness is a quality score of alignment overlap (higher is better)
 - save it per run for debugging and thresholding
- Fail alignment explicitly if weak
 - if best ICP fitness is below our minimum (or transform clearly bad), mark alignment_failed instead of trusting distance metrics
 - otherwise chamfer/hausdorff may be large just because alignment failed, not because geometry is wrong
- THESE ARE ALL PREPROCESSING STEPS BEFORE FINAL METRIC COMPUTATION

How does PCA work here?

- PCA here uses eigenvectors/eigenvalues of the point cloud covariance matrix
- Compute points' centroid and subtract it (center cloud)
- Compute covariance matrix of centered points
- Eigenvectors = principal axes (directions of max variance)
- Eigenvalues = variance magnitude along those axes
- Use eigenvectors as a rotation basis to align shape axes to world axes (X/Y/Z).
- centroid handles translation
- PCA eigenvectors give a canonical orientation guess before ICP
- Centroid handles translation: you compute each shape's center point and shift both shapes so their centers sit at the origin. This removes position offset.
- PCA eigenvectors give orientation guess: from each centered shape, PCA gives 3 main axes (eigenvectors). You rotate one shape so its PCA axes align with the other's (or with global XYZ). This removes most rotation offset.

- Then ICP only needs to do small refinements, instead of solving a hard global misalignment from scratch.

Terms:

- Point cloud: a set of 3D points (x,y,z) samples from a surface. No faces/edges by default, just points

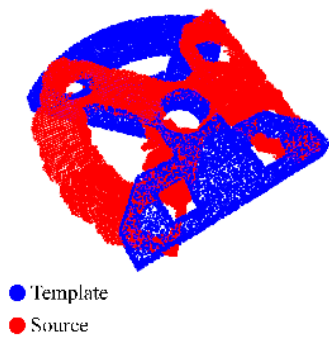


Figure 1. An example of a solution with a negative R^2 value. The estimated transformation (red) does not overlap the template (blue). The result shown is for the base-top plate object, after applying PointNetLK.

2.4. Ground-Truth Estimation

To compute the metrics from Section 2.2, we needed to estimate the ground truth. The process was based on using known information, like the object's orientation on the table, estimated values such as the normal vector on the table and finally, a visual correction for the translation [6]. We placed the 3D camera parallel to the table, which meant the x-axis of the template was too. Figure 5 shows the steps, which are detailed below.

1. We centred both point clouds on the origin by subtracting their mean.
2. Using an estimation of the normal vector on the table, we performed a rotation around the template x-axis, aligning the y-axes of both objects.
3. Given the known rotation of the object on the table, we performed a final rotation around the new y-axis.
4. We performed visual corrections for the rotation and mainly translation, similar to [6].

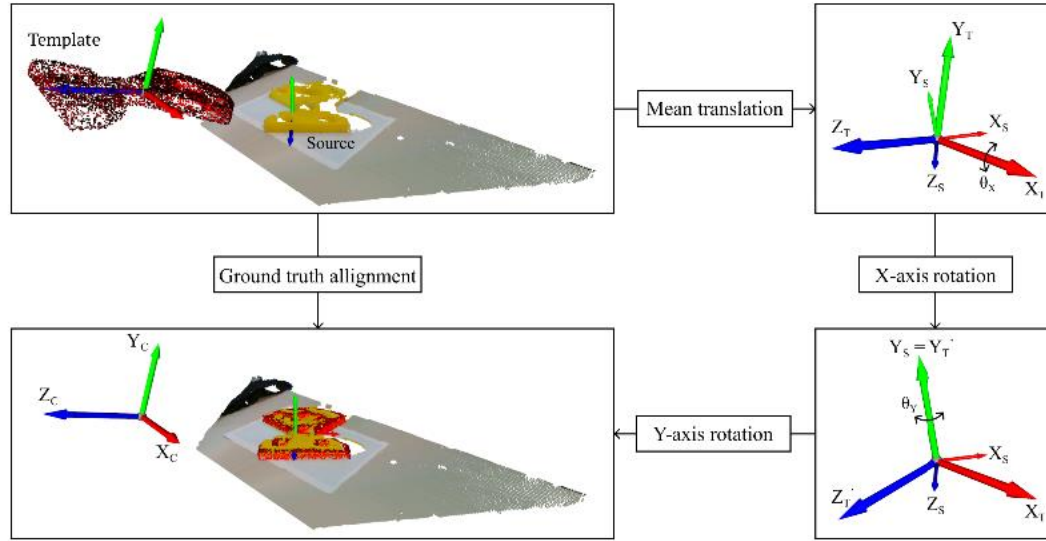


Figure 5. Steps performed to estimate the ground-truth estimation. The red and yellow point clouds represent the template and source, respectively. $X_T Y_T Z_T$ is the template's and $X_S Y_S Z_S$ the source's coordinate system.

Table 1. Parameters checked for each method. A check mark (✓) or range indicates the parameter is verified, a dash (/) means the parameter is not applicable and a lightning symbol (⚡) indicates the parameter is checked, but no convergence was reached. Training models are reported in Section 2.3.

Method	GO-ICP	RANSAC	FGR	PointNetLK	RPMNet	ROPNet
Zero mean	✓	✓	✓	✓	✓	✓
Refinement	✓	✓	✓	✓	✓	✓
Bounding box	⚡	1 → 1.8				/
Voxel size [m]	$10^{-3}, 10^{-2}$	$10^{-4} \rightarrow 10^{-1}$		$0, 10^{-3} \rightarrow 10^{-2}$		
MSE Threshold [m]	$10^{-5} \rightarrow 10^{-1}$	/	/	/	/	/
Trim fraction [/]	$10^{-4} \rightarrow 10^{-1}$	/	/	/	/	/
Training model	/	/	/	✓	✓	✓

We adapted the bounding box to simulate the effect of different cutouts around the object, as in Figure 6. The larger the bounding box, the more environmental information is included in the point cloud. However, methods that can work for larger bounding boxes require less pre-processing. This is especially interesting in real-time applications. Furthermore, the maximal bounding box places additional requirements on preliminary object detection and filtering steps.

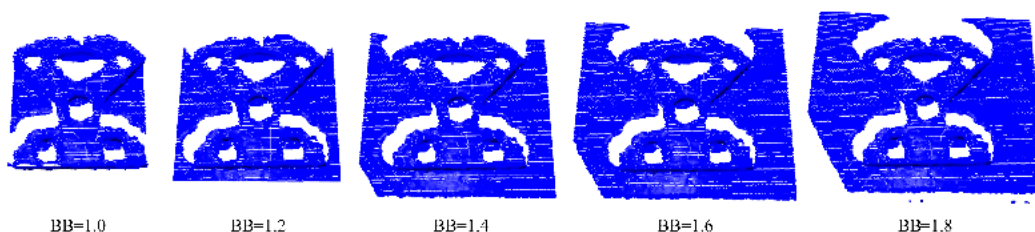


Figure 6. Effect of adapting the bounding box to increase the information around the object in the point cloud

- They compared 6 registration methods for CAD-model-to-scan alignment on real noisy/partial point clouds: GO-ICP, RANSAC, FGR, PointNetLK, RPMNet, ROPNet

Main takeaway:

- Highest accuracy/lowest failures: GO-ICP (but slow)
- Good accuracy-speed tradeoff: RANSAC + ICP refinement (also RPMNet + ICP, PointNetLK + ICP)
- Fastest but less reliable: FGR and some learning methods in harder/noisier cases

For our use case, based on their findings, use:

1. Coarse global alignment with RANSAC
2. Then ICP refinement

If we want maximum accuracy and can tolerate slow runtime, use GO-ICP as a higher-precision baseline

- RANSAC + ICP gave strong precision while staying much faster than GO-ICP
- It is a robust default pipeline for batch benchmarking where throughput matters

According to “Improving Fitting CAD to 3D Point Cloud Acquired with Line-of-Sight Sensor”

- Standard ICP can be systematically wrong when the observed point cloud is incomplete
 - Even with a good initial pose, ICP can converge to a biased alignment if you register against the full CAD surface
 - Their fix: run ICP using only the visible/relevant subset of CAD surfaces (not the whole model)
 - Don't do full-surface ICP by default for incomplete observations
 - Build a visibility/region mask on the CAD and run partial-surface ICP
 - Keep two alignment scores: ICP residual + a second consistency metric and flag disagreement cases for review
 - For strict benchmarking, store whether alignment was full-surface or partial-surface to avoid apples-to-oranges comparisons
-
- WHAT THIS MEANS?
 - If the generated geometry is missing parts (or only partial), normal ICP against the whole reference can misalign and inflate error unfairly.
 - So instead of matching to all reference triangles, match only to the reference region that should be visible/comparable for that case
 - Don't trust one alignment number only
 - Keep ICP fit/error
 - plus another check (e.g. Chamfer on inlier region)
 - if they disagree, mark that run for manual review
 - In results schema, record alignment mode:
 - `alignment_mode = full_surface` or `partial_surface`
 - This prevents comparing scores computed under different alignment assumptions as if they were identical.

Paper summary (Liu et al., Optik 2021, DOI: 10.1016/j.ijleo.2021.167304)

They target part-to-whole registration: incomplete measured blade point cloud vs full CAD model

- Baseline PCA → ICP fails when shapes differ a lot (missing/occluded regions)
 - They propose MRPCA-ICP:
 - extract a consistent major region (MR) from both scan and CAD (using a geometric constraint plane)
 - Do coarse alignment with MR-based PCA (not full-shape PCA)
 - Add orientation-direction consistency logic for principal axes
 - Run ICP for fine refinement
-
- Why is it useful for us?
 - If generated CAD is incomplete/wrong in some regions, full-shape PCA/ICP can align badly and inflate Chamfer/Hausdorff

- The key idea is to align using a shared, stable subregion first then run ICP refinement
- This is especially useful for inference track (underspecified prompts, partial geometry correctness)

MY PROPOSAL:

- Implement a single hybrid pipeline with fallback logic, not one method only.
 - 1) Default (most tasks): RANSAC → ICP
 - Best practical accuracy/speed baseline for benchmark
 - 2) If partial/incomplete overlap is detected: MRPCA init → Partial-surface ICP
 - Use when generated part is missing regions or only partially matches reference
 - This is where paper 2 and 3 ideas apply
 - 3) If alignment confidence is low after both above: optional GO-ICP audit pass
 - Use only for difficult cases (slow, but high-confidence check)

Decision rule to keep it simple:

- Start with RANSAC → ICP
- If ICP fitness < threshold or residual too high, rerun with partial/MRPCA path
- If still weak, mark alignment_failed (or run GO-ICP for verification)

1.	TEASER++	• Very robust global registration under extreme outliers.
2.	Super4PCS / 4PCS	• Fast global coarse alignment for partial overlap.
3.	FGR (Fast Global Registration)	• Faster than RANSAC, sometimes less stable.
4.	Point-to-plane ICP (vs point-to-point)	• Usually converges faster/better on smooth surfaces if normals are good.
5.	Trimmed ICP / robust ICP kernels	• Better when overlap is partial or there are outliers.
6.	CPD / GMMReg (probabilistic registration)	• More robust to noise/partiality; slower.
7.	NDT (Normal Distributions Transform)	• Voxel/probabilistic alignment; common in robotics pipelines.
8.	Feature-based descriptors (FPFH/SHOT) + global matching	• Good coarse initialization before ICP.
9.	Symmetry-aware alignment checks	• Important for CAD parts with rotational/reflection symmetry.
10.	Learning-based registration (PointNetLK, RPMNet, etc.)	• Can be fast, but generalization/stability depends on training domain.

For your benchmark, keep **RANSAC→ICP** as baseline, then add **TEASER++ or Super4PCS** **fallback** and **trimmed/partial ICP** for incomplete matches.

Now, for the metrics

Rank	Metric	What it means / why it matters
1	Valid CAD rate	Whether output parses/converts at all; without this, nothing else matters.
2	Alignment quality (ICP fitness + inlier RMSE)	Ensures geometric distances are trustworthy, not artifacts of bad alignment.
3	Chamfer distance	Average shape mismatch; strong primary geometry metric.
4	Hausdorff 95p	Captures near-worst local errors while being less outlier-sensitive than max Hausdorff.
5	Volume difference (%)	Strong check for global mass/size correctness.
6	Bounding-box error (axis-wise/max)	Quick sanity check for overall dimensions.
7	Topology component count match	Ensures one part vs multiple disconnected pieces is handled correctly.
8	Watertight/manifold check	Required for manufacturability and reliable downstream CAD/mesh ops.
9	Critical dimension error	Direct engineering check on key dimensions (holes, thicknesses, offsets).
10	Tolerance-band pass rate	Fraction of required dimensions/features within tolerance.
11	Hausdorff 99p (diagnostic)	Tracks sharper outlier regions than 95p; useful secondary signal.
12	Point-to-surface distance stats	Mean/median/max distances on actual surfaces, often more stable than point-only.
13	Feature/primitive count match	Checks missing/extra holes, slots, bosses, etc.
14	Self-intersection count	Flags invalid geometry that may still look correct visually.
15	Success rate (generation/render pass)	Operational reliability of the model+pipeline.
16	Replicate consistency (variance)	Measures stability; low variance is key for production trust.
17	Pass@k	Practical "can it get one good output in k tries" metric.
18	Runtime (gen + convert + check)	Throughput and latency for real deployment.
19	Cost per successful sample	Business efficiency metric for model selection.
20	Oriented bounding-box error	Better size/orientation check for rotated parts.
21	Surface area difference (%)	Detects extra/missing surface detail not always visible in volume.
22	Normal consistency	Compares local surface orientation quality.
23	Mass properties (COM/inertia)	Engineering-relevant when density assumptions are fixed.

Rank	Metric	What it means / why it matters
24	Centroid offset	Useful diagnostic; low value alone does not guarantee shape match.
25	Inertia tensor/moments difference	Global shape distribution comparison; secondary to Chamfer/Hausdorff.
26	Euler characteristic/genus match	Topological correctness (holes/handles) at abstract level.
27	Hole/void count match	Structural topology check; useful in mechanical parts.
28	Cross-section profile distance/loU	Strong for parts defined by critical slices.
29	Slice-wise contour distance	Similar to above, useful for profile-controlled geometry.
30	Voxel IoU / Dice	Coarse occupancy similarity; resolution-dependent.
31	Occupancy precision/recall/F1	Voxelized inclusion/exclusion quality; also resolution-dependent.
32	EMD/Wasserstein distance	Strong but expensive to compute at scale.
33	Curvature distribution difference	Surface-detail fidelity metric; noisy on poor meshes.
34	Feature-edge overlap/distance	Checks sharp-edge placement, useful for machined parts.
35	Signed volume difference	Directional over/under material bias; secondary to absolute volume diff.
36	Silhouette IoU (fixed views)	Visual sanity metric; can miss hidden/internal errors.
37	SSIM/LPIPS on renders	Perceptual similarity; useful for UI, weak as primary CAD metric.
38	Registration failure rate	Operational diagnostic of the aligner quality itself.
39	Reliability under perturbation	Robustness to prompt noise/paraphrase; valuable but usually phase-2.
40	Composite weighted score	Good executive summary, but only after raw metrics are sound.

- Normal angle error: measures the angular consistency between the predicted normal and the true normal
- Chamfer distance (CD): by comparing the average distance from the randomly sampled points on the surface of two shapes to the nearest corresponding point, the geometric similarity is measured
- Hausdorff distance: the farthest pair of distances among all pairs of randomly sampled points between the two shapes is taken, reflecting the maximum geometric deviation between them.
- Edge chamfer distance: Calculates the average nearest distance between the points on the edges of two CAD models to characterize the proximity of the contour structure.

Now that all the metrics have been defined, thresholds have to be defined.

Below are **starter thresholds** for a mechanical CAD benchmark in **mm units**, intended for first calibration (not final production).
I chose them to be strict enough to catch meaningful geometry errors but still realistic for current text-to-CAD models.

1. `valid_cad_rate >= 1.00`
Why: hard gate; invalid CAD is unusable.
2. `alignment_quality_icp_fitness >= 0.30`
Why: below this, alignment is usually unreliable for distance metrics.
3. `alignment_inlier_rmse_mm <= 1.00`
Why: keeps registration residual within practical geometric tolerance.
4. `chamfer_distance_mm <= 1.00`
Why: strong global similarity threshold, common in your current setup.
5. `edge_chamfer_mm <= 0.80`
Why: edge fidelity should be stricter than surface average.
6. `hausdorff_95p_mm <= 1.50`
Why: robust near-worst-case tolerance (less brittle than 1.0).
7. `hausdorff_99p_mm <= 2.50`
Why: allows small outlier spikes while penalizing major local errors.
8. `point_to_surface_mean_mm <= 0.80`
Why: average surface deviation should stay sub-mm for good fit.
9. `point_to_surface_max_mm <= 4.00`
Why: catches catastrophic local mismatches without over-penalizing tiny spikes.
10. `volume_diff_percent <= 2.00`
Why: standard engineering-level global mass/volume tolerance.
11. `signed_volume_diff_percent <= 2.00`
Why: same as above but directional bias control.
12. `surface_area_diff_percent <= 3.00`
Why: area is noisier than volume but still useful for shape complexity mismatch.
13. `bbox_error_max_mm <= 1.00`
Why: keeps overall size close to reference.
14. `bbox_error_axis_xyz_mm <= 1.00` (each axis)
Why: prevents one-axis drift hidden by max-only metrics.
15. `obb_error_max_mm <= 1.50`
Why: slightly looser than axis-aligned bbox due to orientation sensitivity.
16. `centroid_offset_mm <= 1.00`
Why: after alignment, centers should nearly coincide.
17. `inertia_tensor_error <= 0.10` (normalized)
Why: controls global mass distribution mismatch.
18. `mass_properties_error <= 0.10` (normalized aggregate)
Why: practical cap on COM/inertia mismatch combined.
19. `component_count_match == 1`
Why: hard topology correctness gate for single-part tasks.

20. `watertight_manifold_pass == 1`
Why: manufacturing and geometry-check prerequisite.
21. `self_intersection_count == 0`
Why: self-intersections indicate invalid geometry.
22. `euler_genus_match == 1`
Why: enforces topological equivalence (holes/handles structure).
23. `void_hole_count_match == 1`
Why: catches missing/extra through features important in mechanical parts.
24. `feature_count_match >= 0.90`
Why: allows minor detection misses but expects near-complete feature fidelity.
25. `critical_dimension_error_mm <= 0.50`
Why: critical dims need tighter engineering tolerance.
26. `tolerance_band_pass_rate >= 0.90`
Why: at least 90% of checked dims/features must pass tolerance.
27. `feature_edge_distance_mm <= 0.80`
Why: edge placement should be close for functional surfaces.
28. `normal_consistency >= 0.95`
Why: high directional agreement for local surface quality.
29. `normal_angle_error_deg <= 12.0`
Why: equivalent to strong normal consistency without over-fragility.
30. `curvature_distribution_error <= 0.15` (normalized distance)
Why: allows minor smoothness artifacts but flags clear shape-detail drift.
31. `cross_section_iou >= 0.90`
Why: profile-critical parts should overlap strongly in key slices.
32. `slice_contour_distance_mm <= 0.80`
Why: contour deviations should remain sub-mm average.
33. `voxel_iou >= 0.88`
Why: good volumetric overlap at fixed voxel resolution.
34. `occupancy_precision >= 0.90`
Why: penalizes extra material hallucination.
35. `occupancy_recall >= 0.90`
Why: penalizes missing material.
36. `occupancy_f1 >= 0.90`
Why: balanced occupancy quality target.
37. `emd_distance <= 1.20` (resolution-normalized mm scale)
Why: stricter global transport mismatch cap, but more tolerant than Chamfer.
38. `silhouette_iou >= 0.92`
Why: strong visual-profile consistency across fixed views.
39. `render_ssim >= 0.90`
Why: conservative visual similarity threshold.
40. `render_lpips <= 0.15`
Why: perceptual dissimilarity should remain low.
41. `registration_failure_rate <= 0.05`
Why: benchmark pipeline should be robust; >5% indicates method instability.

42. `composite_weighted_score >= 0.85`

Why: final acceptance gate after individual metric checks.

Important:

- These are **initial priors**. Final thresholds should be calibrated using your labeled positive/negative set with the tuning script.
- For deterministic track, tighten many of these by ~10–20%.
- For inference track, loosen by ~10–25%.

When first executing the local interface, the log confirms the issue: exited with code 137 means the container is being OOM-killed (out of memory)

Patched to reduce memory usage:

- `fast_mode` now caps sample points to 4000 and voxel pitch to ≥ 4.0
- skips voxel occupancy metrics if estimated voxel grid is too large
- reduces EMD sample size in fast mode
- disabled auto-restart so crashes are visible

Container is being killed by the OS/container runtime

- App loads two meshes and computes heavy metrics
- Some metrics allocate large arrays (point samples, pairwise distances, voxel grids)
- Memory usage spikes beyond the container's allowed RAM

Chamfer/Hausdorff are computed from sampled surface points on both meshes:

1. Sample points on reference surface (uniform-ish over triangle area)
2. Sample points on generated surface
3. For each ref point, find nearest generated point distance
4. For each generated point, find nearest ref point distance

STEP VS STL:

Why STEP?

- Exact B-Rep geometry (surfaces/solids), not triangle approximation
- Better for engineering checks (volume, topology, manufacturability)
- CAD-native interchange standard

Why STL?

- Easy and universal for rendering/printing
- Required for many geometric distance metrics (Chamfer/Hausdorff pipeline)

Best practice:

- Use STEP as a source-of-truth artifact

- Tessellate to STL/point clouds for distance metrics with fixed settings
- STEP for fidelity and canonical storage
- STL for practical numeric comparison metrics

Because most of the similarity metrics you're using are defined on **discrete samples**, not symbolic B-Rep entities.

- **STEP B-Rep is exact:** NURBS/analytic faces, trimmed surfaces, topology graph.
- **Chamfer/Hausdorff/voxel/silhouette** need point sets or triangles, so you must sample/tessellate first.
- Exact B-Rep-to-B-Rep distance over whole solids is possible in theory, but in practice it is:
 - harder to implement robustly,
 - slower,
 - less available in common benchmarking libraries.

So the practical setup is:

1. Keep **STEP as canonical truth**.
2. Compute **STEP-native metrics** (volume, area, topology validity) directly.
3. Tessellate with fixed settings and compute **mesh metrics** (Chamfer/Hausdorff, etc.).

That gives both exact CAD checks and scalable geometric similarity scoring.

Thursday 19th Feb:

- Check CadQuery
- build123d
- pythonOCC/OCP
- FreeCAD Python API

Project where I input a prompt and ask an AI model to edit/modify my already existing STEP file, for example, scale by a factor of 2 or add a gear to it...

- Very doable: AI model can't edit raw STEP text directly though
- Would involve converting prompt to structured CAD operations and a CAD engine applies them.
- Input:
- User uploads .step/.stp
- Backend loads it with a CAD kernel (OpenCascade/FreeCad/CadQuery)
- Prompt understanding:
- Prompt such as "Scale by part by a factor of 2" goes to GPT/Gemini/Whichever AI
- Model returns strict JSON commands such as:

```
[  
  {"op": "scale_body", "factor": 1.05},  
  {"op": "keep_hole_diameters", "enabled": true}  
]
```

- Geometry execution:
- CAD engine executes those commands (scale, offset faces, add features, Boolean union/subtract)
- Validation:
- Check solid is valid
- Run it by the benchmark
- Output:
- Export modified STEP
- Show before after 3D preview and operation history

What's easy first:

- Uniform scale, resize dimensions, fillets/chamfers, holes, cutouts

What's harder:

- Adding parts such as a gear, requires design intent

- Could have an option in the platform where you pick either:
- Generate a new STEP file just from a prompt and compare to reference STEP
- Modify existing STEP file and compare to reference STEP
- Extract information based on inputted STEP file and output an answer

1. **Text-to-STEP Generation**

Create a new CAD part directly from a structured prompt, then benchmark the generated STEP against a reference.

2. **STEP-to-STEP Modification**

Start from an existing STEP model, apply prompt-based design changes, and evaluate how accurately the modified output matches the target specification.

3. **STEP Question Answering (Geometry Understanding)**

Ask targeted questions about an existing STEP file (for example, "How many teeth does this gear have?") and return direct geometric answers from the model.

19th Feb:

- Dashboard now has 3 options for the different task types
- Added a placeholder for option 3 where the user can input what the GTFA is so that the platform can compare its own answer with the GTFA and know what it got wrong
- Added a section where the platform renders the reference STEP and the generated STEP and displays 3D views of both

To fix the scoring system:

1. Keep binary pass/fail
 - Still compute each metric pass/fail vs threshold (for strict compliance)
 - Keep an overall_pass gate for required metrics
2. Add a continuous per-metric quality score (0 to 1)

For each metric:

 - If lower_better: $\text{margin} = (\text{threshold} - \text{value}) / \text{scale}$
 - if higher_better: $\text{margin} = (\text{value} - \text{threshold}) / \text{scale}$
3. Then map margin to score smoothly:
 - $\text{score}_i = 1 / (1 + \exp(-k * \text{margin}))$ (sigmoid, e.g. $k = 2$)

This gives:

- just below threshold → near 0.5 but above
- just above threshold → near 0.5 but below
- far better → near 1
- far worse → near 0

So two failed metric values are no longer both "0"; closer one scores higher.

4. Use metric-specific scale (scale):
 - $\text{scale} = \max(\text{rel_scale} * \text{abs}(\text{threshold}), \text{abs_floor})$
 - Example defaults: $\text{rel_scale} = 0.2$
 - For near-zero thresholds (e.g. self-intersection), use explicit abs_floor from config.
5. Weighted aggregation
 - $\text{Final_quality_score} = 100 * \sum(w_i * \text{score}_i) / \sum(w_i)$
 - Use category weights (e.g. topology, geometry, distance, visual) so not all 42 metrics are equal
6. Report both outputs:
 - overall_pass (strict)
 - $\text{quality_score}_{0_100}$ (ranking)

Test #2:

Prompt:

Create a single CAD output representing a pair of intermeshed external spur gears for benchmark evaluation.

Units and frame:

Units are millimeters.

Use a right-handed XYZ coordinate system.

Place Gear 1 center at (0, 0, 0).

Place Gear 2 center on the +X axis.

Gear axes are parallel to +Z.

Both gears lie in the XY plane and are extruded along Z.

Geometry:

Create two identical spur gears with:

Module $m = 4$

Number of teeth $z = 40$ each

Pressure angle 20 deg (standard involute spur gear)

No helix angle (spur gears, straight teeth)

Gear thickness 20

Compute pitch diameter per gear: $d = m * z = 160$.

Compute pitch radius per gear: $r = 80$.

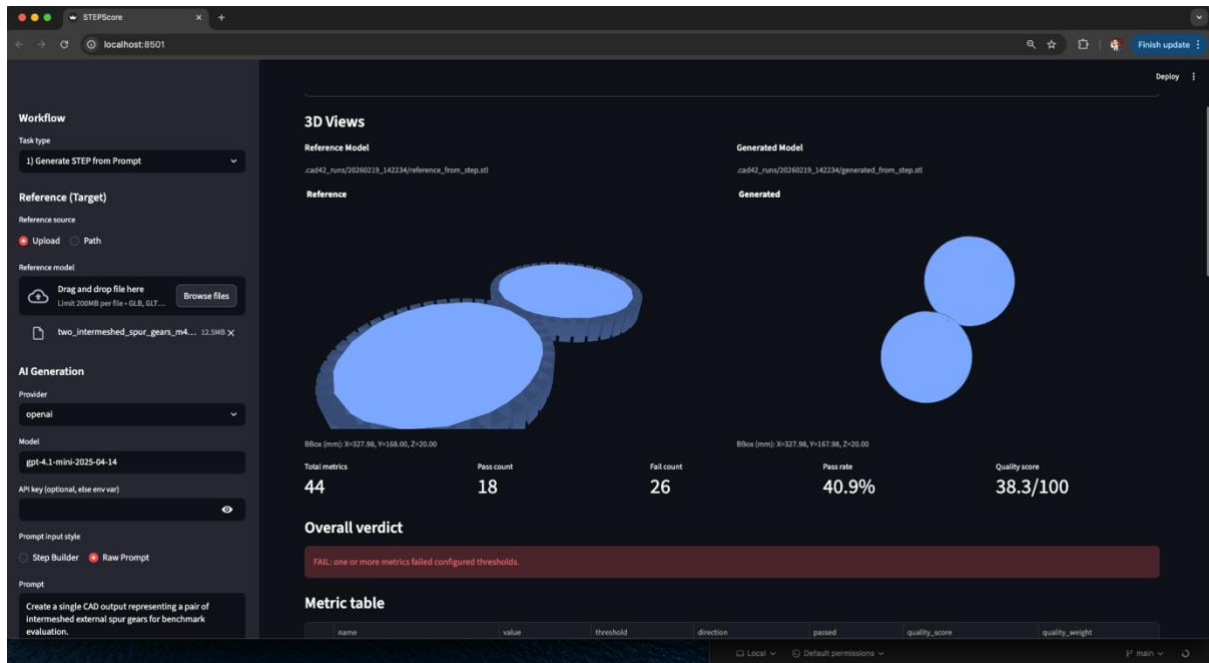
Set center distance for meshing: $a = (d1 + d2)/2 = 160$.

Therefore:

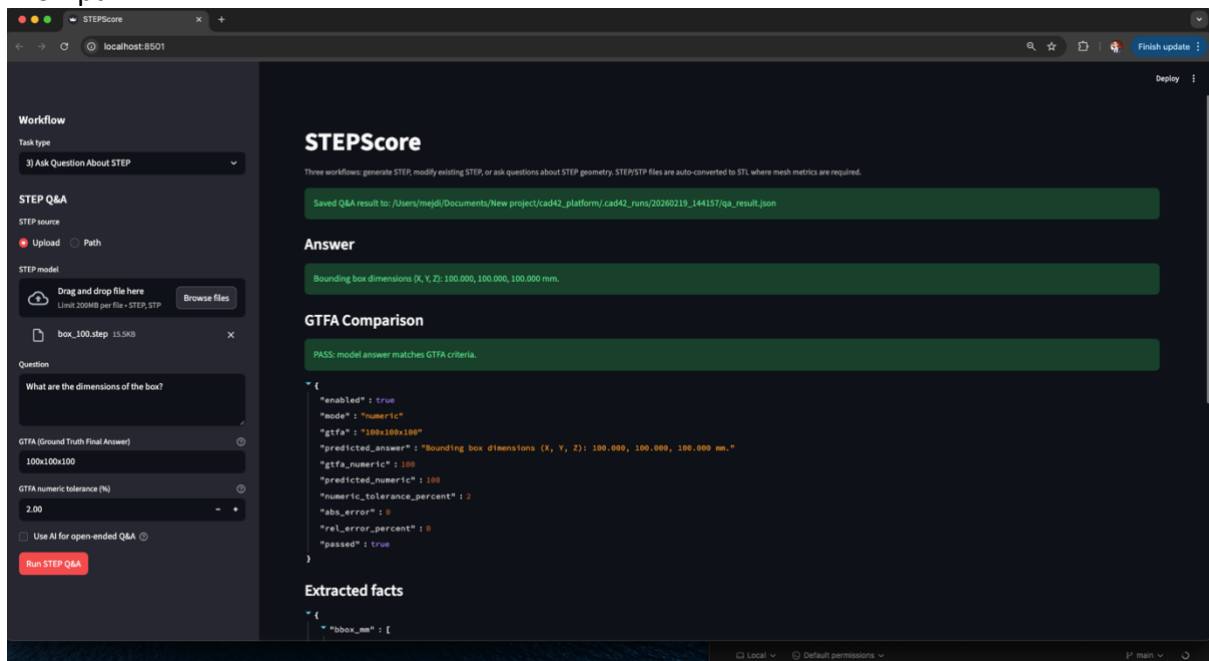
Gear 1 center = (0, 0, 0)

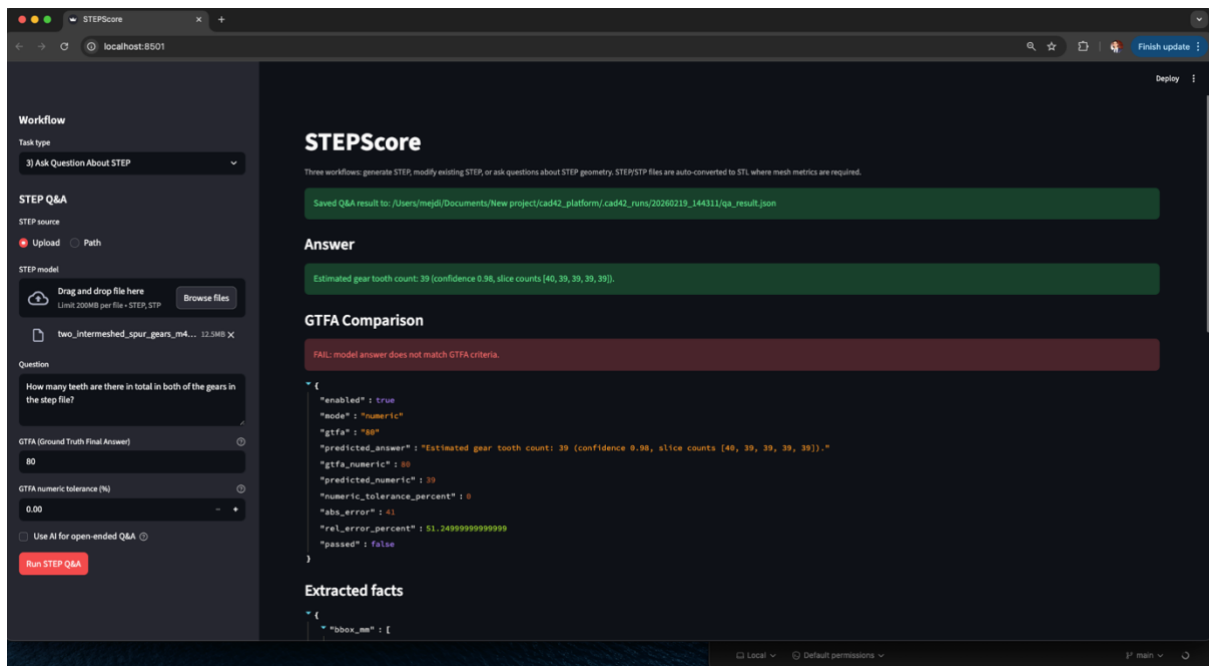
Gear 2 center = (160, 0, 0)

Additive features:
No hubs, bosses, shafts, or keyways.
Only the two toothed gear solids.



Prompt #2:





Based on the tests ran so far, some key fixes need to be implemented as the next steps:

- Firstly, the runtime should be stabilized
 - A clean reproducible environment should be created
 - Add a startup self-check page: CadQuery import, STEP export test, STL conversion test? why is this significant?
3. Lock benchmark policy:
 - Finalise strict pass gates (critical metrics) vs quality ranking (quality_score_0_100)
 - Write one versioned scoring policy doc so all future results are comparable
 4. Calibrate thresholds and weights?
 - Use a calibration set (easy/medium/hard parts) to tune thresholds?
 - Freeze a baseline config?
 5. Build dataset pipeline:
 - Curate 50-100 reference STEP tasks with prompt + GTFA + metadata
 - Split into deterministic and inference tracks

Possible drawing to STEP pipeline:

- Input: accept one drawing package
- front + top + right views
- explicit units and scale
- PNG/JPG/PDF

3. Drawing parser:

- detect view regions, dimension annotations, centerlines, hole callouts

Friday 20th February:

Text2CAD: Generating Sequential CAD Models from Beginner-to-Expert Level Text Prompts

- Text2CAD auto proclaim themselves as the first AI framework for generating text-to-parametric CAD models using designer-friendly instructions for all skill levels.
- They introduce a data annotation pipeline for generating text prompts based on natural language instructions for the DeepCAD dataset using Mistral and LLaVa-NeXT
- The dataset contains 170K models and around 660K text annotations, from abstract CAD descriptions to detailed specifications
- They propose an end-to-end transformer-based auto-regressive network to generate parametric CAD models from input texts.
- They evaluated the performance of their model through a mixture of metrics, including visual quality, parametric precision and geometrical accuracy.

No system right now can generate good parametric CAD models from textual design descriptions. One primary challenge for developing such a system is defining suitable textual descriptions for parametric CAD generation, making it difficult to create deep learning methods that accurately convert these descriptions into precise CAD models.

PRIMARY CHALLENGE:

- The unavailability of the dataset
- A network to map the texts into CAD construction sequences

The big issue with Zoo is that the generated CAD models are obtained in the form of solid-body, and not decomposed to its intermediate sketch-and-extrusion steps as proposed by Text2CAD. Given raw numerical data of any parametric CAD model, current SOTA LLMs (GPT and Llama) may only derive procedural scripting codes for other APIs, such as FreeCAD or OpenSCAD to generate a model.

Such LLM augmented CAD generation approach will not be designer-friendly, not suitable for beginner-level designers, will not automate the development process in easy ways, and will restrict the re-usability of the scripts in case of complex shapes.

1. No training datasets are available that provide natural language-based design instructions as annotations for raw CAD construction sequences
2. Current datasets and generative models for CAD are limited and often not suited for developing knowledge-based CAD applications.
3. Some datasets focus solely on 2D sketch design and other popular ones like ABC, Fusion360 Gallery, Thingi10K and CC3D provide 3D meshes, BRep and other geometry or topology related annotations that are suitable for 3D modeling.
4. DeepCAD dataset, a subset of ABC, and Fusion360 provide CAD construction sequences in the form of sketch and extrusion to deduce design history. However,

CAD models may consist of numerous other types of operation beside extrusion and such construction sequences with other CAD operations are not available in the current datasets.

5. There is NO dataset available that provides textual design descriptions as annotations to create a conversational AI system for CAD modeling.

Text2CAD Data Annotation:

- They outline a process for generating textual annotations for DeepCAD dataset using LLMs and VLMs.
- These annotations describe the corresponding CAD construction workflow in human interpretable format.
- To enrich the DeepCAD dataset with textual annotations, they implement a two-stage description generation pipeline using the capabilities of both LLMs and VLMs.

The two stages:

- 1) Generating abstract shape descriptions using VLMs
- 2) Extracting multi-level textual instructions from LLM based on the shape descriptions and design details provided in the dataset.

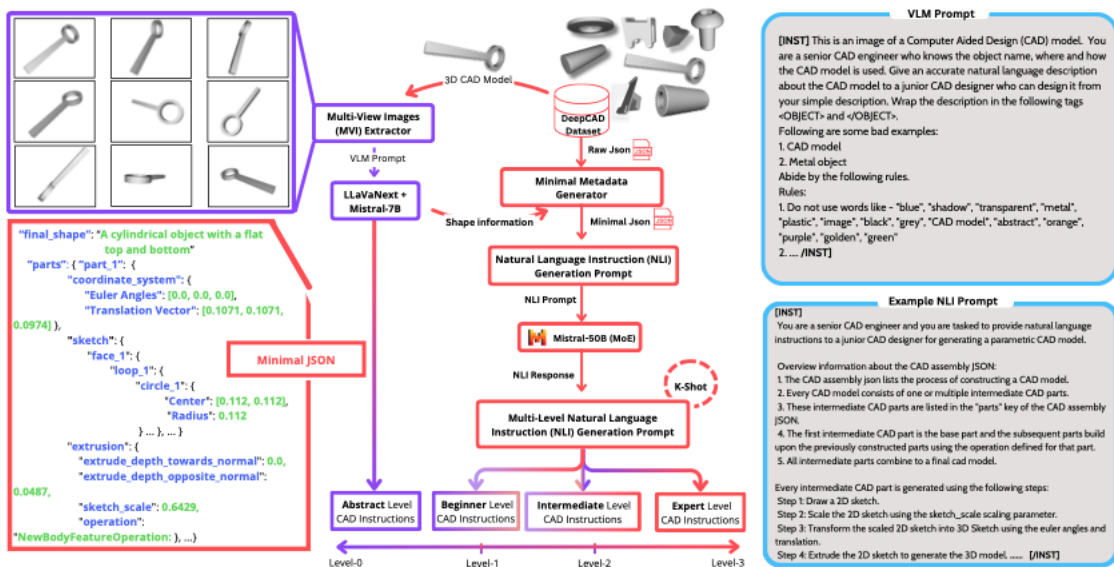


Figure 2: Text2CAD Data Annotation Pipeline: Our data annotation pipeline generates multi-level text prompts describing the construction workflow of a CAD model with varying complexities. We use a two-stage method - (Stage 1) Shape description generation using VLM (Stage 2) Multi-Level textual annotation generation using LLM.

The prompt is:

The CAD model consists of a cylindrical object with a flat top and bottom connected by a curved surface and slightly tapered towards the bottom. This object is created by first setting up a coordinate system, then sketching two concentric circles and drawing a closed loop with lines and an arc on a shared plane. The sketch is then extruded along the normal direction to form a solid body. The resulting part has a height of approximately 0.0487 units.

Shape descriptions using VLM: The initial step of their annotation generation pipeline involves generating abstract object-level descriptions of the CAD models using LLaVa-NeXT model. The objective of this step is to accurately capture the structural descriptions of the 3D shape, such as a 'ring-like structure', a 'cylinder' or 'a hexagon with a cylinder on top'. They generate shape descriptions for both the final CAD model and its intermediate parts. They first produce multi-view images from predetermined camera angles for each individual part and the final CAD model.

Multi-level design instructions using LLM: in this stage, multiple textual annotations corresponding to different design details of a CAD model are generated using Mixtral-50B through a series of steps. The DeepCAD dataset contains CAD construction sequences in JSON format.

They first preprocess the raw CAD construction sequences using a 'Minimal Metadata Generator' which replaces random, meaningless keys with more meaningful terms (*e.g.*, "part_1", "loop_1").

With the minimal metadata at hand, they then craft a prompt to generate detailed NLI (natural language instructions) ensuring a minimal loss of information from the minimal metadata.

language instructions (NLI) ensuring a minimal loss of information from the minimal metadata. Afterward, the NLI responses are refined by LLM using a k-shot [4] "Multi-Level Natural Language Instruction Generation Prompt" to generate multi-level instructions of different specificity and details. We categorize these levels as:

- Abstract level (L0): Abstract Shape Descriptions of the final CAD model extracted using VLM in the first stage.
- Beginner level (L1): Simplified Description - Aimed at laypersons or preliminary design stages, this level provides a simplified account of the design steps, eschewing complex measurements and jargon.

4

- Intermediate level (L2): Generalized Geometric Description - This level abstracts some of the details, providing a generalized description that balances comprehensibility with technical accuracy.
- Expert level (L3): Detailed Geometric Description with Relative Values - Here, the instructions include precise geometric descriptions and relative measurements, catering to users who require an in-depth understanding or are performing the CAD modeling task.

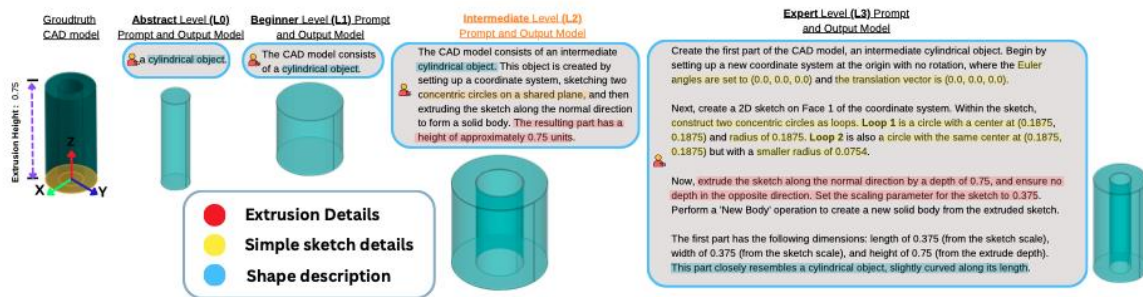


Figure 4: Parametric CAD model generation by Text2CAD transformer using different text prompts. Our text prompts follow a certain structure highlighting the different design aspects of CAD construction workflow (shown in different colors).

4-Level prompt taxonomy:

5. The dataset has four prompt difficulty/detail tiers:
 - **L0 (Abstract)**: coarse shape description.
 - **L1 (Beginner)**: simple procedural instructions.
 - **L2 (Intermediate)**: more geometric detail.
 - **L3 (Expert)**: detailed parametric/relative-value instructions.

The evaluation does utilize both parametric correspondence between the predicted CAD sequences and the input text prompts

*** Conducting visual inspections of the reconstructed 3D CAD models. Currently, there is no standardized benchmark for text-to-3D models that can be directly applied to this task.

Most practical approach is a metadata-first prompt factory (not manual writing!!!)

1. Ingest a curated set of STEP files and normalize units/orientation (mm, right-handed frame)
2. Auto-extract CAD metadata per part (OpenCascade/CadQuery); bbox, volume, surface types, hole counts/types, fillet/chamfer candidates, symmetry axes, key radii/lengths, component count
3. Store one canonical JSON record per part (this is the ground truth intent layer)
4. Generate prompt variants from that JSON using templates: L0, L1, L2, L3
5. Use an LLM only for paraphrasing/rewording, not for inventing geometry; force it to keep all numeric values exactly.
6. Run a prompt linter + validator: required sections present, required dimensions included, no contradictions, units/frame explicit, no forbidden ambiguity for deterministic track
7. Run automatic sanity replay: sample prompts → generate CAD → compare against reference; drop low-quality prompts
8. Package output as dataset rows (task_id, reference_step, prompt_level, prompt_text, expected_constraints, thresholds)

Use this schema:

```
{
  "part_id": "gear_040",
  "units": "mm",
  "frame": {"origin": [0,0,0], "z_up": true},
  "global_dims": {"bbox": [120,120,20]},
  "features": [{"type": "cyl_hole", "dia": 20, "through": true, "axis": "z", "pos": [0,0]}],
  "constraints": ["single_connected_solid", "mirror_symmetry_x"],
  "manufacturing_intent": ["machined_part"],
  "prompt_variants": {"L0": "...", "L1": "...", "L2": "...", "L3": "..."}
}
```

*ONE STEP FILE CAN YIELD MULTIPLE VALIDATED PROMPTS (4-12) WITHOUT MANUAL REWRITING

1 STEP part: extract one canonical metadata JSON

Generate 4 base prompts from it

For each level of complexity, create 1-3 controlled paraphrases (wording changes only, numbers fixed)

That can give up to 12 different prompts per STEP file

Run a linter to ensure each prompt keeps required fields and exact numeric values.

It means: keep geometry facts identical, only change wording style.

Example for one L2 mug prompt:

- Base: "Outer radius 40 mm, height 100 mm, wall thickness 5 mm."
- Paraphrase 1: "Create a cylinder (R40, H100) and hollow it to 5 mm wall thickness."
- Paraphrase 2: "Model a 100 mm tall cup with 40 mm outer radius and 5 mm walls."

What must stay fixed:

- All numeric values
- Units/frame
- Constraints/topology requirements

What can change:

- Sentence structure
- Verb choice ("create/model/build")
- Ordering of non-dependent statements

So "controlled paraphrase" = linguistic variation without changing engineering intent.

This is significant because it tests capability, not just "did the model memorise one phrasing".

Controlled paraphrases are significant for 5 reasons:

1. Measures prompt robustness: a good CAD model should produce similar geometry from equivalent wording
2. Reduces benchmark bias: one lucky/unlucky prompt wording won't dominate the score.
3. Improves statistical confidence: multiple equivalent prompts per part let you estimate variance and consistency
4. Separates language understanding from geometry skill: if numbers/constraints are fixed, failures indicate interpretation weakness, not missing specs.
5. Gives stronger evidence to customers: we can report "Model is stable across phrasing" which is more convincing than single-prompt pass/fail.

Example test:

- Intermeshed gears prompt

L0 (abstract intent only)

V1

Create a CAD model of two identical spur gears that mesh together.

V2

Model a pair of equal external gears in proper mesh.

V3

Build two matching spur gears engaged as a gear pair.

L1 (core specs, minimal structure)

V1

Create two identical external spur gears with module 4 and 40 teeth each, positioned so they mesh correctly.

V2

Model a meshing spur gear pair: both gears are external, $m=4$, $z=40$, equal size.

V3

Build two same-size external spur gears (module 4, 40 teeth each) in valid mesh.

L2 (clear geometry + placement)

V1

Units: mm. Create two identical external spur gears with $module=4$, $teeth=40$, $pressure\ angle=20^\circ$, and $face\ width=20$. Put Gear A at $(0,0,0)$ and Gear B at $(160,0,0)$ with parallel Z axes.

V2

Use millimeters. Model two equal involute external spur gears ($m=4$, $z=40$, $alpha=20^\circ$, thickness 20 mm). Set center distance to 160 mm along X so they mesh.

V3

Create a meshing gear pair in mm: both gears are external spur gears, 40T, module 4, 20° pressure angle, 20 mm face width. Place centers at $x=0$ and $x=160$.

L3 (full structured prompt template)

V1

Create a CAD model containing exactly two external involute spur gears.

Part identity:

Two identical intermeshed spur gears (Gear A and Gear B).

Units and frame:

Units in millimeters. Right-handed XYZ frame. Origin at center of Gear A axis. +Z is gear axis direction.

Base geometry:

For each gear, use involute external spur gear geometry with:

- Module = 4
- Number of teeth = 40
- Pressure angle = 20°
- Face width = 20
- No bore hole
- No hub or keyway unless required to satisfy geometry stability

Placement:

- Gear A center at (0, 0, 0)
- Gear B center at (160, 0, 0)
- Both gear axes parallel to +Z
- Center distance fixed at 160 mm (pitch radii tangent)

Constraints:

- Both gears must be identical
- Teeth must be phased to represent a valid engaged mesh state
- No self-intersections

Topology/output constraints:

- Exactly 2 disconnected solid components (not boolean-fused)
- Manifold solids
- Output as STEP.

V2

Create a two-component mechanical CAD part set.

Part goal:

Generate two identical external spur gears that are correctly intermeshed.

Units and coordinate frame:

Millimeters. Right-handed XYZ. Gear axes aligned with Z. Origin at Gear A shaft center.

Geometry definition:

Gear A and Gear B each must be involute spur gears with: $m=4$, $z=40$, $\alpha=20^\circ$, face width 20 mm, no bore.

Positioning:

Set Gear A center to (0,0,0) and Gear B center to (160,0,0). Maintain parallel Z axes and exact center distance of 160 mm.

Design constraints:
Equal geometry for both gears; valid involute tooth engagement; no floating entities; no geometric overlap violating meshing intent.
Topology/output:
Return exactly two separate watertight solids in one STEP output.

V3

Create a CAD model of an intermeshed spur gear pair with explicit parametric constraints.

Part identity:

Gear A + Gear B, both identical, external involute spur gears.

Units and frame:

mm units; right-handed XYZ; +Z is axial direction; origin at Gear A centerline.

Per-gear parameters:

Module 4, teeth 40, pressure angle 20°, face width 20 mm, solid body with no center bore.

Assembly placement:

Gear centers at (0,0,0) and (160,0,0), axis vectors both (0,0,1). Preserve exact pitch-center relation for meshing.

Functional constraints:

Correct tooth engagement, no self-intersection, no accidental fusion between gears, identical tooth/profile parameters across both gears.

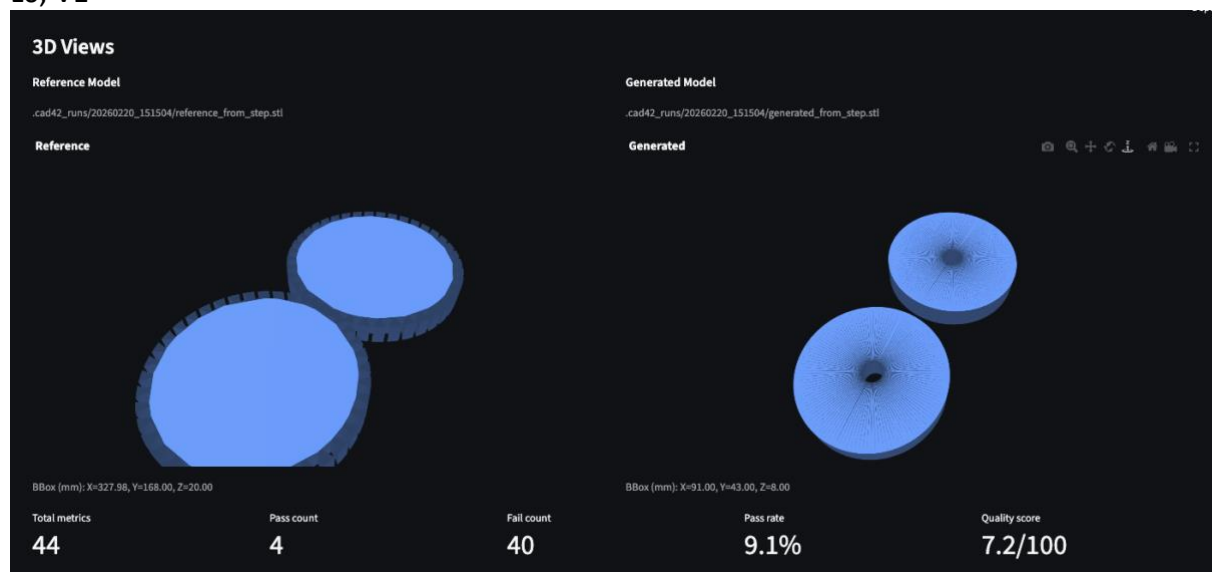
Output constraints:

Exactly two disconnected manifold solids. Exportable STEP geometry only.

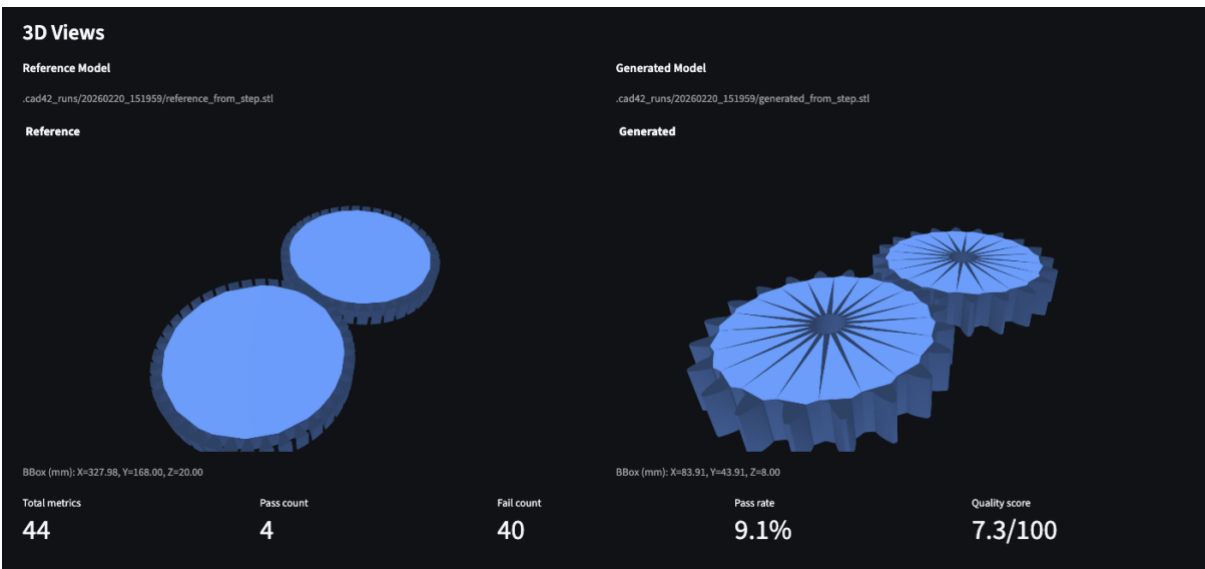
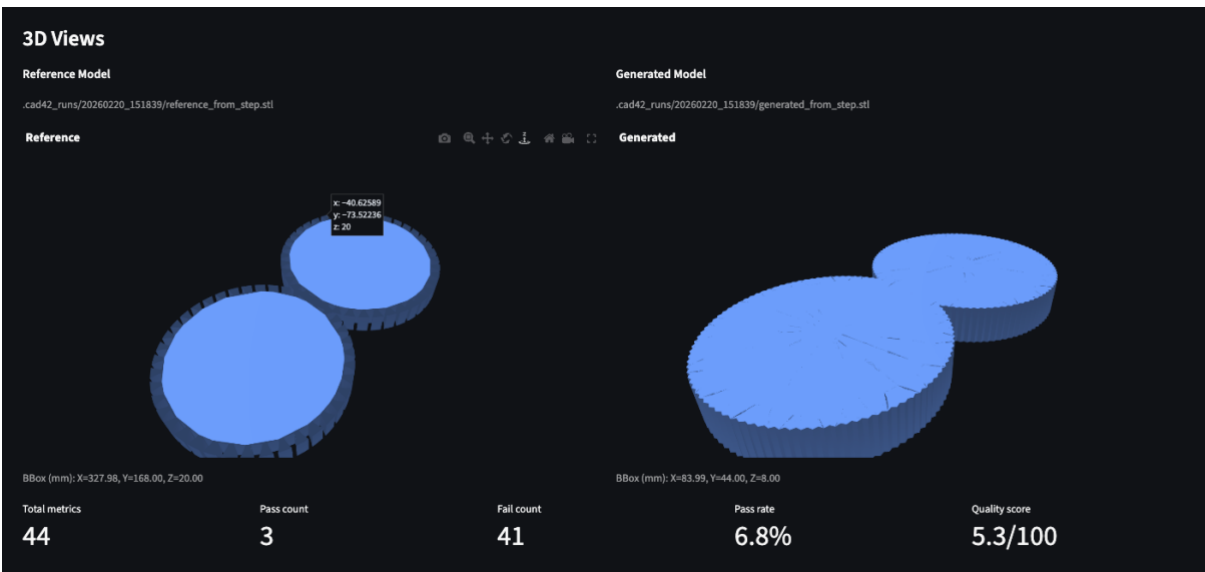
API KEY: sk-proj-tgDNTfnb7mD54OLH-BFPPiOjZQ4aNlOZYVdqj1c1i-RsfCBH0Us0B8x5kB_d29dg3uSbX2Ts_NT3BlbkfJl6JAPoP9apA1n_NQxKW91W5rAm3qoP2LDmN5QmJoEE4hncQdG7zuO2w566Q37HRAH1hjcODDoA

The tests below were run with 10000 sample points and 2.0 mm voxel pitch

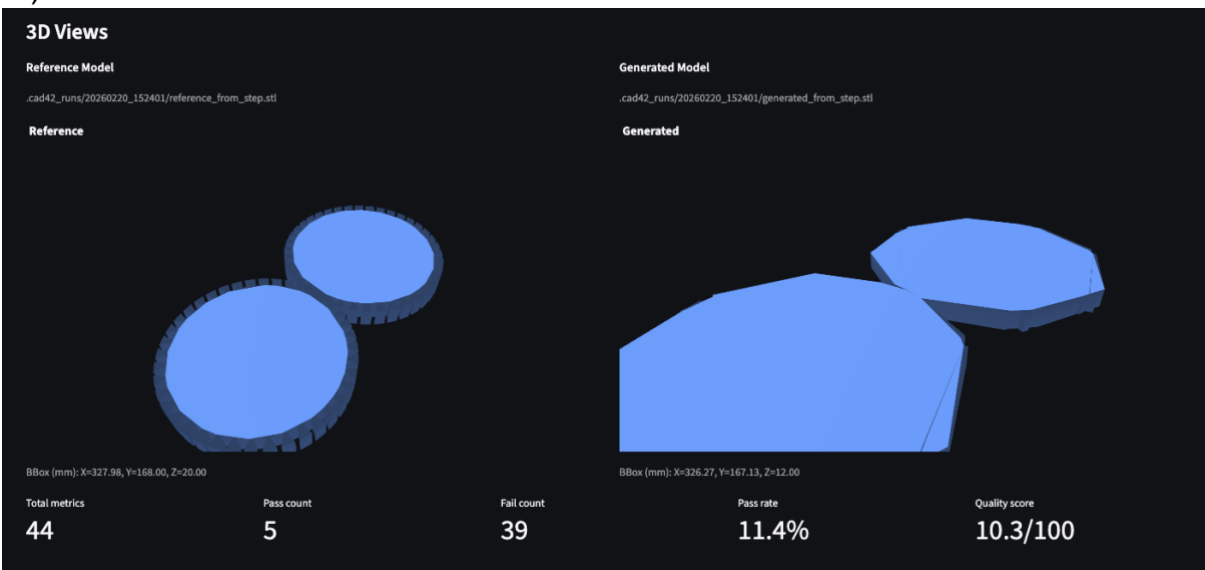
L0, V1



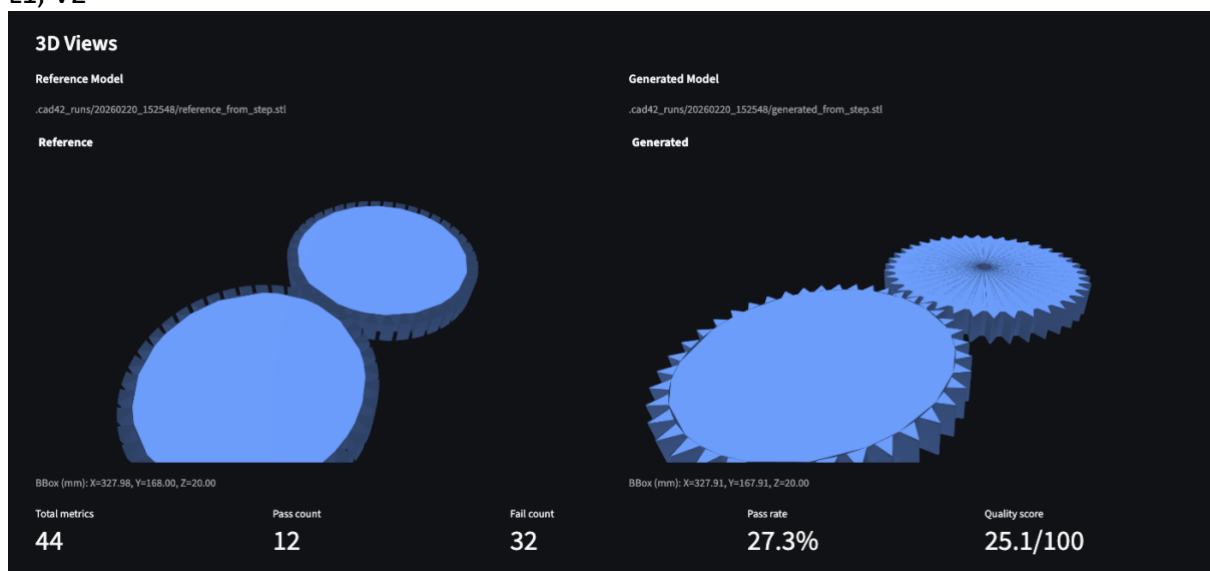
L0, V2



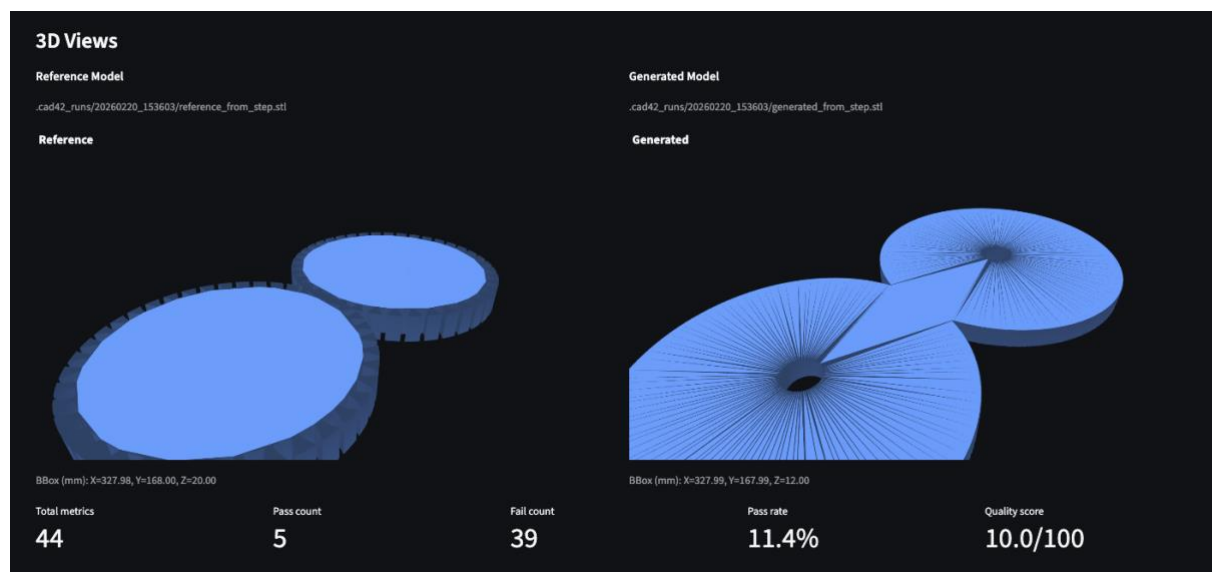
L1, V1



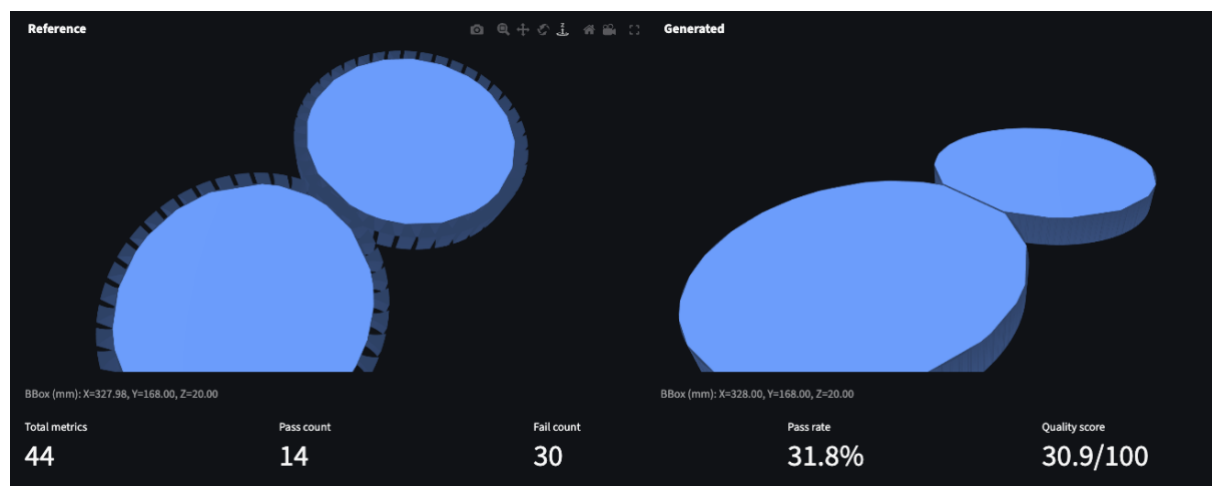
L1, V2



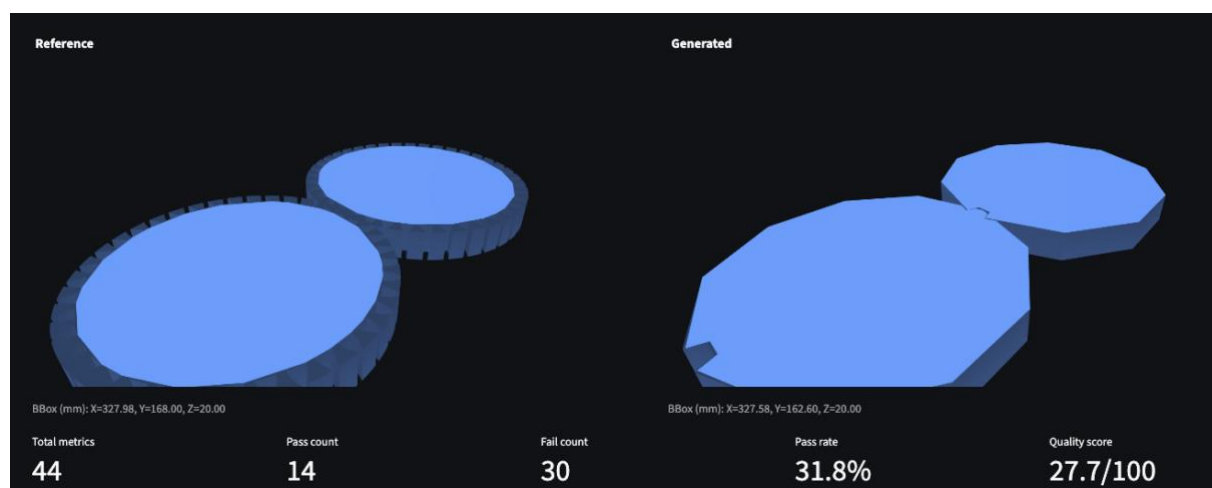
L1, V3



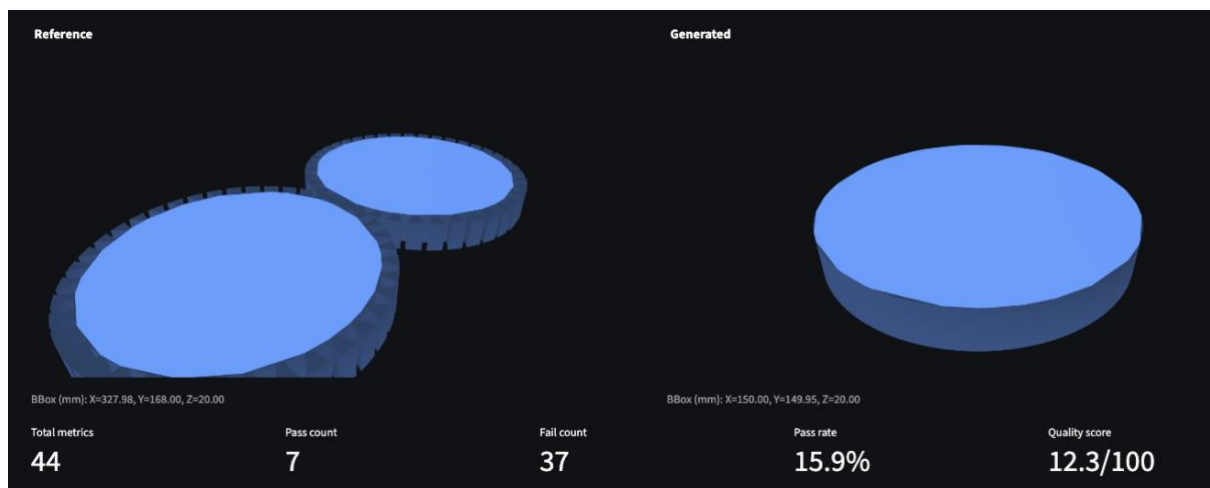
L2, V1



L2, V2

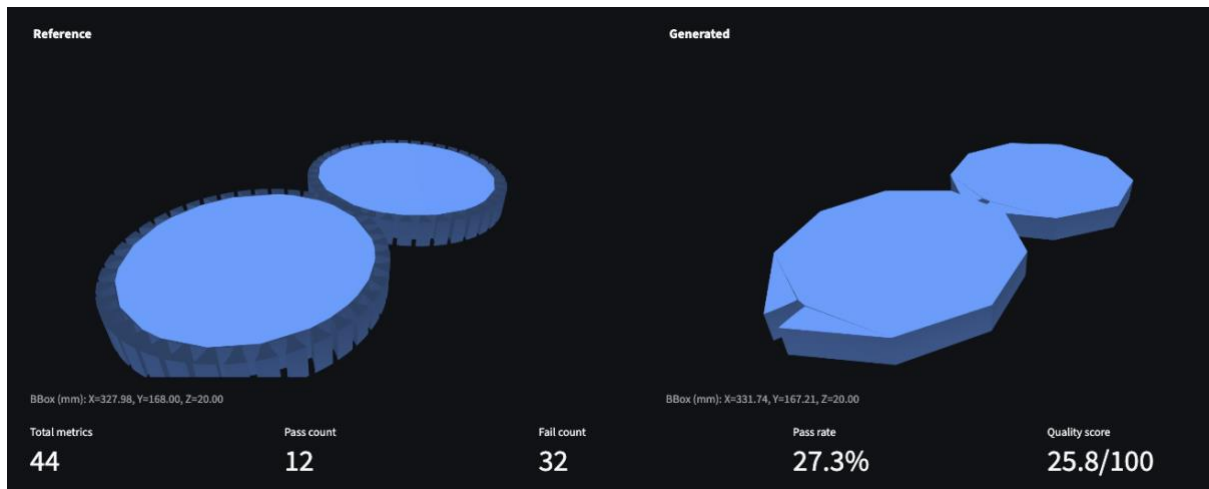


L2, V3

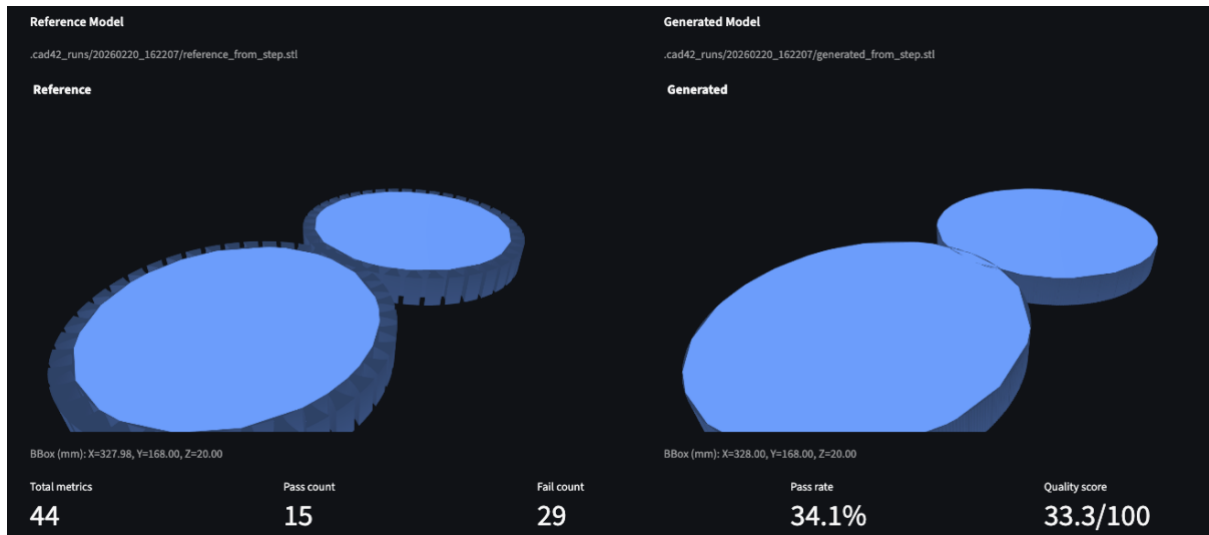


Only one “gear” was produced by the model for L2, V3 hence why the pass rate and quality score were so low.

L3, V1



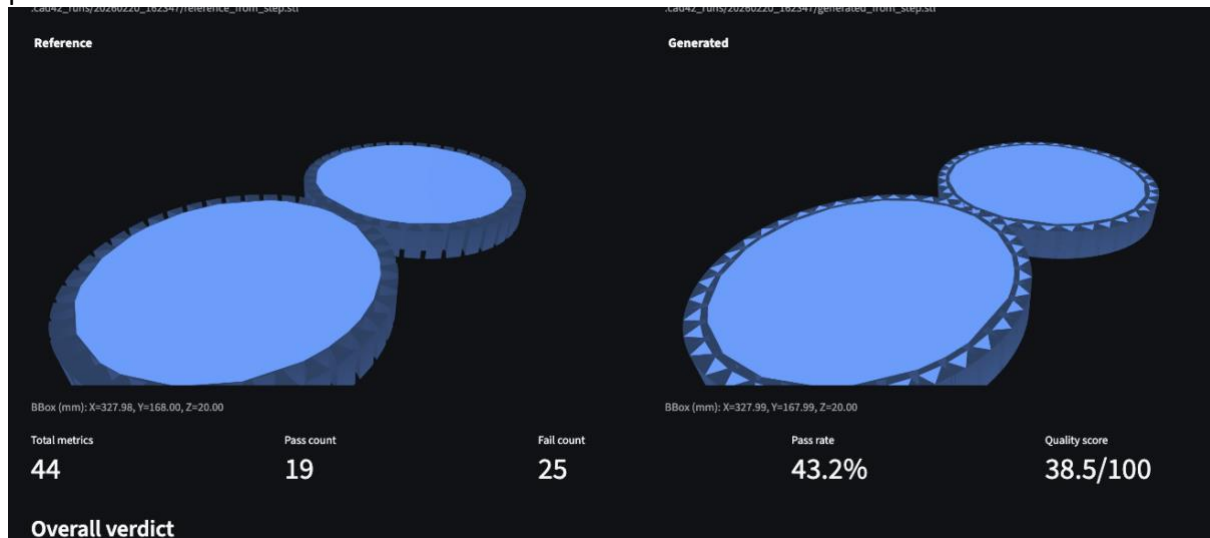
L3, V2



L3, V3

Point clouds are used to generate a mesh network, the mesh network is filled in with polygons which are rendered as pixels. Voxels are points that define a volume, and each

point is rendered.



Which metrics are impacted by topology?

If the topology is different, this means that the model has a different connectivity/shape structure, not just slightly different dimensions.

In practical terms, this means different number/type of holes/voids/tunnels, different face-edge-vertex connectivity, different genus/euler characteristics

The large gap means the generated mesh has many extra topological features (tiny holes/handles/non-manifold-like complexity).

Topology failures are not impacted by sample points size and voxel pitch.

Fast mode = true means that the sample points ≤ 4000 and voxel pitch_{mm} ≥ 4.0 .

These settings can affect sampled/discretised metrics:

- Chamfer/Hausdorff/point-to-surface
- normal, slice metrics, silhouettes, EMD
- voxel metrics

But topology-related metrics are computed from mesh topology directly, not sample points/voxel pitch: `component_count_match`, `watertight_manifold_pass`, `euler_genus_match`, `void_hole_count_match`

Sample points: 35000, Voxel pitch: 2.0mm

Sample points: 80000, voxel pitch: 2.0mm

Sample points: 80000, voxel pitch: 1.0mm

Use 2 outcomes:

1. Geometric fidelity:
 - Chamfer, Hausdorff (95/99), point-to-surface
 - Volume, bbox/critical dimensions
 - Cross-section/silhouette only if needed
2. Engineering validity:
 - Watertight/manifold
 - Component count
 - Euler/genus or hole-count consistency
 - Self-intersection

Then report:

- Pass rate
- Quality score
- Failure reason taxonomy

How to demonstrate trainability?

1. Structured: prompt + exact target representation (CadQuery/STEP) + constraints
2. Canonicalized: fixed units/ frame / origin / orientation
3. Diverse: many part families and feature combinations
4. Labeled for failure: not just success pairs, but error categories and repair traces?
5. Evaluable: deterministic scoring with reproducible thresholds

What makes it worth buying?

1. Unique data moat:
 - Hard-to-source industrial-style CAD tasks

- Clear provenance and rights
2. Signal quality:
 - Tight schema, low noise, no contradictory prompts
 - Multi-level prompts and explicit constraints
 3. Measured model lift:
 - Pre/post training improvement on held-out tests
 - Not only benchmark but include external-style stress tests

1. Generalization evidence

- Improvement on unseen part families, not just memorized templates.

2. Operational usability

- Easy ingestion format, stable versioning, changelog, and metadata.

How to show proof of trainability (this is the key)

Build a **Trainability Dossier** with controlled experiments:

1. Dataset split design

- Train/dev/test split by part family (not random row split).
- Leakage checks and de-duplication.

2. Baseline vs trained model

- Use at least one open model as a proxy.
- Run:
 - baseline (no training),
 - +SFT on your data,
 - +SFT with your highest-quality subset,
 - +SFT + repair-feedback data (if available).

3. Learning curve

- Show performance at 1k / 5k / 10k / 20k examples.
- Buyer wants to see slope still rising.

4. Ablations

- Remove key fields (constraints, tolerances, frame) and show degradation.
- This proves your annotation design matters.

5. Metrics report

- Pass@1, pass@k, topology pass, dimensional pass, composite quality.
- Report confidence intervals and replicate variance.

6. Cost-performance

- Show gain per 1k examples and per training dollar.

Minimum bar before pitching buyers

Aim to show:

1. **Statistically significant lift** on held-out test.
2. **OOD lift** (unseen parts) not just in-domain gain.
3. **Rights-clean data pipeline** and licensing docs.

4. **Reproducible scripts** to regenerate all results.

Lift on trainable proxies means: take a model we can fine-tune, train it on our CAD data and shows it performs better than before? That improvement is the lift.

Practical pipeline for project:

1. Input prompt + reference STEP
2. Generate k candidates from model (not just 1)
3. Run the 44 metrics against reference
4. Keep:
 - candidate code, metric vector, composite/quality score, failure tags (alignment, topology, dimensions)
5. For each task: mark best candidate, for failed tasks create one human-corrected CAD target
6. Build training datasets:
 - SFT set: prompt → corrected CAD code
 - Preference set: prompt, chosen candidate, rejected candidate using metric score ranking
 - Verifier set: prompt, candidate, metrics/features → pass probability
7. Fine-tune a trainable proxy model on SFT/preference data
8. Re-evaluate on a locked holdout task set
9. Report lift before v after with confidence intervals
10. After fail, either convert it into a negative/ranking example or add a corrected target and use it for generator training

Yes, but as a **targeted layer**, not the main scorer.

Use a hybrid setup:

1. Keep automated metrics as the primary score (fast, scalable, reproducible).
2. Add human review for:
 - borderline cases near threshold, metric disagreements (e.g., volume pass but geometry distance fail), and high-value samples (hard tasks).
3. Review only a sample (e.g., 10-20% of runs) to control cost.
4. Use a strict rubric (fit-for-function, missing critical feature, topology correctness, manufacturability risk).
5. Have 2 reviewers on overlap samples and track agreement (Cohen's kappa) so human judging is auditable.

How do i pick the thresholds without it seeming arbitrary too?

Use a **calibration protocol**, not intuition. Practical method:

1. **Define what "pass" means first**

- Example: "usable without geometry rework."
 - This is your target label, not the metric.
- 2. Build a labeled calibration set**
 - 200–500 model pairs.
 - Include: exact matches, near matches, wrong parts, topology failures.
 - 2 engineers label each pair: Pass / Rework / Fail (then resolve disagreements).
 - 3. Set thresholds from distributions**
 - For each metric, plot values for human-Pass vs Fail.
 - Pick threshold from data, e.g.:
 - $T = 95\text{th percentile of Pass}$ for lower-better metrics (Chamfer, Hausdorff).
 - $T = 5\text{th percentile of Pass}$ for higher-better metrics (IoU, fitness).
 - 4. Optimize against business cost**
 - Choose operating point by minimizing:
 - $\text{Expected Cost} = c_{FP} * FP + c_{FN} * FN$
 - If false-pass is expensive, bias stricter.
 - 5. Validate on unseen holdout families**
 - Freeze thresholds on calibration set.
 - Test on holdout parts/families to check generalization.
 - 6. Add confidence intervals**
 - Bootstrap threshold and report CI.
 - If unstable, metric is not robust enough for gating.
 - 7. Use 3 bands, not just binary**
 - Pass, Review, Fail (gray zone near threshold).
 - Prevents brittle decisions.
 - 8. Document every threshold in a table**
 - Metric, threshold value, dataset size, derivation rule, date/version, owner.
 - This is what makes it non-arbitrary and auditable.

How to decide the thresholds for the metrics?

1. Define the ground truth first
 - Human label each run as pass/rework/fail
2. Calibrate on a dedicated set
 - Use a stratified calibration dataset (easy, borderline, clear failures across part families)
3. Pick thresholds by objective, not percentile alone
 - For each metric, sweep candidate thresholds and compute confusion matrix vs human labels.

- Choose threshold that minimizes a cost function:

$$\text{Cost} = c_{FP} \cdot FP + c_{FN} \cdot FN$$

(where false-pass usually costs more than false-fail).

4. **Validate on unseen holdout**

- Freeze thresholds from calibration.
- Test on holdout families/runs only.

5. **Report uncertainty**

- Bootstrap confidence intervals for each threshold and for final pass-rate.

6. **Separate metric roles**

- Gate metrics (topology/manifold/component count/hard dimensions): strict thresholds.
- Quality metrics (Chamfer/Hausdorff/EMD etc.): continuous scoring.
- Diagnostic metrics: reported, not used for pass/fail.

7. **Add a review band**

- Instead of hard binary at one point, use Pass / Review / Fail around threshold to handle noise.

8. **Version and freeze**

- Publish thresholds_v1 and do not change mid-benchmark; recalibrate only in v1.1/v2.

Accepted foundations this follows:

- ROC/PR + cost-sensitive thresholding (standard ML evaluation practice).
- Decision rules with uncertainty (standard conformity assessment / metrology practice).

False pass (FP): benchmark says good but an engineer would say bad

False fail (FN): benchmark says bad but engineer would say good

1. **Cost function = penalty score**

You assign a penalty to each mistake type:

$$\text{Total Cost} = c_{FP} \cdot FP + c_{FN} \cdot FN$$

Then choose thresholds that make this total cost smallest.

3. **Why FP is often more expensive**

In your use case, a false pass can:

- Overstate model capability
- Let poor CAD into downstream work
- Damage trust with clients/buyers

A false fail usually means:

- You reject some good outputs
- You lose efficiency/time, but quality risk is lower

So many teams set $c_{FP} > c_{FN}$, e.g. 5:1 or 3:1.

4. **Tiny example**

- Threshold A: FP=2, FN=10

- Threshold B: FP=6, FN=3
If $c_{FP} = 5, c_{FN} = 1$:
- $\text{Cost}(A) = 5 * 2 + 1 * 10 = 20$
- $\text{Cost}(B) = 5 * 6 + 1 * 3 = 33$
So A is better, even with more FN.

- We want to act like a fair examiner
- For each model output, decide pass or fail and a quality score
- Thresholds must come from data, not guesses

End-to-end process:

1. Build tasks: reference parts + prompts
2. Run models many times (replicates): this creates many generated models
3. For each generated model, compare against reference: this creates one pair with metrics
4. Collect many pairs (300-800) for calibration
5. Human-review those pairs and assign labels
6. Use those labeled pairs to tune thresholds
7. Test tuned thresholds on unseen holdout families
8. Freeze thresholds as v1 and use them for benchmark reporting

Why 300-800 pairs?

- Thresholds should represent many cases:
- Different part families
- Different prompt levels
- Good outputs, borderline outputs, bad outputs

How does threshold tuning work?

- Pick one metric (e.g. Chamfer)
- Try many possible threshold values
- For each threshold, check against human labels
- Count mistakes: false pass, false fail
- Pick threshold with lowest mistake cost

Mistake cost formula:

$$\text{Total Cost} = c_{FP} \cdot FP + c_{FN} \cdot FN$$

If false pass is worse for your business, set c_{FP} higher.

Concrete example

1. Threshold A gives FP=2, FN=10.
2. Threshold B gives FP=6, FN=3.
3. If false pass is 5x worse: $C = 5 \cdot FP + 1 \cdot FN$

4. A cost = 20, B cost = 33.
5. Choose A.

Very simple "start tomorrow" plan

1. Create a CSV of labeled pairs (reference, generated, metrics, human label).
2. Label at least 300 pairs with two reviewers.
3. Run tuner to get thresholds.
4. Validate on holdout families.
5. Freeze as [thresholds v1.json](#).

1. Data decides values

- Don't hand-pick thresholds.
- Pick the values that best match human-labeled outcomes on calibration data.

2. Each metric must justify itself

- Keep a metric only if it adds predictive value on holdout data.
- If a metric is redundant/unstable, demote to diagnostic.

3. Freeze and validate

- Tune on calibration set.
- Validate on unseen families.
- Publish versioned thresholds and performance.

So you don't "know" beforehand which values are best.

You **measure** which values work best.

Practical simplification:

1. Use only 6–10 core metrics for final pass/fail.
2. Keep the rest as diagnostics.
3. Tune only the core set first.

That removes most of the complexity and avoids arbitrary overfitting.

Implemented. I added a full threshold-tuning pack in:

/Users/mejdi/Documents/New project/cad42_platform/threshold_tuning

Files created:

1. [build_pairs_from_runs.py](#)
Builds [pairs_for_labeling.csv](#) + [metrics meta.json](#) automatically from [.cad42_runs](#).
2. [auto_tune_thresholds.py](#)
Automatically tunes thresholds from labeled CSV (objective/cost-sensitive, NaN policy, metric subset support) and outputs:
 - [metric tuning report.csv](#)
 - [tuning summary.md](#)
 - [thresholds recommended.json](#)
 - [threshold overrides.json](#) (ready for STEPScore UI)
3. [pairs_labeled_template.csv](#)
CSV template with all 44 metric columns.
4. [README.md](#)
Overview and workflow.

5. [labeling_guide.md](#)
How to label positive/negative/review.
6. [run_commands.md](#)
Copy-paste commands end-to-end.

Also linked this from:

- [README.md](#)

Quick start:

```
cd "/Users/mejdi/Documents/New project/cad42_platform"
```

Then label [pairs for labeling.csv](#), then run:

Why are 300-800 pairs needed?

$$n \approx \frac{Z^2 p(1-p)}{d^2}$$

where p is expected sensitivity/specificity (or pass-rate component), d is desired margin of error, $Z=1.96$ for 95% CI.

Examples:

- $p=0.90$, $d=0.05$ -> ~138 positives needed
- $p=0.85$, $d=0.05$ -> ~196 negatives needed
- $d=0.04$ pushes this to ~216 / ~306

- Tune threshold on a calibration set to optimize an objective (F1, balanced accuracy, cost-sensitive objective)
- Validate on a separate holdout set
- Use ROC-based cutpoint methods or cost/utility-based methods
- Avoid tuning and evaluating on the same data!!!

Practical recommendation:

- Pilot testing: 200 labeled pairs
- v1 credible calibration: 400-600 labeled pairs
- Stronger evidence: 700-900 pairs for tighter confidence intervals + more family stratification
- Heavy imbalance should be avoided
- 40-60% positive and same for negative
- At least 100 positives and 100 negatives for stable initial tuning
- Negative should mean overall unusable/fail by the human rubric, not one metric fail
- Positive = usable CAD output (passes human acceptance criteria)
- Negative = unusable or requires substantial rework
- Review = borderline/uncertain

***Negative should not be defined as “single metric fail” because this creates circular tuning (metrics labeling themselves).

Define usable CAD as:

1. Correct part intent (right feature set/topology class).
2. Within dimension tolerances for critical dimensions.
3. Single valid solid (or required component count), watertight/manifold.
4. No blocking geometry defects (self-intersections/non-manifold artifacts).
5. Requires no major geometric rebuild for intended use.

Then label with fixed categories:

1. Pass: meets all acceptance checks.
2. Rework: minor edits only.
3. Fail: major rebuild needed.

For threshold tuning:

- Use positive = Pass
- Use negative = Fail
- Keep Rework as separate or map to negative depending on strictness.

So yes, every benchmark has a judgment layer, but it becomes defensible when:

- criteria are explicit,
- raters follow the same rubric,
- agreement is measured (e.g., kappa),
- and thresholds are fitted to those fixed labels.

Voxel pitch size cannot be smaller than 0.50 or else:

Great question. Two parts:

1. What these metrics mean

- `occupancy_precision`

Of all voxels occupied by **generated**, how many are also occupied by **reference**.

Formula: $|A \cap B| / |B|$

- `occupancy_recall`

Of all voxels occupied by **reference**, how many are captured by **generated**.

Formula: $|A \cap B| / |A|$

- `occupancy_f1`

Balance of precision and recall.

Formula: $2PR / (P + R)$

Here A = reference occupied voxel set, B = generated occupied voxel set.

2. Why they can be 0 (or NaN)

- They are 0 when overlap is essentially none ($|A \cap B| \approx 0$) or one voxel set is empty.
- They are NaN when voxelization is skipped by the memory guard.

For your newest runs, they are actually **NaN**, not 0:

- 20260226_151815, 20260226_151348, 20260226_151117 all show nan.
- Reason: voxel grid estimate is huge at `voxel_pitch_mm=0.1` (e.g. ~987M voxels), above the code cap (8M), so code sets voxel metrics to NaN.

If you want these to compute, increase voxel pitch (e.g. 0.5 or 1.0) or use fast mode.

CLAUDE API KEY:

sk-ant-api03-3x68-0n7lWSQSDeQsLY4C1Jypl8hAcOBp-MoC-
W4N02n_cbliZcHGdmSw48_ASvcBSjnf-qOrHBa-laB3NQE2w-F8ZgPQAA

OPENAI API KEY:

sk-proj-tgDNTfnb7mD54OLH-BFPPiOjZQ4aNI0ZYVdqj1c1i-
RsfCBH0Us0B8x5kB_d29dg3uSbX2Ts_NT3BlbkFJl6JAPoP9apA1n_NQxKW91W5rAm3qoP2LD
mN5QmJoEE4hncQdG7zuO2w566Q37HRAH1hjcODDoA

Monday 9th March:

Week Goal

Ship a buyer-ready v1 benchmark package: stable harness, tuned thresholds, validation results, and clean documentation.

1. Day 1: Lock Scope + Stability

- Freeze grading profile to a core set (use your refined profile or the tighter core set).
- Freeze run settings preset (`fast_mode`, `sample_points`, `voxel_pitch_mm`) for calibration.
- Confirm harness runs end-to-end on 10 jobs.

	Deliverables: run_config_v1.json, updated metric policy doc in /Users/mejdi/Desktop/ISIDOR/New project/cad42_platform.
2. Day 2: Run Calibration Batch	- Execute harness_manifest_600_calibration.csv. - Target: high completion, no crash loops, no missing artifacts. - Deliverables: run folder in /Users/mejdi/Desktop/ISIDOR/New project/cad42_platform/.stepscore_harness_runs.
3. Day 3: Label Calibration Pairs (Sprint 1)	- Extract rows with build_pairs_from_runs.py. - Label first half (usable_label: positive/negative) with short notes. - Deliverables: pairs_calibration_labeled.csv draft.
4. Day 4: Labeling Complete + Tune	- Finish labels. - Run auto_tune_thresholds.py with cost objective (fp_cost > fn_cost). - Deliverables: thresholds_v1.json, metric_tuning_report.csv, tuning_summary.md.
5. Day 5: Validate + Blind Test + Pitch Pack	- Run validation and blind manifests with thresholds_v1. - Produce one summary table: pass rate, false pass rate, family breakdown, top failure modes. - Deliverables: validation_report_v1.md, blind_report_v1.md, benchmark_method_v1.md.
Definition of Done (end of week)	- Frozen thresholds (v1) + reproducible config. - Separate calibration vs blind results. - One concise buyer-facing methods/results pack.

The labels are simple ground-truth judgments for each (reference, generated) pair.

Use one label per pair:

- positive = usable/acceptable output
 - negative = not usable/unacceptable output
 - review = ambiguous, skip for tuning
-
- positive: geometry is functionally correct and within the intended tolerance
 - negative: wrong topology, missing key features, major dimension drift, bad manufacturability
 - review: borderline case we don't want influencing thresholds yet

Then, the tuner finds metric thresholds that best separate positive from negative according to my judgment. Without labels, threshold tuning is arbitrary.

Model choice can bias tuning, do not tune from one model only!!

1. Generate candidates from multiple sources:
 - openai, anthropic, scripted perturbations? (non-AI negatives)
2. Balance contributions:
 - keep similar counts per source/family/prompt level
 - don't let one model dominate rows
3. Tune on mixed calibration set
 - label overall positive/negative
 - run threshold tuner on the mixed set
4. Validate robustness:
 - re-run scoring on held-out sets per model
 - check that one model isn't disproportionately favoured

Scripted perturbations = automatically edited variants of a reference CAD/mesh to create controlled 'wrong' outputs.

Example:

- scale on axis by +3% (x only)
- shift a hole center by +1.5mm
- change hole diameter by +0.8mm
- remove one fillet/chamfer
- delete one feature
- add small rotation misalignment

Useful as we know exactly what changed, it gives non-AI negatives for threshold tuning, reduces bias to any single model's failure style

also won't the results of the generated step files and consequently the threshold tuning depend on how good my prompts are?

Example: removing a fillet

- edge_chamfer_mm increases: filleted edges become sharper/different edge loci
- curvature_distribution_error increases: curvature histogram shifts
- normal_angle_error_deg worsens /normal_consistency drops
- chamfer_distance_mm and hausdorff may increase
- bbox often unchanged
- volume_diff and surface_area_diff change a little unless fillets are large or many

Possible parts to generate for the calibration pairs:

- Stepped shafts
- Simple box holes
- Ring spacer
- L-bracket
- Flange
- Slotted plate
- Shafts with fillets/chamfers
- Shafts with keyways
- Shafts with gears

Using Claude Opus 4.6, a seemingly simple task resulted in 21 metrics failing out of 44. This needs to be refined as the label positive would be given to this task. The thresholds have not been tuned yet and when going back to this task later, the model should succeed.

Tuesday 10th March

1. Core prismatic
 - Box with through hole
 - Box with blind hole
 - Slotted plate
 - Counterbore plate
 - Countersink plate
 - Pocketed plate?
 - Multi-hole bolt pattern plate
 - L-bracket
 - Ribbed bracket (multi-rib)
 - U-channel bracket
 - T-slot block
 - Mounting foot block
2. Shaft/rotational
 - Plain shaft
 - Stepped shaft
 - Keyed shaft
 - Shaft with snap-ring grooves
 - Shaft with cross-drilled hole
 - Shoulder shaft with fillets
 - Spacer ring
 - Flanged bushing
 - Sleeve with internal bore
 - Hub with keyway
3. Shell/Thin-wall
 - Open-top shell box
 - Shell enclosure with standoffs?
 - Shell with mounting bosses
 - Shell with vent slots
 - Shell with cable cutout
 - Shell with gasket lip
 - Two-piece enclosure half
 - Thin ribbed shell panel
 - Shell with internal partition
4. Revolved + Mixed
 - Flange disk with bolt circle
 - Flange with hub and bore
 - Simple wheel hub
 - Spacer with chamfered edges
 - Cap with internal cavity
 - Nozzle-like reducer
5. Gears

- Spur gear coarse
 - Spur gear fine
 - Gear blank with simplified teeth
 - Double-gear on shared hub?
 - Gear with lightening holes
 - Rack segment proxy
 - Helical-like proxy
6. Off-axis / Multi-setup
- Block with 30 degree hole
 - Block with 45 degree pocket
 - Off-axis boss on inclined plane
 - Skewed slot feature
 - Compound-angle drill pair
 - Oblique rib feature
7. Draft / Manufacturing style
- Drafted box with core pocket
 - Drafted boss + fillets
 - Drafted rib network
 - Thin-walled drafted cup
 - Drafted bracket
 - Drafted housing half
8. Feature-stress variants
- High fillet-count block
 - High chamfer-count block
 - Many small holes plate
 - Dense slot pattern
 - Tiny feature near edge
 - Near-tangent pocket/fillet case
 - Thin wall + large fillet conflict case
 - Small-radius + large-body mix
 - Close-spacing holes stress case
9. Assemblable single-part fit features
- Dowel-pin hole pattern plate
 - Mating boss-hole test part
 - Press-fit bore variant
 - Clearance-hole variant block
 - Slot + bolt pattern mount
 - Bearing seat with shoulder
 - Flange pilot fit feature
 - Locator notch feature

Great question. Let me lay out the threshold tuning situation clearly.

Where You Are Now

You tuned 45 metric thresholds on **147 manually labeled pairs** (93 positive, 54 negative). Top result: avg F1 = 0.874. That's a decent start, but 147 pairs from a single model (Claude) on a single run is **not robust enough** to sell as a commercial benchmark.

The Core Problem with Current Tuning

Overfitting risk is high. Here's why:

1. **Small sample** — 147 pairs is thin. A single metric like `surface_area_diff_percent` achieving F1=0.989 on 147 samples could easily drop to 0.85 on new data.
2. **Single model bias** — All 147 pairs come from Claude Opus. A model that fails differently (e.g., GPT making different geometric errors) could invalidate your thresholds entirely.
3. **Single prompt set** — The prompts have known gaps (keyed shaft angular position, etc.), so some "failures" are really prompt failures, not genuine bad geometry. Your thresholds learned to penalize prompt artifacts, not actual quality.
4. **No train/test split** — You tuned AND evaluated on the same 147 pairs. There's no held-out validation set to confirm the thresholds generalize.

What Robust Threshold Calibration Actually Needs

Step 1: More labeled data (at minimum ~500 pairs)

- Multiple models (Claude, GPT, Gemini, open-source)
- Multiple prompt levels (L2 and L3)
- All 22 families well-represented
- You already have the 600-package manifest ready with 150 unique parts × multiple models

Step 2: Train/validation/test split

- **Calibration set (~60%)**: Tune thresholds on this
- **Validation set (~20%)**: Check for overfitting during tuning
- **Blind set (~20%)**: Final score, never touched during tuning. This is what you report to customers

Your 600-package manifest already has this

split: `harness_manifest_600_calibration.csv`, `harness_manifest_600_validation.csv`, `harness_manifest_600_blind.csv`

Step 3: Cross-validation on the calibration set

- Instead of a single threshold sweep, do k-fold (e.g., 5-fold) cross-validation
- For each metric, pick the threshold that has the best **average** F1 across all folds
- This prevents overfitting to a specific subset of pairs

Step 4: Per-family thresholds (optional but powerful)

- A `chamfer_distance_mm` threshold of 1.0 might be too tight for complex parts (`shell_enclosure`) but too loose for simple parts (`ring_spacer`)
- Consider family-specific threshold overrides for the most impactful metrics

The Immediate Next Steps

1. **Fix the prompts** (the rewrite I was about to do) — so the next run produces cleaner labels
2. **Generate more reference parts** — scale from 73 to 300+ unique parts across all 22 families
3. **Run the harness against multiple models** — Claude + at least one other (GPT, Gemini)
4. **Label a larger set** (~500+) using your 3D viewer
5. **Re-tune with proper cross-validation and a held-out blind set**

Do you want me to focus on any of these specific steps first? The prompt fix and part generation are prerequisites for everything else, but if you want to improve the tuning logic itself right now, I can

rewrite `run_threshold_tuning.py` with k-fold cross-validation and a proper train/validation split.

Got it. Let's focus on what matters — getting robust thresholds.

What You Can Generate Right Now

You have **two dataset generators** already built:

Basic families ([generate_parametric_dataset.py](#)) — 9 families

Family	Description	Current parts
box_hole	Block + centered hole	9
stepped_shaft	3-step coaxial shaft	9
flange	Disk + bore + bolt holes	9
ring_spacer	Hollow cylinder	9
slotted_plate	Plate + stadium slot	9
l_bracket	L-shape + holes	9
pillow_block	Block + bore + 4 mounting holes	8
pulley	3-step cylinder + bore	8
u_channel	Open-top channel + holes	7

Advanced families ([generate_advanced_dataset.py](#)) — 13 families

Family	Description	Current parts
manifold_block	Block + orthogonal channels + blind hole	7
keyed_shaft_hub	Shaft + hub + keyway	6
shell_enclosure	Shelled box + bosses + cutout	6
counterbore_countersink_plate	Plate + cbore/csink holes	6
ribbed_bracket	L-bracket + triangular ribs	6
threaded_parts_male	Shaft + external helix	7
threaded_parts_female	Sleeve + internal helix	6
loft_transition	Circle-to-rectangle loft	6
sweep_path_part	Tube swept along S-curve	6
drafted_part	Tapered block + hole	6
helical_features	Shaft + helical flight	6
chamfer_fillet_stress_cases	Block + fillets + chamfers	6
off_axis_features	Block + tilted boss + tilted hole	6

Total: 22 families, ~150 unique parts currently.

You can scale each generator to produce as many variants as you want — they're parametric with random seeds. To get 500+ labeled pairs for robust calibration, you'd want roughly **25-30 parts per family**.

What do you want to do — should I scale up the part generation to 500+, or should I first rewrite the tuning script with proper cross-validation and train/test splits so the pipeline is ready when the data comes in?

Step 2: Train/validation/test split

- Calibration set (~60%): Tune thresholds on this
- Validation set (~20%): Check for overfitting during tuning
- Blind set (~20%): Final score, never touched during tuning. This is what you report to customers, this all sounds like machine learning. would this count as a machine learning problem?

Yes, absolutely — this **is** a machine learning problem, even though there's no neural network involved.

What you're doing is:

- Taking **input features** (44 geometric metrics)
- Learning **decision boundaries** (thresholds) from **labeled data** (your 147 human-labeled pairs)
- To **predict** a binary outcome (pass/fail)

That's literally a binary classification task. Your threshold tuning is just a very interpretable version of what a classifier does. The overfitting risk, the need for train/test splits, cross-validation — it's all the same fundamentals.

The difference is you're using **per-metric univariate thresholds** instead of something like a random forest or logistic regression that combines all 44 metrics together. Which is actually a smart design choice for a benchmark product — your customers can understand and audit "chamfer_distance must be $\leq 1.2\text{mm}$." They can't audit a black-box model.

But it does raise a question: **have you considered using an actual ML classifier alongside the thresholds?**

For example:

- A logistic regression or gradient-boosted tree trained on your 44 metrics could give you a single **quality probability score** that's more accurate than any individual threshold
- You keep the per-metric thresholds for interpretability and transparency (customers see exactly which metrics failed)
- But you add a combined ML score as an overall "StepScore confidence" that weighs all metrics together
- Some metrics matter more than others, and metric interactions matter (e.g., high chamfer distance is fine if voxel IoU is still high)

The thresholds tell you **what** failed. The ML model tells you **how bad** the overall result actually is.

Would that be useful for your product, or do you want to keep it purely threshold-based?

1. A Combined ML Score Is More Accurate Than Individual Thresholds

Right now your system does this: if ANY single metric fails its threshold, the whole part fails. That's a hard vote — 44 independent pass/fail checks.

The problem is that metrics are **correlated and redundant**. `chamfer_distance_mm`, `hausdorff_95p_mm`, and `hausdorff_99p_mm` all measure roughly the same thing — surface deviation. If one is slightly over threshold due to mesh noise but the other two pass, the part is probably fine. But your current system still fails it.

A trained classifier sees all 44 metrics **simultaneously** and learns patterns like:

- "When `chamfer_distance` is 1.1 (slightly over threshold) but `voxel_iou` is 0.96, humans always label this positive"
- "When `surface_area_diff` is 5% but `render_ssim` is 0.95, the visual match is still good"

This is fundamentally more powerful because it handles **borderline cases** — which is exactly where threshold-based systems struggle and where your customers will complain.

For StepScore specifically: Your current 0.874 average F1 means roughly 1 in 8 predictions is wrong. A combined model would likely push that to 0.93+ because it can handle the ambiguous middle zone that individual thresholds misjudge.

2. Per-Metric Thresholds Stay for Transparency

AI companies buying your benchmark will ask: "Why did this part fail?"

If you only give them a black-box score of 0.43, that's useless to them. They can't debug their model. They can't figure out if their geometry is wrong or their topology is wrong or their scaling is off.

The per-metric thresholds give them a **diagnostic breakdown**:

- ❌ `chamfer_distance_mm` = 2.3 (threshold: 1.2) → surface geometry is off
- ❌ `silhouette_iou` = 0.71 (threshold: 0.92) → shape outline doesn't match
- ✅ `volume_diff_percent` = 0.8% (threshold: 2.0%) → volume is correct
- ✅ `bbox_error_total_mm` = 0.3 (threshold: 1.5) → bounding box is right

Now the customer knows: "My model gets the overall dimensions right but the detailed surface shape is wrong." That's actionable information they can use to improve their training data or fine-tuning.

For StepScore specifically: This is your product's main selling point. No other benchmark gives this level of per-metric interpretability for CAD generation. The ML score doesn't replace this — it complements it.

3. A Weighted Confidence Score Captures Metric Importance

Not all 44 metrics are equally important. Right now they're treated equally — `bbox_error_axis_x_mm` failing has the same weight as `voxel_iou` failing. But in reality:

- `voxel_iou` at 0.85 is a much bigger deal than `bbox_error_axis_x_mm` at 1.6mm
- `render_ssim` at 0.88 (just under 0.90 threshold) is cosmetically fine for most applications
- `topology_genus_match` being wrong (genus 1 vs genus 0) means there's a hole that shouldn't exist — that's catastrophic for manufacturing

A logistic regression would learn coefficients like:

```
score = sigmoid(  
    -3.2 * voxel_iou_failed  
    -2.8 * chamfer_distance_failed  
    -2.1 * topology_genus_failed  
    -0.4 * render_ssim_failed          ← less weight, cosmetic  
    -0.2 * bbox_error_x_failed        ← almost irrelevant alone  
    ...  
)
```

Those weights tell you which metrics **actually predict human judgment**. That's incredibly valuable information for your product — you can tell customers "these are the metrics that matter most for CAD quality" backed by data, not opinion.

For StepScore specifically: You could offer tiered quality levels. "Manufacturing-ready" requires high scores on structural metrics (topology, volume, chamfer). "Visually acceptable" only requires render and silhouette metrics. The ML weights tell you which metrics belong in which tier.

4. Metric Interactions Are Real and Your Thresholds Miss Them

This is the most important point. Consider two scenarios:

Scenario A — Keyed shaft with keyway rotated 90°:

- `voxel_iou` = 0.85 (pass — volume mostly overlaps)
- `silhouette_iou` = 0.08 (fail — outline completely different from one view)
- `chamfer_distance` = 1.8 (fail)
- `volume_diff` = 1.2% (pass — same amount of material)

Your threshold system says: 2 fail, 2 pass. But a human looks at this and says "totally wrong part" — the keyway is in the wrong place. The **combination** of high `voxel_iou` + catastrophically low `silhouette_iou` is the signature of a rotational error. No individual threshold captures that pattern.

Scenario B — Box with hole, slightly undersized:

- `voxel_iou` = 0.91 (pass)

- silhouette_iou = 0.94 (pass)
- chamfer_distance = 1.3 (fail — slightly over 1.2 threshold)
- volume_diff = 3.1% (fail — slightly over 2.0% threshold)

Your threshold system says: 2 fail. But a human looks at this and says "close enough, acceptable." The part is just 3% smaller overall — it's a uniform scaling error, not a geometric mistake.

A gradient-boosted tree would learn:

- Scenario A pattern → always negative (rotational misalignment)
- Scenario B pattern → usually positive (minor scaling, shape is correct)

For StepScore specifically: Your keyed shaft problem is exactly Scenario A. The metrics individually don't explain why it fails — it's the interaction between volume metrics (which look okay) and visual metrics (which are catastrophic) that reveals the rotational error. An ML model learns this automatically from your labeled data.

How This Would Work in Practice

Your StepScore output per part would look like:

```
StepScore Quality: 0.82 (Acceptable)    ← ML combined score
Grade: B                                ← derived from score

Metric Breakdown:                        ← threshold-based diagnostics
✅ Volume:      1.2% diff (threshold: 2.0%)
✅ Bounding Box: 0.4mm error (threshold: 1.5mm)
⚠ Surface:      1.3mm chamfer (threshold: 1.2mm)
❌ Visual:       0.71 silhouette IoU (threshold: 0.92)
✅ Topology:     genus match ✓
```

The ML score captures the overall picture. The thresholds give the diagnostic detail. Customers get both.

Want me to implement this? It would be a logistic regression or XGBoost trained on your 147 labeled pairs as a starting point, with the architecture ready to retrain as you scale to 500+ labels.

Full Guide: ML-Powered StepScore Scoring System

The Architecture

You'll have two layers working together:

Layer 1: Per-Metric Thresholds (diagnostic)

- 44 individual pass/fail checks
- Tells the customer WHAT failed and WHY
- Interpretable, auditable

Layer 2: ML Combined Score (predictive)

- Single model sees all 44 metrics at once
 - Outputs probability 0.0 - 1.0
 - Tells the customer HOW GOOD the overall result is
 - Captures metric interactions and weights
-

Step 1: Prepare Your Data

What you have now:

```
labeled_pairs_for_review.xlsx
├── 147 rows
├── 44 numeric metric columns
├── 1 label column (positive/negative)
└── metadata columns (part_id, family, prompt_level, etc.)
```

What you need to do:

a) Extract the feature matrix (X) and labels (y)

```
import pandas as pd
import numpy as np

df = pd.read_excel("labeled_pairs_for_review.xlsx")

# Your 44 metric columns (the features)
METRIC_COLS = [c for c in df.columns if c not in [
    "pair_index", "part_id", "family", "prompt_level",
    "provider", "model", "reference_path", "generated_path",
    "label", "quality_score_0_100"
]]

X = df[METRIC_COLS].copy()
y = (df["label"] == "positive").astype(int)  # 1 = good, 0 = bad
```

b) Handle missing values and infinities

Some metrics might be NaN or inf for parts where computation failed. ML models can't handle these raw.

```
X = X.replace([np.inf, -np.inf], np.nan)
```

```
# Option 1: Fill NaN with the worst possible value for that metric
# (makes the model treat missing data as a failure signal)
for col in X.columns:
    if y[X[col].notna()].mean() > 0.5:
        # Higher values tend to be "good" → fill missing with 0
        X[col] = X[col].fillna(0)
    else:
        # Lower values tend to be "good" → fill missing with a large number
        X[col] = X[col].fillna(X[col].max() * 2)
```

c) Split into calibration / validation / blind

```
from sklearn.model_selection import StratifiedShuffleSplit

# First split: 80% working set, 20% blind (never touch during dev)
split1 = StratifiedShuffleSplit(n_splits=1, test_size=0.20,
                                random_state=42)
working_idx, blind_idx = next(split1.split(X, y))

X_working, X_blind = X.iloc[working_idx], X.iloc[blind_idx]
y_working, y_blind = y.iloc[working_idx], y.iloc[blind_idx]

# Second split: 75% calibration, 25% validation (of the working set)
split2 = StratifiedShuffleSplit(n_splits=1, test_size=0.25,
                                random_state=42)
cal_idx, val_idx = next(split2.split(X_working, y_working))

X_cal, X_val = X_working.iloc[cal_idx], X_working.iloc[val_idx]
y_cal, y_val = y_working.iloc[cal_idx], y_working.iloc[val_idx]
```

You'll end up with roughly:

- **Calibration:** ~88 pairs (train thresholds + ML model here)
- **Validation:** ~30 pairs (check for overfitting)
- **Blind:** ~29 pairs (final report, never touched)

Step 2: Train the Per-Metric Thresholds (Improved)

Your current `run_threshold_tuning.py` tunes on all 147 pairs. Now you tune only on the calibration set and validate:

```
from sklearn.metrics import f1_score, precision_score, recall_score

def tune_single_metric(values, labels, metric_name):
    """Find optimal threshold for one metric using calibration data."""
    candidates = np.percentile(values.dropna(), np.arange(1, 100, 0.5))

    best = {"f1": 0}
    for t in candidates:
        for direction in ["lte", "gte"]:
            if direction == "lte":
                preds = (values <= t).astype(int)
            else:
                preds = (values >= t).astype(int)

            f1 = f1_score(labels, preds, zero_division=0)
```

```

        if f1 > best["f1"]:
            best = {
                "metric": metric_name,
                "threshold": float(t),
                "direction": direction,
                "f1": f1,
                "precision": precision_score(labels, preds,
zero_division=0),
                "recall": recall_score(labels, preds, zero_division=0),
            }
        return best

# Tune on calibration set
thresholds = {}
for col in METRIC_COLS:
    result = tune_single_metric(X_cal[col], y_cal, col)
    thresholds[col] = result

# Validate – check each threshold on the validation set
for col, t in thresholds.items():
    if t["direction"] == "lte":
        val_preds = (X_val[col] <= t["threshold"]).astype(int)
    else:
        val_preds = (X_val[col] >= t["threshold"]).astype(int)

    val_f1 = f1_score(y_val, val_preds, zero_division=0)
    t["val_f1"] = val_f1
    t["overfit_gap"] = t["f1"] - val_f1 # Big gap = overfitting

```

Red flag: If `overfit_gap > 0.15` for a metric, that threshold is unreliable and probably memorised noise in the calibration set.

Step 3: Train the ML Combined Score

This is where the real power comes in. You have three good options:

Option A: Logistic Regression (recommended to start)

Most interpretable. Coefficients directly tell you metric importance. Works well with small data.

```

from sklearn.linear_model import LogisticRegressionCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

model = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", LogisticRegressionCV(
        Cs=20, # try 20 regularisation strengths
        cv=5, # 5-fold cross-validation WITHIN
calibration set
        scoring="f1",
        penalty="l1", # L1 = automatic feature selection
        solver="saga",
        max_iter=5000,
        random_state=42,

```

```

    ))
])

model.fit(X_cal, y_cal)

# Predict probabilities
cal_probs = model.predict_proba(X_cal)[:, 1]
val_probs = model.predict_proba(X_val)[:, 1]

```

Option B: Gradient Boosted Trees (more accurate, less interpretable)

Better at capturing metric interactions. Needs more data to shine.

```

from sklearn.ensemble import GradientBoostingClassifier

model = Pipeline([
    ("clf", GradientBoostingClassifier(
        n_estimators=100,
        max_depth=3,          # shallow trees = less overfitting
        min_samples_leaf=10,  # conservative with 88 training samples
        subsample=0.8,
        learning_rate=0.05,
        random_state=42,
    ))
])

model.fit(X_cal, y_cal)

```

Option C: Both (production setup)

Train both models. If they agree → high confidence. If they disagree → flag for review.

```

prob_lr = model_lr.predict_proba(X_val)[:, 1]
prob_gb = model_gb.predict_proba(X_val)[:, 1]

# Ensemble: average the probabilities
prob_ensemble = (prob_lr + prob_gb) / 2

# Disagreement detection
disagreement = np.abs(prob_lr - prob_gb)
# High disagreement = uncertain prediction = flag for human review

```

Step 4: Evaluate on Validation Set

```

from sklearn.metrics import classification_report, roc_auc_score

val_probs = model.predict_proba(X_val)[:, 1]
val_preds = (val_probs >= 0.5).astype(int)

print("=== Validation Performance ===")
print(classification_report(y_val, val_preds, target_names=["negative",
"positive"]))
print(f"AUC-ROC: {roc_auc_score(y_val, val_probs):.3f}")

```

Compare this to your threshold-only system:

```

# Threshold-only prediction: pass only if ALL metrics pass

```

```

threshold_preds = np.ones(len(X_val))
for col, t in thresholds.items():
    if t["direction"] == "lte":
        threshold_preds *= (X_val[col] <= t["threshold"]).astype(int)
    else:
        threshold_preds *= (X_val[col] >= t["threshold"]).astype(int)

print("=== Threshold-Only Performance ===")
print(classification_report(y_val, threshold_preds,
    target_names=["negative", "positive"]))

```

You'll almost certainly see the ML model outperform the threshold-only approach, especially on precision (fewer false negatives).

Step 5: Extract Metric Importance

This is gold for your product — which metrics actually predict human judgment.

```

# For Logistic Regression:
coefs = model.named_steps["clf"].coef_[0]
importance = pd.Series(np.abs(coefs),
    index=METRIC_COLS).sort_values(ascending=False)

print("=== Top 15 Most Predictive Metrics ===")
for metric, weight in importance.head(15).items():
    sign = "+" if coefs[list(METRIC_COLS).index(metric)] > 0 else "-"
    print(f" {sign} {metric}: {weight:.3f}")

```

This tells you things like:

- voxel_iou has coefficient +2.8 → strongest positive predictor of quality
- hausdorff_99p_mm has coefficient -1.9 → strong negative predictor
- bbox_error_axis_x_mm has coefficient -0.1 → barely matters

For your product: You can rank metrics by importance and highlight the top ones in the StepScore report. Customers see what matters most.

Step 6: Calibrate the Score Output

Raw model probabilities aren't always well-calibrated (i.e., 0.7 should mean "70% chance of being good"). Fix this:

```

from sklearn.calibration import CalibratedClassifierCV

# Wrap the model with Platt scaling on the validation set
calibrated_model = CalibratedClassifierCV(
    model, method="sigmoid", cv="prefit"
)
calibrated_model.fit(X_val, y_val)

# Now probabilities are properly calibrated
calibrated_probs = calibrated_model.predict_proba(X_val)[:, 1]

```


This ensures that when StepScore says "0.82 quality", it genuinely means ~82% chance a human would accept this part.

Step 7: Final Blind Set Evaluation

Only do this once, at the very end, when you're happy with everything.

```
blind_probs = calibrated_model.predict_proba(X_blind)[: , 1]
blind_preds = (blind_probs >= 0.5).astype(int)

print("=== BLIND SET (Final Report) ===")
print(classification_report(y_blind, blind_preds, target_names=["negative",
"positive"]))
print(f"AUC-ROC: {roc_auc_score(y_blind, blind_probs):.3f}")
```

This is the number you report to customers. "StepScore achieves X% F1 on a held-out blind test set labeled by human CAD experts."

Step 8: Integrate Into Your Streamlit Platform

Add to app.py:

```
import joblib

# Load the trained model
MODEL_PATH = Path("models/stepscore_classifier.joblib")
stepscore_model = joblib.load(MODEL_PATH) if MODEL_PATH.exists() else None

# In the scoring section, after computing all 44 metrics:
if stepscore_model is not None:
    feature_vector = pd.DataFrame([metrics_dict])[METRIC_COLS]
    quality_prob = stepscore_model.predict_proba(feature_vector)[0, 1]

    # Convert to 0-100 scale
    stepscore = round(quality_prob * 100, 1)

    # Grade bands
    if stepscore >= 90: grade = "A"
    elif stepscore >= 75: grade = "B"
    elif stepscore >= 50: grade = "C"
    else: grade = "F"

    st.metric("StepScore", f"{stepscore}/100", grade)
```

Step 9: Save and Version Everything

```
import joblib

# Save model
joblib.dump(calibrated_model, "models/stepscore_classifier.joblib")

# Save metadata
metadata = {
```

```

    "training_date": "2026-03-10",
    "n_calibration": len(X_cal),
    "n_validation": len(X_val),
    "n_blind": len(X_blind),
    "val_f1": float(f1_val),
    "val_auc": float(auc_val),
    "blind_f1": float(f1_blind),
    "feature_columns": METRIC_COLS,
    "model_type": "LogisticRegressionCV_L1",
    "metric_importance": importance.to_dict(),
}
json.dump(metadata, open("models/model_metadata.json", "w"), indent=2)

```

Scaling Roadmap

Labeled Pairs	What You Can Do
147 (now)	Logistic regression with L1, 5-fold CV. Expect ~0.88 F1. Proof of concept.
300-500	Gradient boosting becomes viable. Per-family thresholds. Reliable validation set.
500-1000	Ensemble models. Confidence intervals on scores. Production-ready.
1000+	Neural approaches if needed. Per-family ML models. Statistical guarantees for customers.

Summary: What To Do Right Now

1. **Split your 147 pairs** into calibration/validation/blind
2. **Train logistic regression** on the calibration set
3. **Compare** ML score vs threshold-only on the validation set
4. **Extract metric importance** — this alone is a valuable product insight
5. **Integrate** the combined score into your Streamlit app
6. **Generate more parts** and label more pairs to scale up

Want me to build this whole pipeline as a script you can run?

What They Actually Tell You

Strip away the ML jargon. What these models do is answer one question:

"Given these 44 measurements about a generated CAD part, would a human expert accept it or reject it?"

Logistic Regression — What It Actually Learns

It learns a **weighted sum**. That's it. Nothing fancy.

Imagine after training, it produces something like this:

```
Human acceptance score =  
+4.1 × voxel_iou  
+2.3 × render_ssim  
-3.5 × chamfer_distance_mm  
-2.8 × hausdorff_95p_mm  
-0.9 × volume_diff_percent  
-0.1 × bbox_error_axis_x_mm  
+0.0 × some_irrelevant_metric  
... (all 44 metrics)  
- 1.7 (baseline bias)
```

Then it runs that sum through a sigmoid function to squash it into a 0-1 probability.

What this tells you concretely:

- **The coefficients ARE the answer.** A coefficient of +4.1 on `voxel_iou` means this is the single most important metric for predicting whether a human says "yes". A coefficient of 0.0 means that metric is useless noise — humans don't care about it when judging quality.
- **The signs tell you direction.** Positive coefficient = higher values mean better quality. Negative coefficient = higher values mean worse quality. This is obvious for something like `chamfer_distance_mm` (lower is better) but might reveal surprises — maybe a metric you assumed was important turns out to have a near-zero weight.
- **The magnitudes tell you relative importance.** `voxel_iou` at 4.1 matters roughly 40x more than `bbox_error_axis_x_mm` at 0.1. This directly tells you: if you had to pick only 5 metrics for a "lite" benchmark, which 5 would you pick? The ones with the biggest coefficients.

Real example from your data: You have 44 metrics, but probably only 8-12 of them actually matter for predicting human judgment. The rest are either redundant (measuring the same thing as another metric) or irrelevant (humans don't notice the difference). Logistic regression with L1 regularisation will literally zero out the irrelevant ones, giving you a clean ranked list of what matters.

Gradient Boosted Trees — What They Add

Logistic regression assumes the metrics combine **linearly** — each metric contributes independently. But that's not always true.

A gradient boosted tree learns **rules**. After training, if you look inside, you'd find decision paths like:

```
IF voxel_iou > 0.88
  AND chamfer_distance < 1.5
  → 94% chance human says POSITIVE

IF voxel_iou > 0.88
  AND chamfer_distance < 1.5
  BUT silhouette_iou < 0.3
  → 12% chance human says POSITIVE (rotational error!)

IF voxel_iou < 0.60
  → 3% chance human says POSITIVE (doesn't matter what else is good)
```

What this tells you that logistic regression can't:

- **Threshold effects.** Maybe `voxel_iou` above 0.85 is fine and above 0.95 is great, but the difference between 0.95 and 0.99 doesn't matter at all. Logistic regression treats every increment equally. Trees learn that there's a cliff at 0.85 and a plateau above 0.95.
- **Metric interactions.** The keyed shaft problem is a perfect example. High `voxel_iou` + low `silhouette_iou` = rotational error. Neither metric alone tells you this. Logistic regression can't learn "these two metrics together have a special meaning." Trees can, because they branch on one metric and then branch again on another.
- **Conditional importance.** Maybe `surface_area_diff_percent` only matters when the part has curved surfaces (flanges, shafts) but is irrelevant for prismatic parts (box_hole, l_bracket). A tree naturally learns this by splitting on family-related features first.

What Does the Final Score Mean for Your Customer?

When StepScore reports **0.82**, it means:

"Out of all the parts in our calibration dataset that had similar metric profiles to yours, **82% were accepted by human CAD experts** as geometrically correct."

That's it. It's a data-backed probability of human acceptance. Not an opinion. Not a heuristic. A statistical prediction grounded in real human labels.

The threshold breakdown then tells them **why** it's 0.82 and not 0.95 — "your chamfer distance is high, meaning the surface shape deviates from the reference."

So Which One Should You Use?

Start with logistic regression. With 147 samples, it's more reliable and the coefficients give you immediate product value — a ranked list of which metrics matter.

Add gradient boosting later when you have 500+ labeled pairs. With small data, trees tend to memorise rather than generalise.

The honest answer is: logistic regression will probably get you 90% of the way there. The interactions that trees capture are real, but they're second-order effects. The linear weights alone will be a massive improvement over treating all 44 metrics equally.

No — you don't pick them. That's the whole point. **The labeled data picks them for you.**

How Logistic Regression Actually Works

You give the algorithm two things:

1. Your 44 metric values for each of 147 pairs (the inputs)
2. Your human labels — positive or negative — for each pair (the answers)

The algorithm then asks: **"What set of coefficients, when applied to these 44 metrics, would best reproduce the human's judgments?"**

It does this by iterative optimisation. It starts with random coefficients, makes predictions, compares them to your labels, adjusts the coefficients to reduce errors, and repeats. Thousands of times. Until it converges on the set of coefficients that produces the fewest mistakes on your labeled data.

A Concrete Walkthrough

Say you have 3 labeled pairs (simplified to 3 metrics):

```
Pair 1: voxel_iou=0.95  chamfer=0.5    silhouette=0.91 → POSITIVE (human
said good)
Pair 2: voxel_iou=0.40  chamfer=4.2    silhouette=0.30 → NEGATIVE (human
said bad)
Pair 3: voxel_iou=0.88  chamfer=1.1    silhouette=0.12 → NEGATIVE (human
said bad - rotational error)
```

The algorithm starts with random coefficients, say $[+1.0, -1.0, +1.0]$:

Iteration 1:

```
Pair 1 score:  $1.0 \times 0.95 + (-1.0) \times 0.5 + 1.0 \times 0.91 = 1.36$  → sigmoid → 0.80 →
predict POSITIVE ✓
Pair 2 score:  $1.0 \times 0.40 + (-1.0) \times 4.2 + 1.0 \times 0.30 = -3.50$  → sigmoid → 0.03 →
predict NEGATIVE ✓
Pair 3 score:  $1.0 \times 0.88 + (-1.0) \times 1.1 + 1.0 \times 0.12 = -0.10$  → sigmoid → 0.48 →
predict POSITIVE ✗ WRONG
```

Pair 3 is wrong. The model thinks it's borderline positive, but the human said negative. So the algorithm adjusts: it needs to make `silhouette_iou` more important, because that's what's low in Pair 3 and is the distinguishing signal.

After many iterations, it converges on something like $[+2.1, -1.4, +3.8]$:

Pair 1 score: $2.1 \times 0.95 + (-1.4) \times 0.5 + 3.8 \times 0.91 = 4.76 \rightarrow \text{sigmoid} \rightarrow 0.99 \rightarrow$
 POSITIVE ✓
 Pair 2 score: $2.1 \times 0.40 + (-1.4) \times 4.2 + 3.8 \times 0.30 = -3.90 \rightarrow \text{sigmoid} \rightarrow 0.02 \rightarrow$
 NEGATIVE ✓
 Pair 3 score: $2.1 \times 0.88 + (-1.4) \times 1.1 + 3.8 \times 0.12 = 0.76 \rightarrow \text{sigmoid} \rightarrow 0.32 \rightarrow$
 NEGATIVE ✓

Now it gets all three right. And notice what the coefficients tell us: `silhouette_iou` got the biggest weight (+3.8) because it was the metric that distinguished Pair 3 (rotational error) from Pair 1 (good part). The algorithm figured this out from the data, not from anyone's opinion.

So What Decides the Coefficients?

Your human labels decide them. The coefficients are the mathematical answer to: "What weighting of these 44 metrics best predicts what the human labeled?"

- If you labeled most keyed shafts as negative because of rotational errors, the model will learn to give high weight to `silhouette_iou` and `cross_section_iou` (the metrics that catch rotational errors).
- If you labeled borderline `chamfer_distance` cases as positive (because they're close enough), the model will learn to give moderate weight to chamfer distance (it matters, but small violations are okay).
- If `bbox_error_axis_x_mm` was never the reason you rejected a part, its coefficient will converge near zero (the algorithm learns it's irrelevant to human judgment).

The coefficients are literally an encoded version of your expert judgment, extracted from your labeling patterns.

What About Regularisation?

Here's the one "arbitrary" choice — but it's not really arbitrary. With 44 metrics and only 147 samples, the algorithm could technically find coefficients that perfectly predict all 147 labels but fail on new data (overfitting). Regularisation prevents this by penalising large coefficients.

L1 regularisation (what I recommended) adds a constraint: "find good coefficients, but also keep them as small and sparse as possible." This forces the model to zero out metrics that aren't genuinely useful, leaving only the ones with real predictive signal.

The **strength** of regularisation is the one hyperparameter you choose — but even this is chosen by data, not by you. `LogisticRegressionCV` tests 20 different strengths using cross-validation and picks the one that generalises best:

```
LogisticRegressionCV(
    Cs=20,      # try 20 regularisation strengths
    cv=5,       # test each on 5 different data splits
    scoring="f1", # pick the one with best F1
)
```

So even the "tuning" of the model is automated and data-driven.

The Bottom Line

Nothing is arbitrary. The pipeline is:

1. **You** label parts as positive/negative using your expert judgment
2. **The algorithm** finds coefficients that reproduce your judgment patterns
3. **Cross-validation** picks the regularisation strength that prevents overfitting
4. **The validation set** confirms it actually works on data it hasn't seen

Your expertise enters through the labels, not through the coefficients. The math handles the rest.

What Would Make StepScore Invaluable

Let me think about this from the perspective of your customer — an AI company building a CAD generation model. What do they desperately need that doesn't exist?

1. Failure Taxonomy

Right now you say "this part failed." But AI teams need to know **how** it failed so they can fix their model. You already have the raw metrics — the missing piece is categorising failures into named types:

- **Rotational error** — correct geometry, wrong orientation (high voxel_iou + low silhouette_iou)
- **Scaling error** — right shape, wrong size (high render_ssim + high volume_diff)
- **Topological error** — wrong number of holes, missing features (genus mismatch)
- **Missing feature** — forgot the keyway, forgot a bolt hole (face count mismatch + local geometry deviation)
- **Extrusion error** — right 2D profile, wrong height or direction
- **Translation error** — right part, wrong position in space (high chamfer + low bbox_error)

These patterns already exist in your 44 metrics. You just need to codify them. An AI team can then say "our model has a 23% rotational error rate on cylindrical parts" and know exactly what to fix in their training pipeline.

Nobody offers this. Every other benchmark says pass/fail. You'd tell them the diagnosis.

2. Difficulty Grading Per Part

Not all parts are equally hard to generate. A box with a hole is trivial. A shelled enclosure with bosses and a side cutout is hard. A helical sweep is very hard.

If you grade each reference part by difficulty (based on empirical pass rates across multiple models), customers can see:

- "Our model scores 95% on Easy parts, 72% on Medium, 31% on Hard"
- "Compared to the industry average: Easy 93%, Medium 68%, Hard 28%"

This is far more useful than a single aggregate score. It tells the customer exactly where their model breaks down — and that's worth paying for.

You can compute difficulty empirically: run 3+ models on every part, and the pass rate across models IS the difficulty. A part that all models fail is "Hard." A part all models pass is "Easy."

3. Prompt Sensitivity Analysis

You already have L2 and L3 prompts. This is an underexploited goldmine.

For each part, you can report:

- **L2 score** (vague prompt) vs **L3 score** (detailed prompt)
- **Prompt sensitivity** = L3 score - L2 score

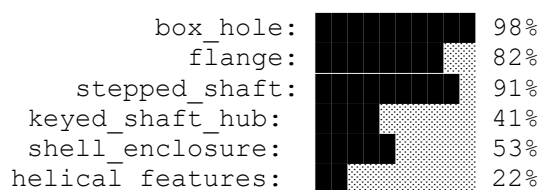
A model with high prompt sensitivity is fragile — it only works when you spell everything out. A model with low sensitivity is robust — it can interpret vague engineering descriptions.

AI companies care deeply about this because their end users won't write L3-level prompts. They'll say "make me a flange with 4 bolt holes" not "create a cylinder diameter=136 at origin (0,0,0) with bolt holes at radius=41.0535..."

This is a unique differentiator. No other benchmark tests prompt robustness for CAD.

4. Per-Family Performance Profiles

Instead of one leaderboard number, give each model a **radar chart** across families:



Model A might be great at prismatic parts but terrible at curved geometry. Model B might be the opposite. This helps customers pick the right model for their specific use case (aerospace vs consumer products vs mechanical assemblies).

5. Geometric Capability Tags

Tag each part with the geometric operations it requires:

- extrude, cut, fillet, chamfer, shell, loft, sweep, helix, boolean_union, polar_pattern, mirror, draft_angle, off_axis_workplane

Then report per-operation pass rates:

Model X capability profile:

extrude:	98%	✓
cut (hole):	94%	✓
boolean_union:	87%	✓
shell:	61%	⚠
loft:	34%	✗
helix:	12%	✗

This is incredibly actionable. An AI company immediately knows "we need more loft and helix examples in our training data." You've just told them exactly how to improve.

6. Reproducibility Guarantees

Your benchmark runs deterministically — same prompt, same reference, same metrics engine. Publish the metric engine as open source (or at least publish the metric definitions precisely). Keep the reference parts and prompts proprietary.

This means:

- Customers can verify your scores independently
- They trust the numbers because the methodology is transparent
- But they still need to buy access to the reference set and the harness

This is the model that MMLU, HumanEval, and SWE-bench use — open methodology, curated dataset.

7. Temporal Tracking

When a customer buys access, give them an API where they can submit model outputs and get scored over time. Store their historical results. Show them:

```
Version 1.0 (Jan):  StepScore 54
Version 1.1 (Feb):  StepScore 61  ↑13%
Version 1.2 (Mar):  StepScore 58  ↓5% ← regression on shell_enclosure
family
```

This makes StepScore sticky. Once they're tracking their model's progress on your benchmark, they can't leave. Every model version gets scored against the same reference set, so the numbers are directly comparable over time.

What Matters Most

If I had to rank these by impact:

1. **Failure taxonomy** — immediate product value, built from data you already have

2. **Per-family profiles + difficulty grading** — makes the benchmark actionable, not just a number
3. **Geometric capability tags** — tells customers exactly what to fix
4. **Prompt sensitivity** — unique differentiator, you already have L2/L3 infrastructure
5. **Temporal tracking** — makes the product sticky and recurring revenue
6. **Reproducibility** — builds trust, industry standard practice

The ML scoring layer we discussed earlier supports all of this but isn't the headline feature. The headline is: **"StepScore doesn't just tell you your model is bad at CAD — it tells you exactly why, exactly where, and exactly what to fix."**

That's what no other benchmark does, and that's what makes it worth paying for.

8. Multi-Model Leaderboard

This is the obvious commercial play. You run every major model through StepScore and publish a public leaderboard:

Rank	Model	StepScore	Easy	Med	Hard
1	Claude Opus 4	78.3	96%	81%	42%
2	GPT-5.2	74.1	95%	76%	38%
3	Gemini Ultra 2	71.8	93%	74%	33%
4	DeepSeek V4	68.2	91%	70%	29%
5	Llama 4 405B	52.7	84%	51%	12%

Why this matters commercially: AI companies will pay to be evaluated because their competitors are on the board. If Anthropic sees OpenAI beating them on CAD generation, they want to know why and how to improve. The leaderboard creates competitive pressure that drives demand for your detailed per-family breakdowns and failure analysis.

The public leaderboard is free marketing. The detailed diagnostic reports are the paid product.

9. Manufacturability Scoring

Right now you compare generated geometry to reference geometry. But the real question engineers care about is: **"Can I actually manufacture this part?"**

Add checks for:

- **Minimum wall thickness** — is any wall thinner than what CNC/3D printing can produce?
- **Internal angles** — are there sharp internal corners a tool can't reach?
- **Overhang detection** — would this part need support structures in additive manufacturing?
- **Draft analysis** — can this part be injection molded?
- **Watertight mesh** — is the solid manifold with no gaps or self-intersections?

This shifts StepScore from "does it match the reference" to "is this actually a useful engineering output." That's a fundamentally more valuable question. A generated part could perfectly match the reference but still be unmanufacturable if the reference itself had issues.

This also opens a second product line — scoring CAD outputs not against a reference but against manufacturing constraints. That's useful even outside the benchmarking context.

10. Code Quality Metrics

You're benchmarking CAD **code** generation, not just geometry. The generated CadQuery/OpenSCAD code itself has quality signals:

- **Code compiles and runs** — basic bar, but worth tracking as a separate metric
- **Code efficiency** — 10 lines vs 200 lines for the same part
- **Parametric flexibility** — does the code use variables or hardcoded numbers? Could you change a dimension and get a valid variant?
- **Idiomatic API usage** — does it use the right CadQuery patterns (`.faces(">Z").workplane()`) or fight the API with raw geometry hacks?
- **Determinism** — does running the code twice produce identical output?

AI companies care about code quality because their users will read and modify the generated code. A part that geometrically matches but was generated with spaghetti code is less valuable than clean, parametric code.

11. Partial Credit Scoring

Right now it's binary — pass or fail. But there's a huge difference between:

- A shaft that's 2% too short (almost perfect)
- A shaft that's missing entirely (catastrophic)

Introduce a continuous quality scale per feature:

Part: `keyed_shaft_hub_0068`

Feature breakdown:

Shaft cylinder:	98/100	(diameter off by 0.3mm)
Hub cylinder:	95/100	(length off by 0.8mm)
Keyway slot:	15/100	(wrong angular position)
Overall assembly:	69/100	

This requires decomposing the reference into individual features and scoring each one. It's harder to implement but dramatically more informative. A customer learns "my model gets basic cylinders right but struggles with feature placement" — that's a specific training signal.

12. Cross-Format Testing

CadQuery is one framework. But the industry uses many:

- **OpenSCAD** — popular in open-source
- **STEP file direct generation** — no code intermediate

- **FreeCAD macro** — Python-based, different API
- **OnShape FeatureScript** — cloud-native CAD
- **SolidWorks API** — industry standard

If your benchmark tests the same part across multiple output formats, you can measure whether a model truly understands 3D geometry or just memorised CadQuery syntax. A model that scores 90% on CadQuery but 30% on OpenSCAD doesn't understand CAD — it understands one API.

This is a longer-term play but it positions StepScore as format-agnostic, which is essential for enterprise adoption.

13. Regression Detection Service

Beyond temporal tracking, offer automated alerts:

"Your model's v2.3 update caused a **14% regression on shell_enclosure parts** and a **8% regression on fillet operations**. All other families improved or stayed flat."

AI companies push model updates constantly. Having an automated canary that catches CAD-specific regressions before they ship to production is extremely valuable. This is a monitoring product, not just a benchmark — and monitoring products have recurring revenue.

14. Training Data Recommendations

This is the premium tier. Based on a model's failure patterns, recommend what training data they need:

StepScore Training Recommendations for Model X:

CRITICAL gaps (address immediately):

- Helical/sweep geometry: 12% pass rate
 - Need 500+ training examples of helix/sweep operations
- Shell operations: 34% pass rate
 - Need examples of open-face shelling with varying wall thickness

MODERATE gaps (next priority):

- Multi-body boolean unions: 67% pass rate
 - Current training data likely has simple single-body parts only
- Off-axis workplanes: 58% pass rate
 - Add training examples with rotated coordinate frames

STRENGTHS (maintain):

- Prismatic extrusion: 96% pass rate
- Hole patterns: 93% pass rate

You're not selling them training data — you're selling them a diagnosis of what training data they need. That's consulting-level value delivered automatically by your benchmark.

15. Industry-Specific Benchmark Packs

Different industries care about different geometry:

- **Automotive:** sheet metal, stamped parts, complex surfaces
- **Aerospace:** thin-walled structures, turbine blade profiles, tight tolerances
- **Consumer electronics:** shell enclosures, snap fits, internal bosses
- **Oil & gas:** flanges, pipe fittings, valve bodies, manifolds
- **Medical devices:** organic shapes, implant geometries, micro-features

Package your reference parts into industry-specific benchmark suites. An aerospace company doesn't care about snap fits. A consumer electronics company doesn't care about pipe flanges. Targeted packages command higher prices and feel more relevant.

The Full Product Vision

FREE TIER:

- Public leaderboard (drives traffic and awareness)
- Aggregate StepScore per model

STANDARD TIER (\$):

- Per-family performance breakdown
- Difficulty-graded results
- Prompt sensitivity analysis
- Failure taxonomy per part
- Geometric capability profile

ENTERPRISE TIER (\$\$):

- API access for continuous scoring
- Regression detection alerts
- Training data recommendations
- Industry-specific benchmark packs
- Cross-format testing
- Custom reference parts from customer's own CAD library

CONSULTING TIER (\$\$\$):

- "We'll analyze your model's failures and tell you exactly how to improve"
- Powered by StepScore data + human expertise

The benchmark is the wedge. The diagnostic platform is the product. The consulting is the premium upsell.

Monday 16th March:

Issue with metrics:

1. Sampling bias (point clouds):
 - Distance metrics are computed on point samples from the surface
 - Thin ribs, sharp edges, small holes and fillets get fewer points than big flat faces
 - If a feature has few points, the nearest-neighbour distances grow even when the geometry is almost correct.
 - Result is that “chamfer, hausdorff, point_to_surface” look worse for small feature parts.
2. Tesselation bias (STEP to STL):
 - STEP is exact, STL is approximate
 - Curved/lofted parts become triangle meshes with varying density
 - Two identical parts can have different triangulation if they were generated differently
 - This changes point distribution, normals and edge locations
 - Result: curved families score worse than prismatic parts
3. Alignment sensitivity:
 - If a part is symmetric, multiple orientations are equivalent
 - ICP can converge to the wrong rotational alignment or a mirrored alignment
 - Even a small alignment drift makes distance metrics blow up
 - Result: symmetric parts or off-axis parts get penalised more
4. Voxel granularity:
 - Voxel metrics depend on grid resolution
 - Thin walls can disappear if the pitch is larger than the wall thickness
 - That drives voxel_iou, occupancy_precision/recall, cross_section_iou down
 - Result: shells, ribs and thin plates score worse unless you reduce pitch
5. Metric coverage mismatch:
 - Render metrics are visual proxies
 - Complex curves and fine detail change outlines more than blocky parts
 - A small geometric error can cause a big silhouette loss for curved shapes
 - Result: curved or “visually complex” parts score worse

To make this fair:

1. Family-stratified calibration:
 - Pick families: prismatic, rotational, shell, loft/sweep, off-axis, gear
 - For distance/voxel/normal metrics, set thresholds per family
 - Keep global thresholds for topology and bbox/volume
2. Per-family metric reports:
 - For each family, compute median + 95th percentile for each metric
 - This tells us which metrics are biased by shape type

Way to make scoring fair without weakening the benchmark:

- Apply family-specific thresholds for metrics that are known to be sensitive (distance/voxel/normal metrics).
- Avoid punishing shells/lofts/gears with thresholds calibrated on blocks

How to do it safely:

- Use family only for threshold overrides, not for changing metric definitions
- Keep a global score alongside family score to prevent gaming
- Require a family selection in the UI, also mark unknown if unsure
- Possibly AI linter which picks the family based on the prompt

Minimal UI change:

- Add a dropdown for family
- use it to select threshold_profile for that run

The thresholds are tuned the same way global thresholds are tuned but done per family using labelled examples from that family only:

How thresholds differ:

- For prismatic parts: can keep tight distance/voxel thresholds because geometry is stable
- For shells/lofts/gears: you loosen distance/normal/voxel thresholds because tessellation + sampling noise is higher
- Topology + bbox/volume thresholds stay mostly unchanged across families

How to tune per family (simple, repeatable):

1. Collect labelled pairs per family.
 - Example: 30-50 usable + 30-50 not usable for each family
2. For each metric, compute distributions for usable vs not-usable
3. Pick a threshold that separates the two groups. Default: threshold at the point that maximises F1 or minimises false-pass
4. Save these to a thresholds_by_family.json

What changes in practice:

- chamfer_distance_mm might be 0.8 for prismatic, 1.5 for shells, 2.0 for loft/sweep
- voxel_iou might be 0.90 for prismatic, 0.80 for shells (thin walls).
- normal_angle_error_deg might be 8 for prismatic, 15 for lofts

